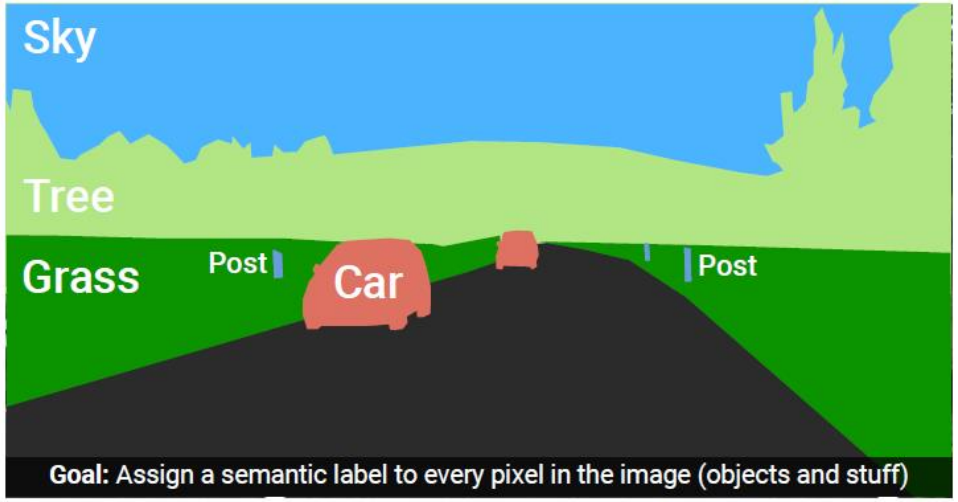
Image Classification

Computer Vision

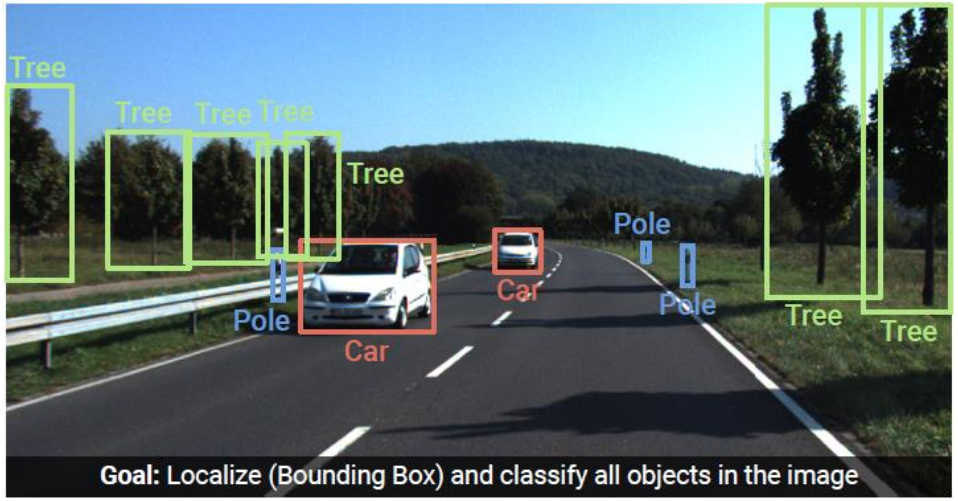
Image Understanding (Recognition)



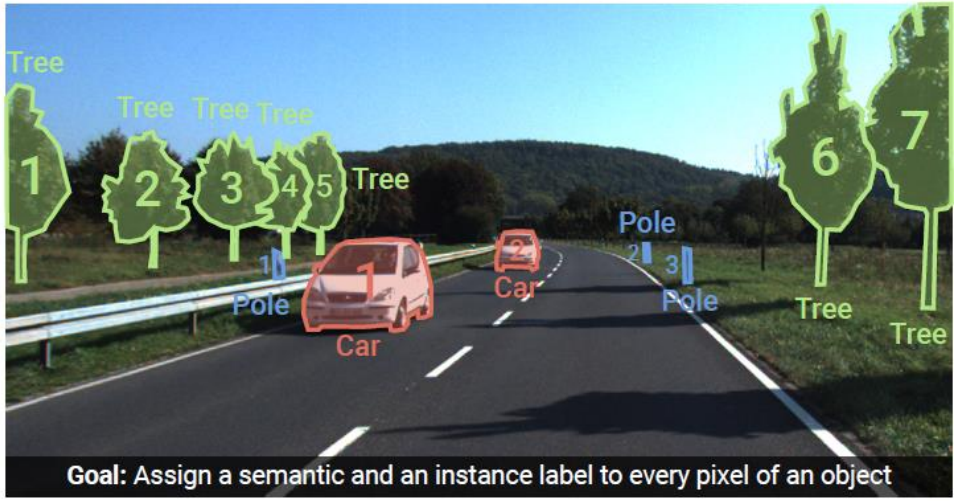
Image Classification



Semantic Segmentation



Object Detection



Instance Segmentation

Need for ML

prime example (and also foundation for detection and segmentation):

image classification (whole-image class recognition) according to generic object categories (e.g., cat)



plain keypoint-feature matching only really works for specific instances of a class

→ need to compare with generic objects (e.g., kind of “abstract cat”)

Let’s learn it from data …



What if you haven’t seen this very cat before?

What Is Machine Learning?

learning from experience/data: exploiting statistical dependencies with the aim of **generalization** to new data

training: **ML algorithm + data = explicit algorithm** (to be used later)

→ reduction of complexity and much better generalizability compared to handcrafted algorithms

analogy: Humans do not hit the ground running (storage capacity of DNA limited) but have learning capabilities.

Main Areas of AI/ML

empowered by one key component: learning from data (ML)

tabular data

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	...	PoolArea	PoolQC	Fence	MiscFeature	MiscVal	MoSold	YrSold	SaleType	SaleCondition	SalePrice
0	1	60	RL	65.0	8450	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	2	2008	WD	Normal	208500
1	2	20	RL	80.0	9600	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	5	2007	WD	Normal	181500
2	3	60	RL	68.0	11250	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	9	2008	WD	Normal	223500
3	4	70	RL	60.0	9550	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	2	2006	WD	Abnorml	140000
4	5	60	RL	84.0	14260	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	12	2008	WD	Normal	250000
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1455	1456	60	RL	62.0	7917	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	8	2007	WD	Normal	175000
1456	1457	20	RL	85.0	13175	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	MnPrv	NaN	0	2	2010	WD	Normal	210000
1457	1458	70	RL	66.0	9042	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	GdPrv	Shed	2500	5	2010	WD	Normal	266500
1458	1459	20	RL	68.0	9717	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	4	2010	WD	Normal	142125
1459	1460	20	RL	75.0	9937	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	6	2008	WD	Normal	147500

[1460 rows x 81 columns]

language models

The Lord of the Rings

Article Talk

From Wikipedia, the free encyclopedia

(Redirected from Lord of the rings)

This article is about the book. For other uses, see The Lord of the Rings (disambiguation). "War of the Ring" redirects here. For other uses, see War of the Ring (disambiguation).

The Lord of the Rings is an epic[1] high fantasy novel[2] by the English author and scholar J. R. R. Tolkien. Set in Middle-earth, the story began as a sequel to Tolkien's 1937 children's book *The Hobbit*, but eventually developed into a much larger work. Written in stages between 1937 and 1949, *The Lord of the Rings* is one of the best-selling books ever written, with over 150 million copies sold.[2]

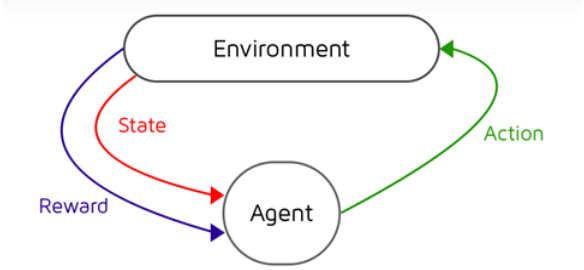
The title refers to the story's main antagonist,[3] Sauron, the Dark Lord who in an earlier age created the One Ring to rule the other Rings of Power given to Men, Dwarves, and Elves, in his campaign to conquer all of Middle-earth. From homely beginnings in the Shire, a hobbit land reminiscent of the English countryside, the story ranges across Middle-earth, following the quest to destroy the One Ring, seen mainly through the eyes of the hobbits Frodo, Sam, Merry, and Pippin. Aiding Frodo are the Wizard Gandalf, the Men Aragorn and Boromir, the Elf Legolas, and the Dwarf Gimli, who unite in order to rally the Free Peoples of Middle-earth against Sauron's armies and give Frodo a chance to destroy the One Ring in the fire of Mount Doom.

Although often mistakenly called a trilogy, the work was intended by Tolkien to be one volume in a two-volume set along with *The Silmarillion*.[3][T 3] For economic reasons, *The Lord of the Rings* was first published over the course of a year from 29 July 1954 to 20 October 1955 in three volumes rather than one[3][4] under the titles *The Fellowship of the Ring*, *The Two Towers*, and *The Return of the King*; *The Silmarillion* appeared only after the author's death. The work is divided internally into six books, two per volume, with several appendices of background material.[5] These three volumes were later published as a boxed set, and even finally as a single volume, following the author's original intent.

computer vision



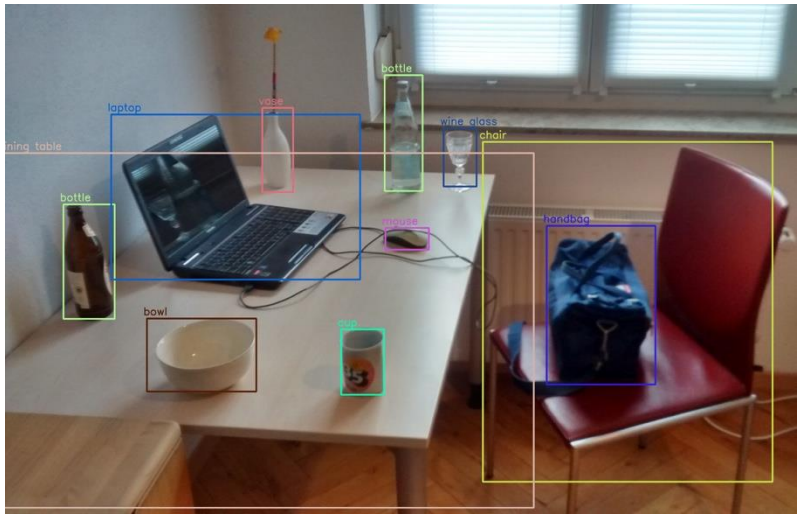
control



When to Use ML (Learning from Data)

automation

too complex for rules



object recognition, chat bot, …

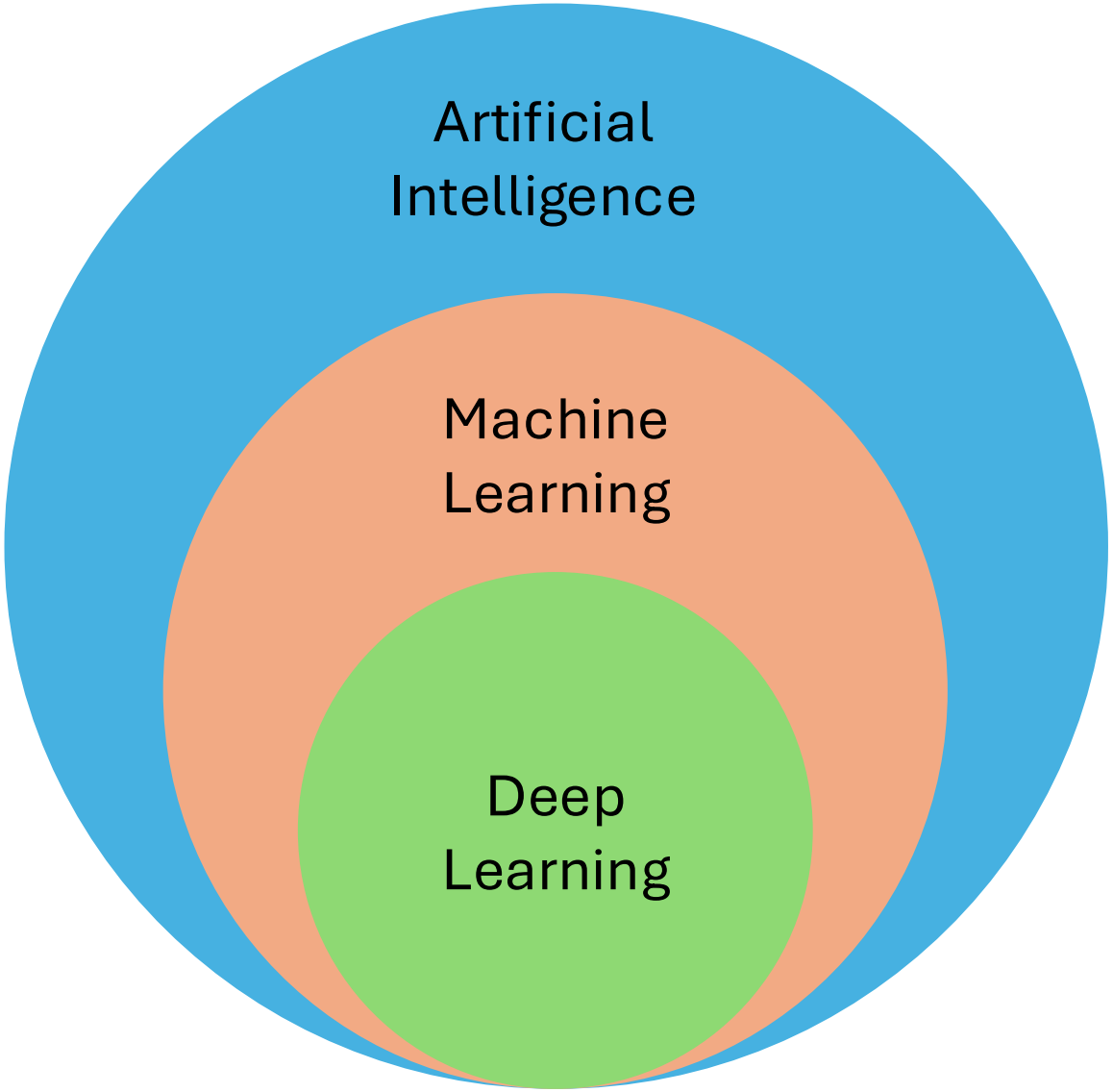
uncertainty

too complex for humans



AlphaFold

protein structure predictions, demand forecasting, …



blend of diverse components from different domains (statistics, optimization, computer science, …)

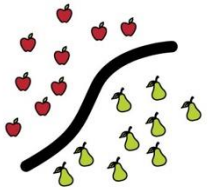
Deep Learning: special kind of ML methods using *deep* neural networks (e.g., CNNs, transformers)

MACHINE LEARNING

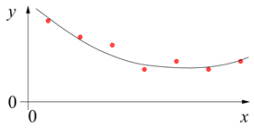
training target available (labeled or past data)

SUPERVISED

CLASSIFICATION



REGRESSION



data not labeled in any way

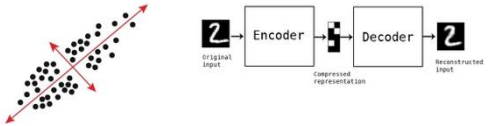
UNSUPERVISED

CLUSTERING



SELF-SUPERVISED

DIMENSIONALITY REDUCTION



ASSOCIATION



no supervision, but goal-based interaction with environment

REINFORCEMENT LEARNING

LEARN STATE OR ACTION VALUES

LEARN POLICY DIRECTLY



learning by teacher (high-dimensional curve fitting)

learning by observation (pattern recognition)

learning by trial-and-error (sequential decision making)

Supervised Learning

Supervised Learning

Target Quantity

- **known in training**: labeled samples or observations from past
- to be **predicted** for unknown cases (e.g., future values)

Features

(prepared) input information that is
- correlated to target quantity
- known at prediction time

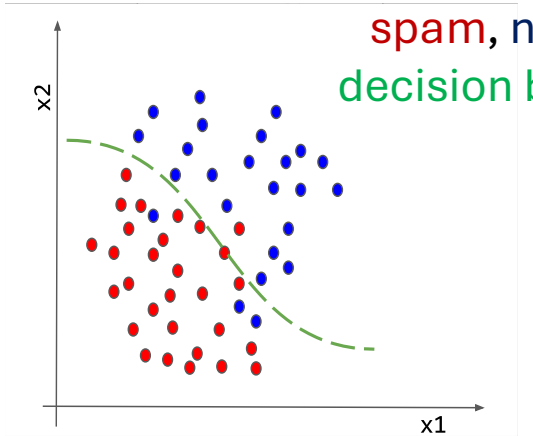


Example: Spam Filtering

classify emails as spam or no spam

*use accordingly **labeled emails as training set***

use information like occurrence of specific words or email length as **features**

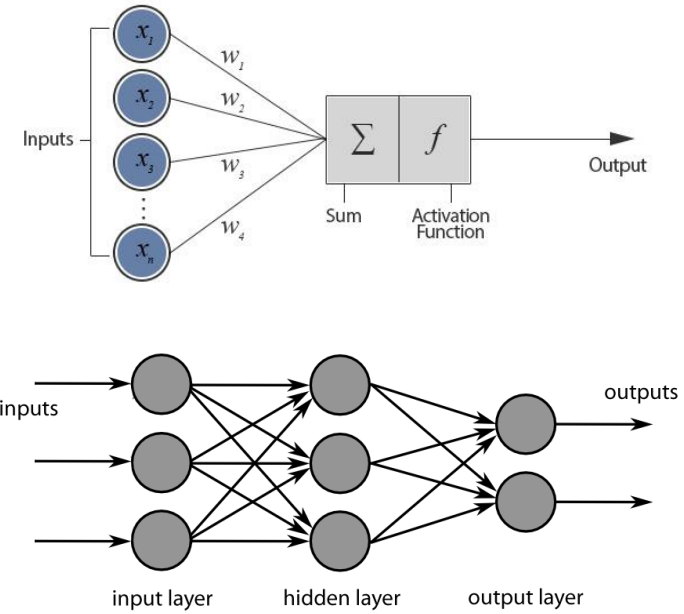


Algorithmic Families of Supervised Learning

parametric models

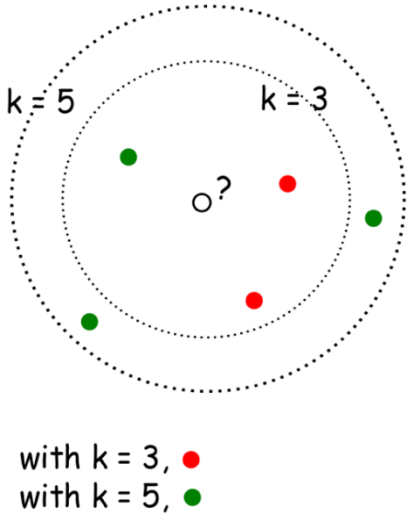
linear models

neural networks



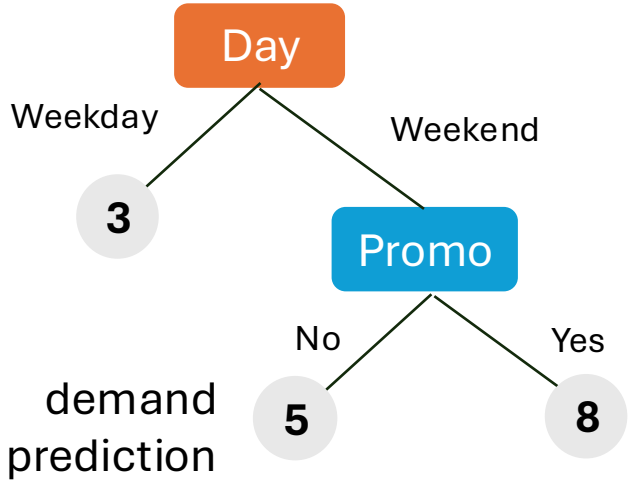
non-parametric models: similarity-based learning (analogies)

nearest-neighbor-style/local methods



kernel machines (e.g., support-vector machines)

decision trees: rule learning



often used in ensemble methods

- bagging: random forests
- boosting: gradient boosting

Supervised Learning in Mathematical Terms

map input to output: $y = f(\boldsymbol{x})$

usually high-dimensional

training (e.g., curve fitting / parameter estimation):

fit data set of $(y_i, \boldsymbol{x}_i)$ pairs → minimize deviations between y and $f(\boldsymbol{x})$

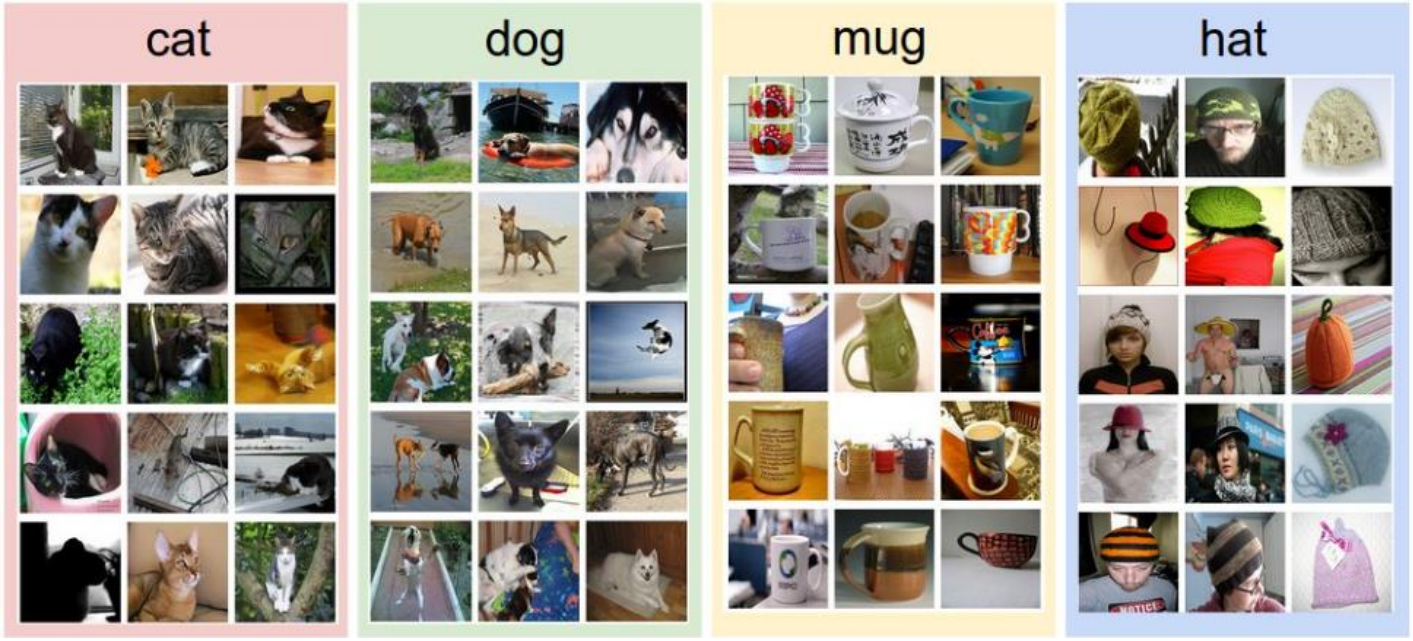
discriminative models: predict conditional distributions $p(y|\boldsymbol{x})$

as opposed to generative models: predict $p(y, \boldsymbol{x})$ (or just $p(\boldsymbol{x})$) → $\boldsymbol{x}$ not given, more difficult

Example: Image Classification

training data set:

labels:



test data with learned classifier:

$f($  $) =$ “Cat”

$f($  $) =$ “Dog”

Supervised Learning Objective

goal: find a function f that best predicts y from $\boldsymbol{x}$

in parametric models:

1. select a model class (e.g., linear, quadratic, neural network)
2. estimate parameters within that class → training

parameter estimation = finding the best parameters for a fixed function form

(subtask within supervised learning, not the full objective)

Training Recipe

combine:

- **model** (e.g., linear function & Gaussian distribution)
- **objective function** (e.g., squared residuals)
- **optimization algorithm** (e.g., gradient descent)
- **regularization** (e.g., L2, dropout)

Linear Regression

model (for each observation i):

$$y_i = \underbrace{\boldsymbol{w} \cdot \boldsymbol{x}_i + b}_{f(\boldsymbol{x}_i; \boldsymbol{w}, b)} + \varepsilon_i$$

typically several variables $\boldsymbol{X}$
→ also several parameters $\boldsymbol{w}$

predictive parameters $\boldsymbol{w}, b$
(σ^2 nuisance parameter)

assumption:

$$\varepsilon \sim \mathcal{N}(0, \sigma^2)$$

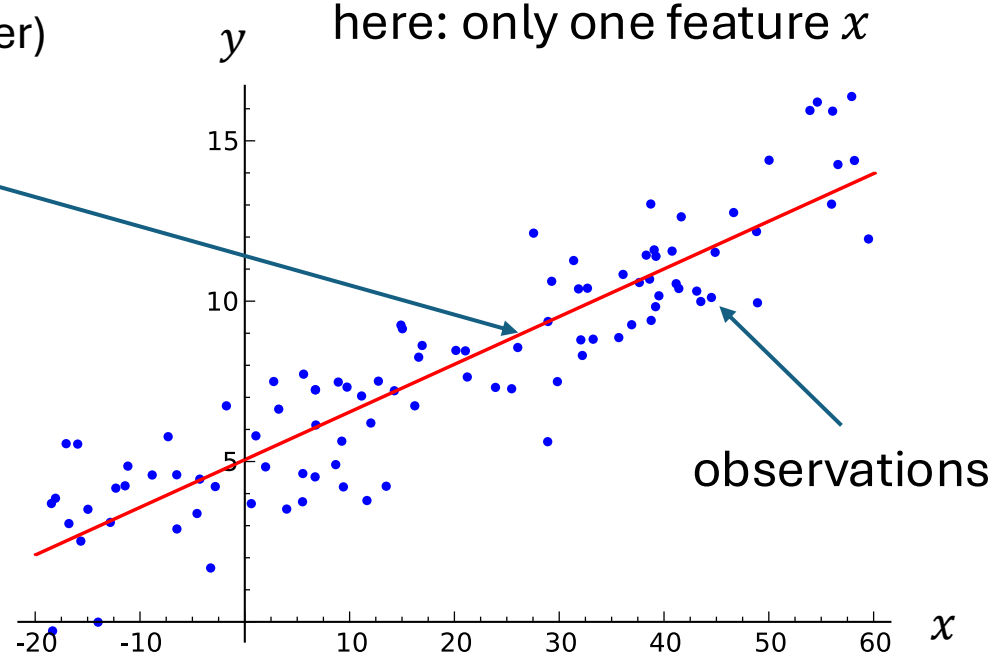
Gaussian mean variance

$$\rightarrow p(y|\boldsymbol{x}; w, b, \sigma^2) = \mathcal{N}(\boldsymbol{w} \cdot \boldsymbol{x} + b, \sigma^2)$$

different $p(y|\boldsymbol{x})$ for each $\boldsymbol{x}$,
sharing parameters $\underbrace{w, b, \sigma^2}_{\boldsymbol{\theta}}$

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^{n} \left(y_i - f(\boldsymbol{x}_i; \boldsymbol{w}, b)\right)^2$$

here: only one feature x



following Gaussian distribution

data: vector with p different feature values for this example i

$$\mathbb{E}[Y|\boldsymbol{X} = \boldsymbol{x}_i] = f(\boldsymbol{x}_i) = \boldsymbol{w} \cdot \boldsymbol{x}_i + b$$

vector with slope parameters (one for each of the p features, but identical for all examples), aka weights

single intercept parameter, aka bias

training → parameter estimation

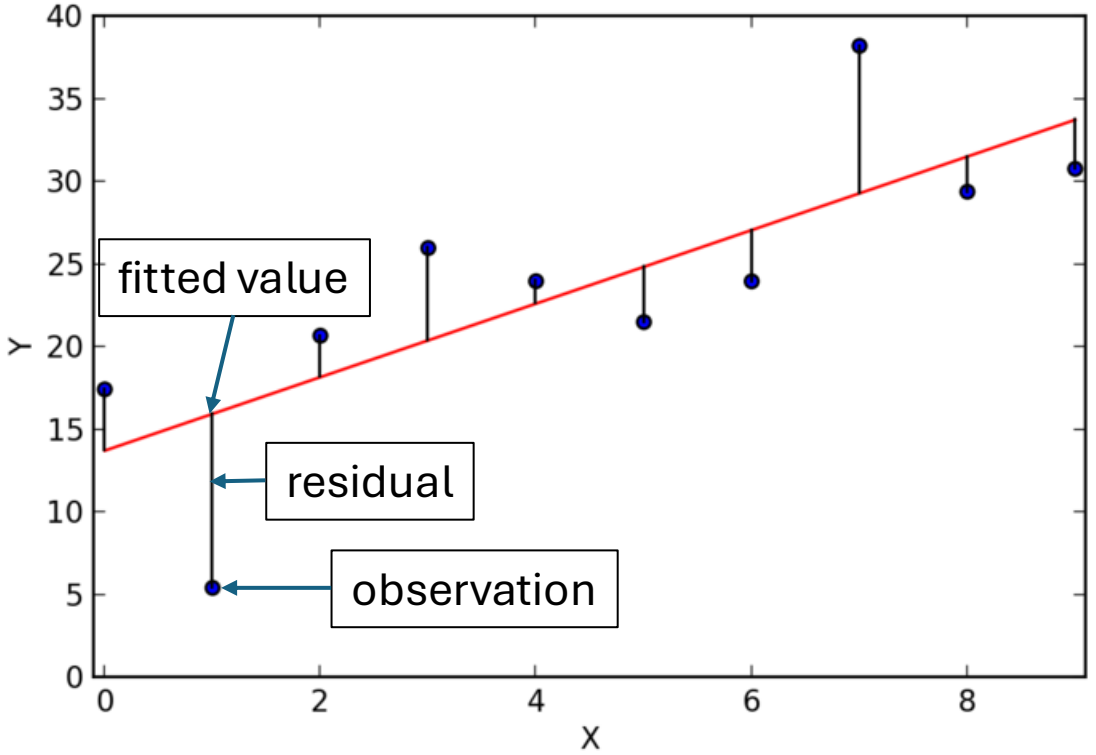
Least Squares

idea: choose parameters $\boldsymbol{\theta}$ ($\boldsymbol{w}, b$ in linear regression) that minimize sum of squared residuals r_i

$$\underset{\boldsymbol{\theta}}{\operatorname{argmin}} \sum_{i=1}^{n} r_i(\boldsymbol{\theta})^2$$

examples for residuals:

- mean estimation: $r_i(\mu) = x_i - \mu$
- regression: $r_i(\boldsymbol{\theta}) = y_i - f(\boldsymbol{x}_i; \boldsymbol{\theta})$



Least Squares Estimation: Minimization

minimization of objective

$$J(\boldsymbol{\theta}) = \sum_{i=1}^{n} r_i(\boldsymbol{\theta})^2$$

candidate solution: need to ensure it is a minimum

first-order optimality condition: set gradient of objective to zero

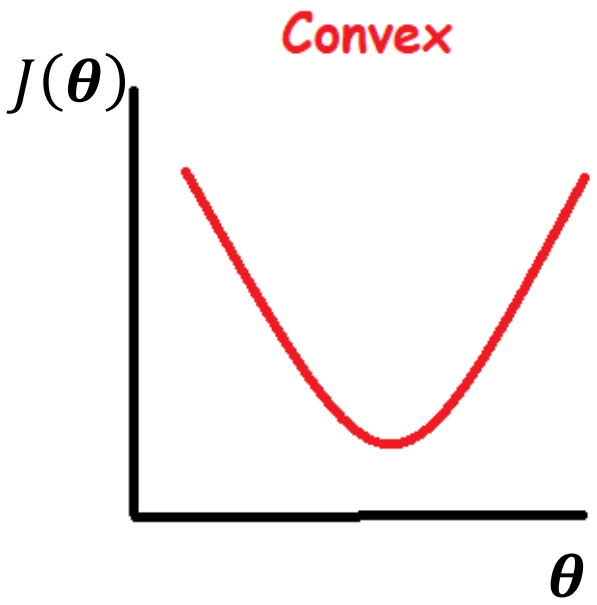
$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \sum_{i=1}^{n} 2 r_i(\boldsymbol{\theta}) \nabla_{\boldsymbol{\theta}} r_i(\boldsymbol{\theta}) = 0$$

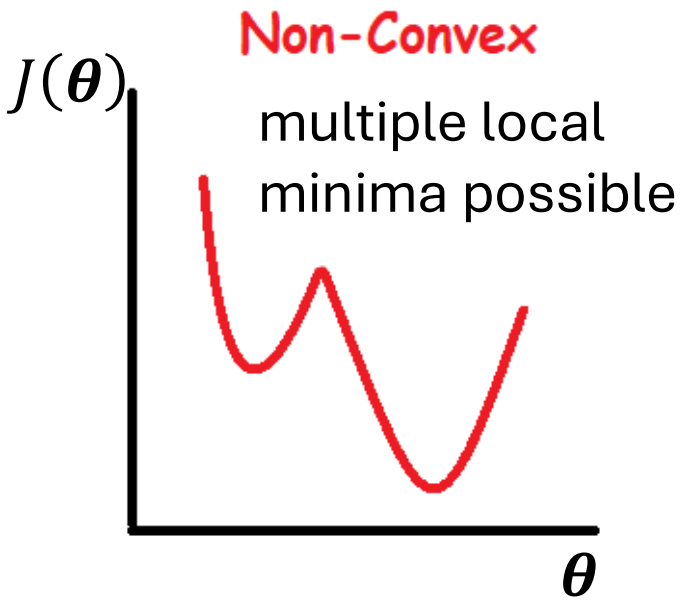
$$\nabla_{\boldsymbol{\theta}} = \begin{bmatrix} \frac{\partial}{\partial \theta_1} \\ \frac{\partial}{\partial \theta_2} \\ \vdots \end{bmatrix}$$

General Nonlinear Case

$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = 0$ gives stationary points

For convex problems (like linear least squares), this is the global minimum.





if $f(\boldsymbol{x}; \boldsymbol{\theta})$ nonlinear in $\boldsymbol{\theta}$: typically no closed-form solution to $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = 0$

→iterative numerical optimization

Gradient Descent

iteratively moving in direction of steepest descent (negative gradient):

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \cdot \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$$

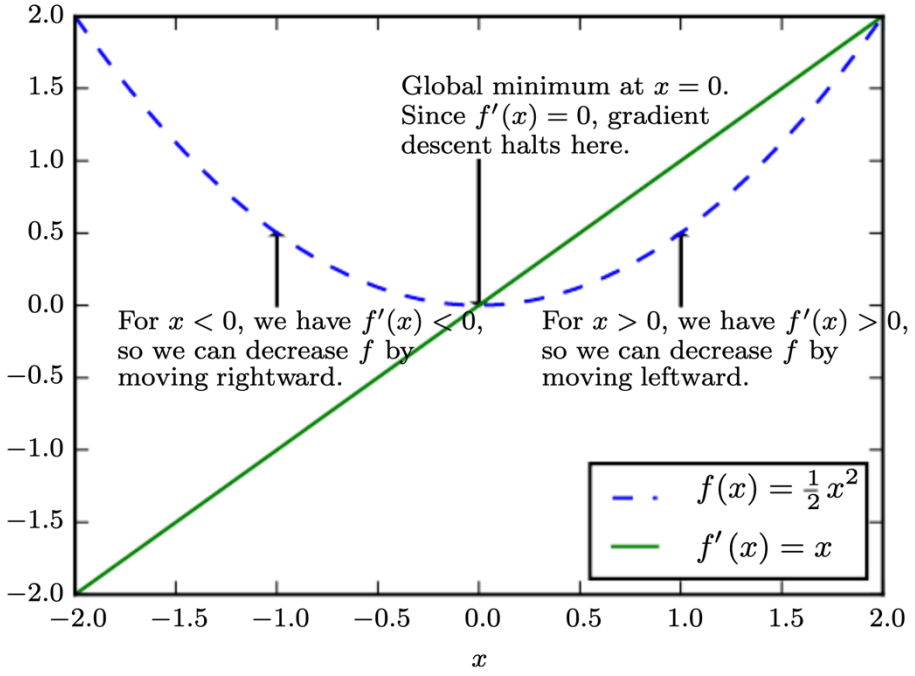
first-order

step size (learning rate): hyperparameter controlling how aggressively to move downhill

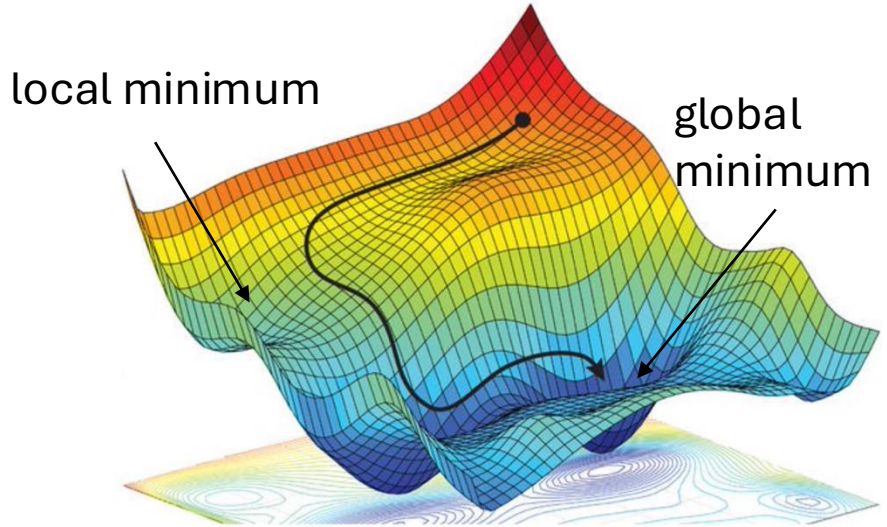
repeatedly take small steps downhill on the loss surface until a minimum is reached

typically use automatic differentiation (autodiff) to calculate gradients (systematic application of chain rule on computation graph)

called backpropagation in neural networks



source



Loss Function

least squares: minimizes sum of squared residuals between predictions and targets (independent of the model used, linear model → linear least squares)

loss functions generalize squared residuals to arbitrary prediction errors (still independent of the model used): $L_i(\boldsymbol{\theta}) = L\big(y_i, f(\boldsymbol{x}_i; \boldsymbol{\theta})\big)$

$L_i(\boldsymbol{\theta})$ quantifies prediction error (e.g., residual) on one sample

further examples:

- absolute error
- cross-entropy (classification)

Cost Function

aggregate (average) losses over data set:

$$J(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n} L_i(\boldsymbol{\theta})$$

to be minimized according to model parameters $\boldsymbol{\theta}$

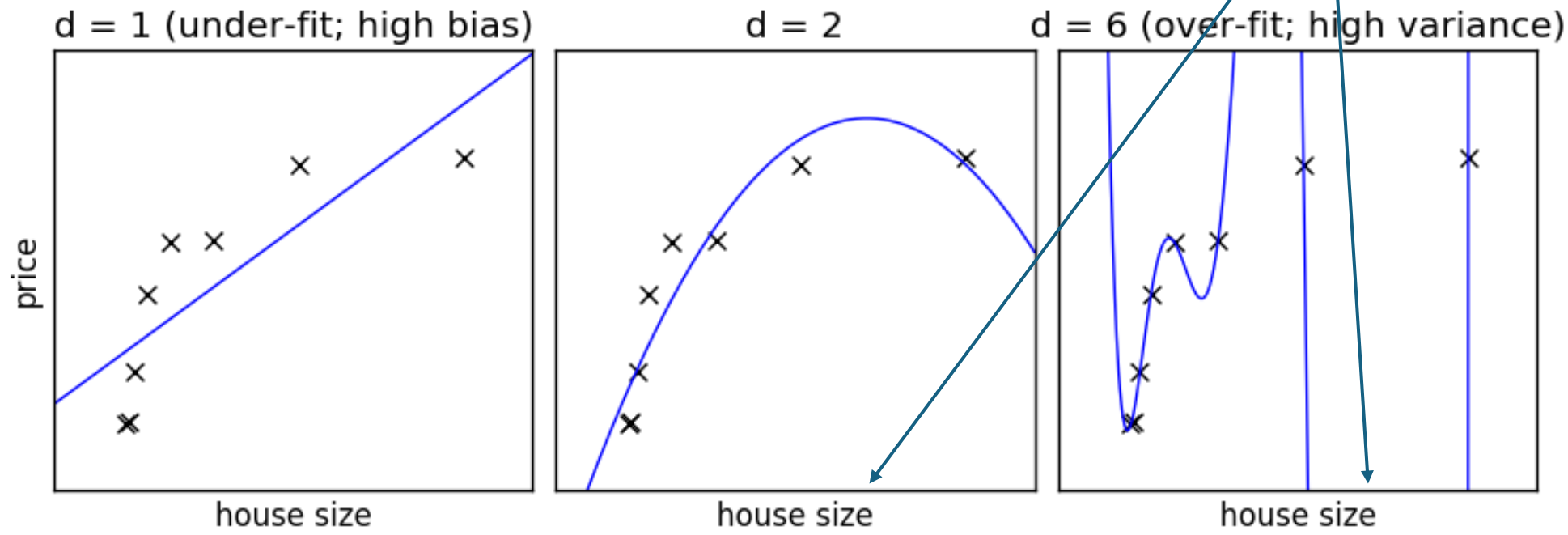
→ objective function

Model Selection: Under- and Over-Fitting

example: polynomial fit

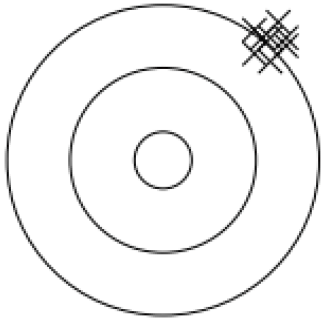
need to predict new samples (not in training data set): interpolation

degree of fitted polynomial

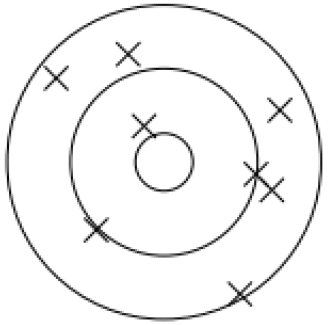


Bias, Variance, Irreducible Error

think of fitting ML algorithms as repeatable processes with different data sets



bias
due to too simplistic model
(same for all training data sets)
"underfitting"



variance
due to sensitivity to specifics (noise)
of different training data sets
"overfitting"

irreducible error (aka Bayes error): inherent randomness

reason: target generated from random variable $Y|\boldsymbol{X} = \boldsymbol{x} \sim p(y|\boldsymbol{x}; \boldsymbol{\theta})$

→ limiting accuracy of ideal model

e.g., Gaussian for regression or
Bernoulli for classifiction
if almost deterministic (i.e., narrow distribution) → low irreducible error

Bias-Variance Tradeoff

models of higher complexity have lower bias but higher variance

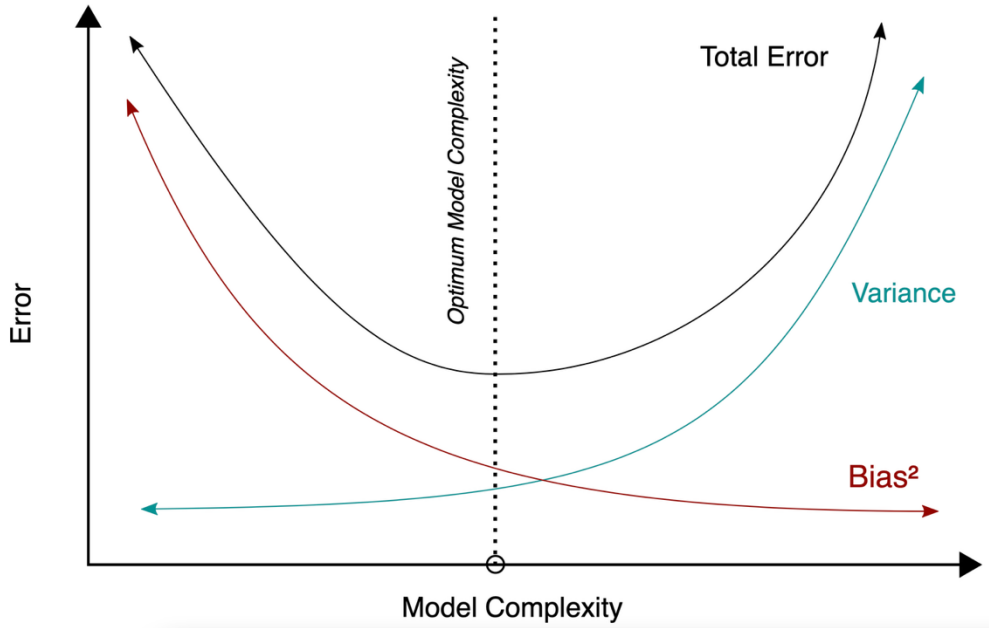
(given the same number of training examples)

generalization error follows U-shaped curve: overfitting once model complexity (number of parameters) passes certain threshold

overfitting: variance term dominating test error

→ increasing model complexity increases test error

but beware: training error keeps decreasing with more complex model …



Cross-Validation (CV)

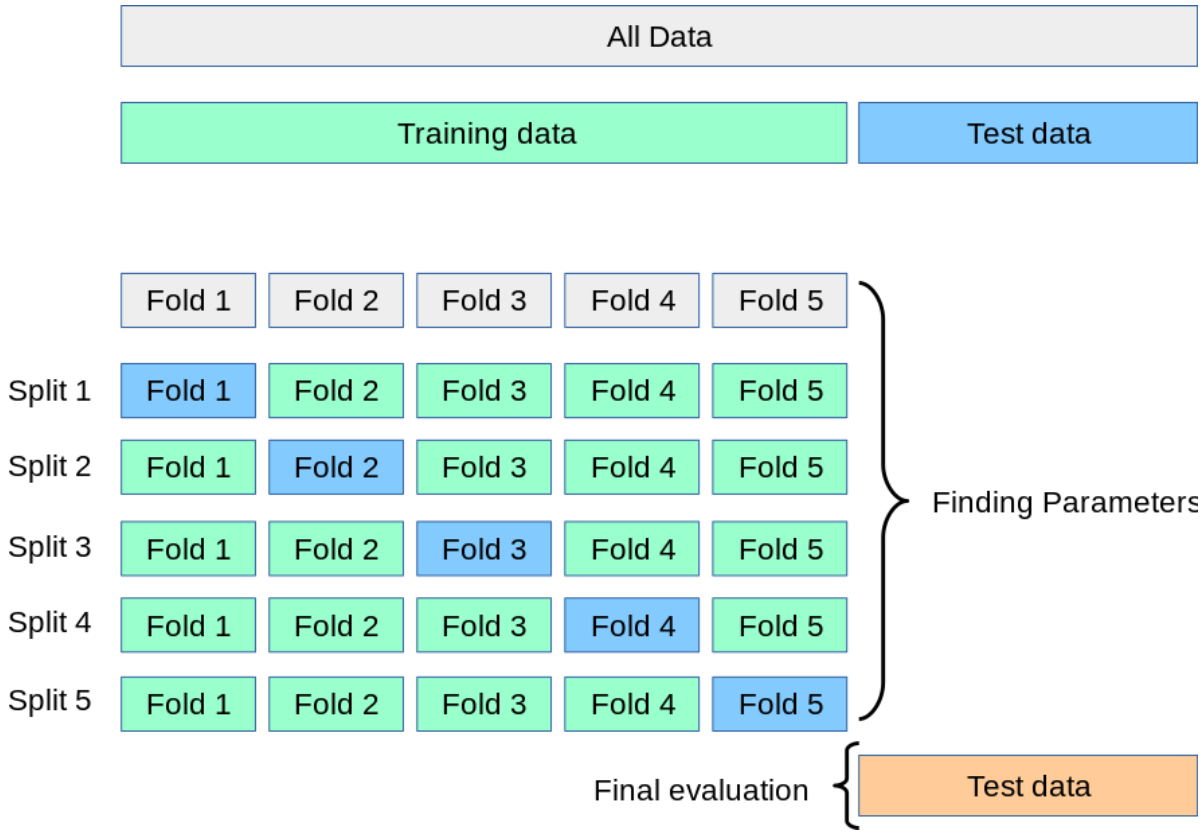
goal: evaluate model performance on unseen data → detect overfitting

basic idea:

- split data into training and validation parts multiple times
- for each split, train on training part and test on validation part
- average test performance across splits

bias-variance trade-off in choosing k

k-fold CV:

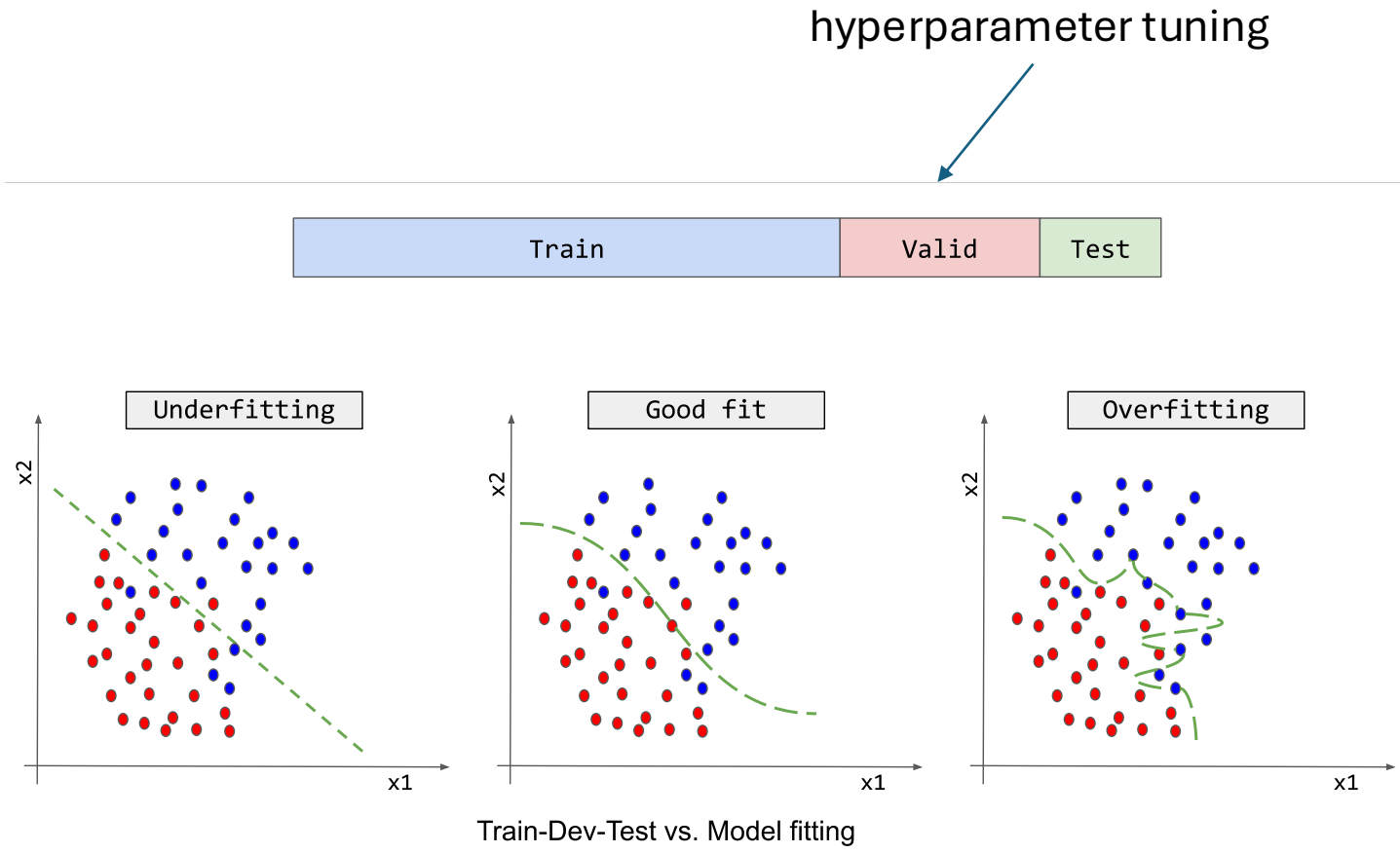


Hyperparameters

model complexity often controlled via hyperparameters (parameters not fitted but set in advance)

examples:

- degree of fitted polynomial d
- number of considered nearest neighbors k
- maximum depth of decision tree
- number of hidden nodes in neural network layer



Train-Dev-Test vs. Model fitting

Generalization

core of ML: **empirical risk minimization** (training error) as proxy for minimizing unknown population risk (test error)

generalization gap: difference between test and training error

curse of dimensionality: typically, many features (dimensions)

→ lots of data needed to densely sample volume

Curse of Dimensionality

data sparsity:

dimensions (i.e., features) increase

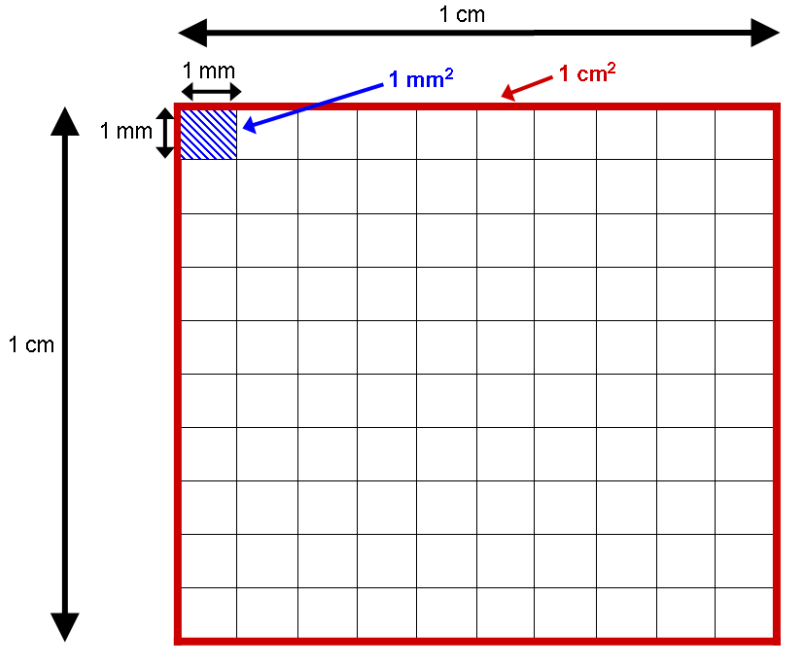
→ volume of space increases rapidly

→ data points become isolated

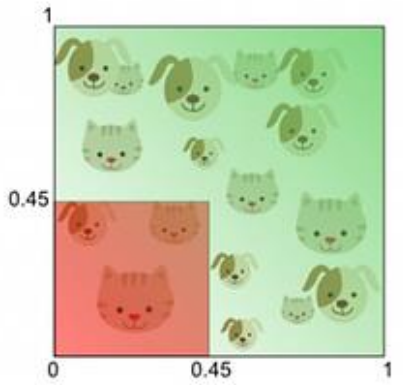
→ difficult to find meaningful patterns (concept of proximity breaks down)

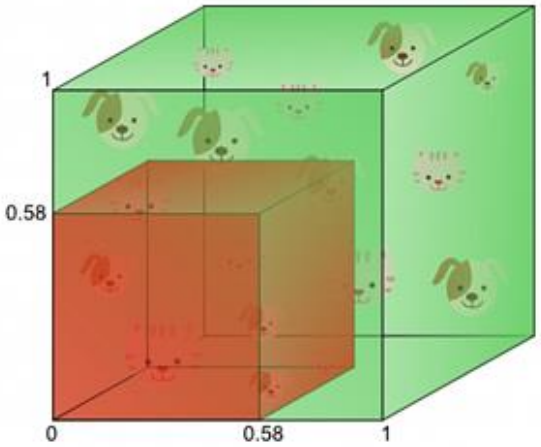
→ overfitting

→ exponential data (i.e., samples) requirements



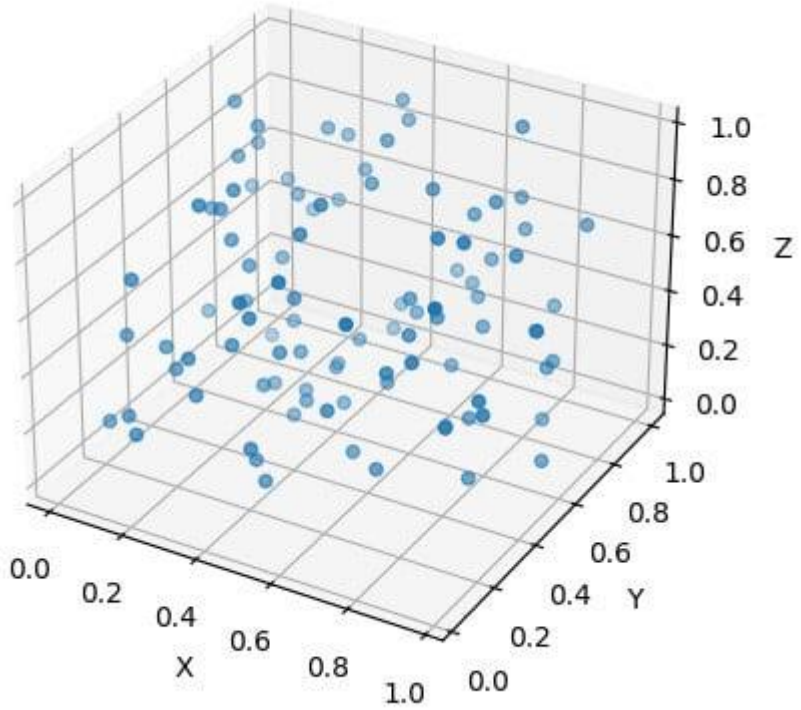


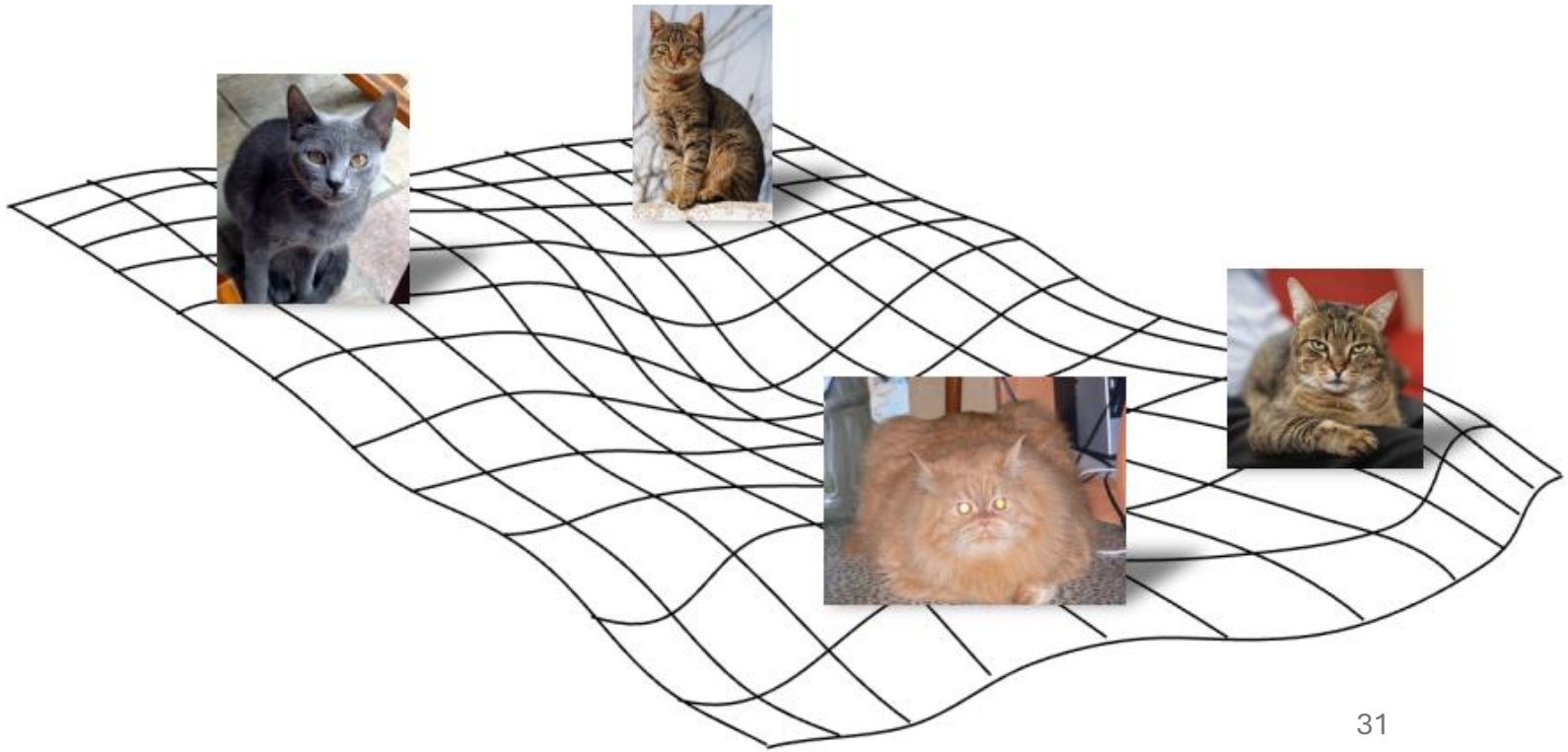




Manifold Hypothesis

luckily, reality is friendly: most high-dimensional data sets reside on lower-dimensional manifolds → enabling effectiveness of ML





Classification

Classification: Logistic Regression

binary classification: $y = 1$ or $y = 0$

→ predict probabilities p_i for $y = 1$

- logit (log-odds) as link function
- Y following Bernoulli distribution

still Gaussian linear model

$$\text{logit}(\mathbb{E}[Y|\boldsymbol{X} = \boldsymbol{x}_i]) = \ln\left(\frac{p_i}{1-p_i}\right) = \boldsymbol{w} \cdot \boldsymbol{x}_i + b$$

$$p_i = \frac{1}{1 + e^{-(\boldsymbol{w} \cdot \boldsymbol{x}_i + b)}}$$

Multi-Classification: Softmax

for classification in K categories:

multinomial logistic regression

- use "score" function with K outputs

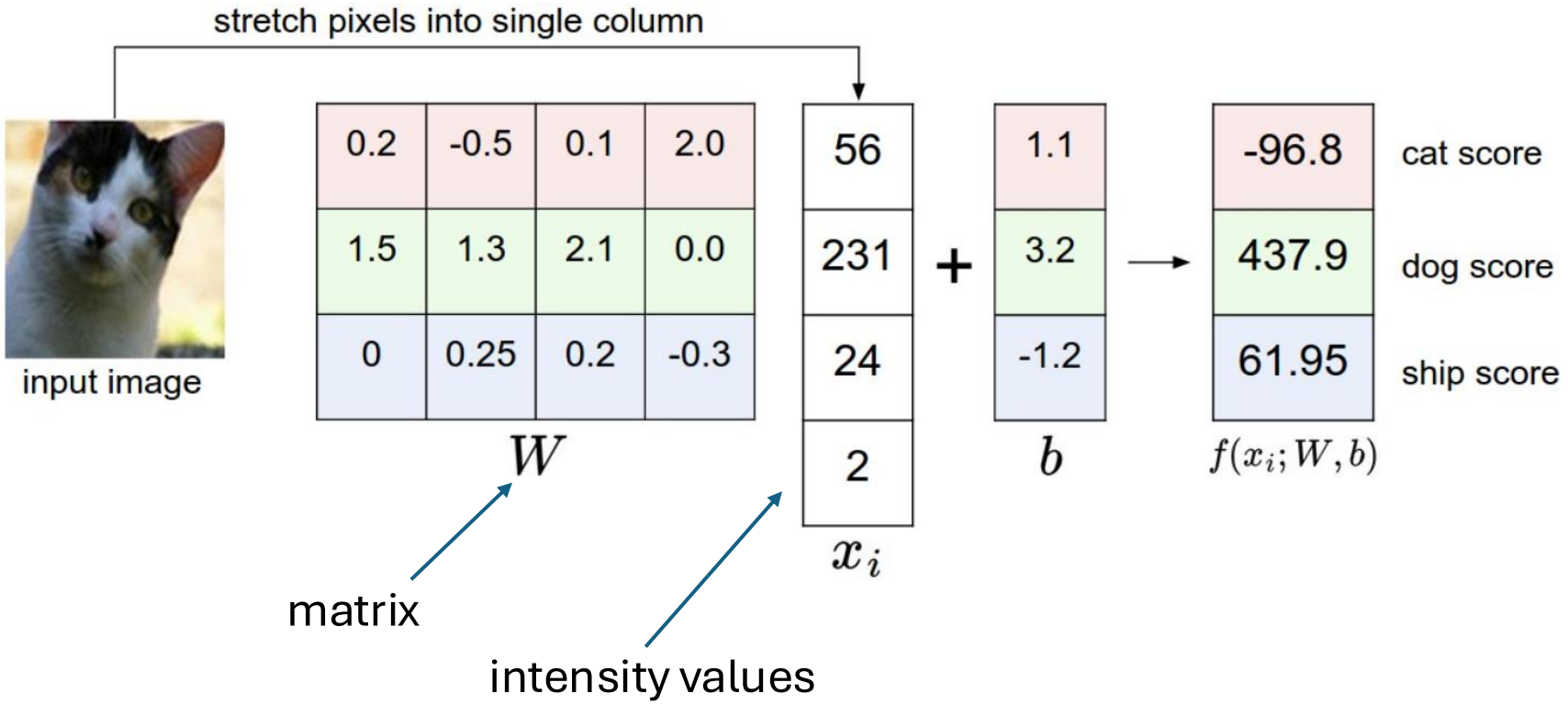
$$f_k(\boldsymbol{x}_i) = \boldsymbol{w}_k \cdot \boldsymbol{x}_i + b_k$$

- and convert into probabilities by means of softmax function

$$\frac{\exp\left(f_k(\boldsymbol{x}_i)\right)}{\sum_{l=1}^{K} \exp\left(f_l(\boldsymbol{x}_i)\right)}$$

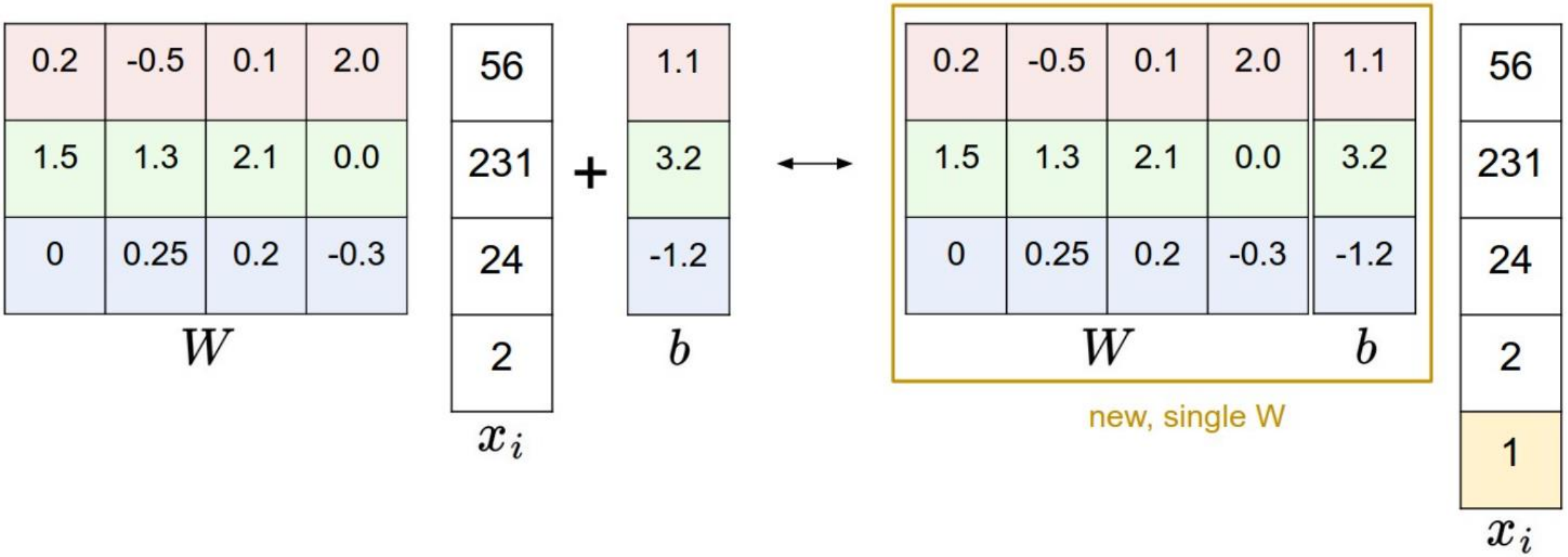
Image Classification with Linear Regression

simplified example: 4-pixel image, 3 classes

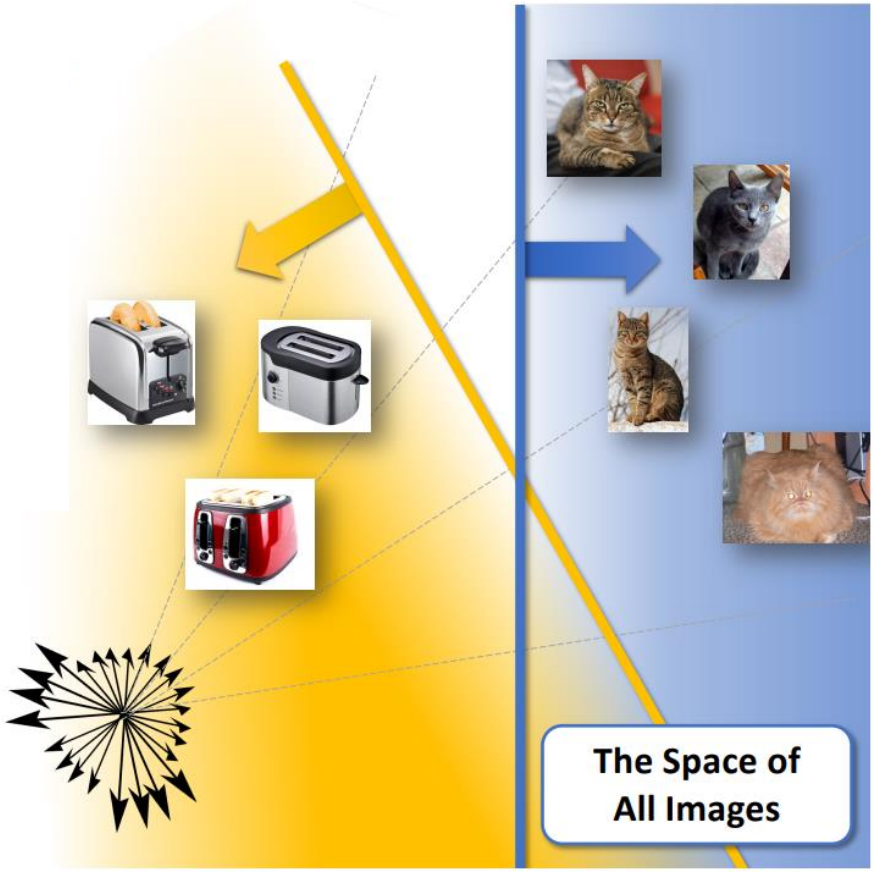


need for one common image size per model

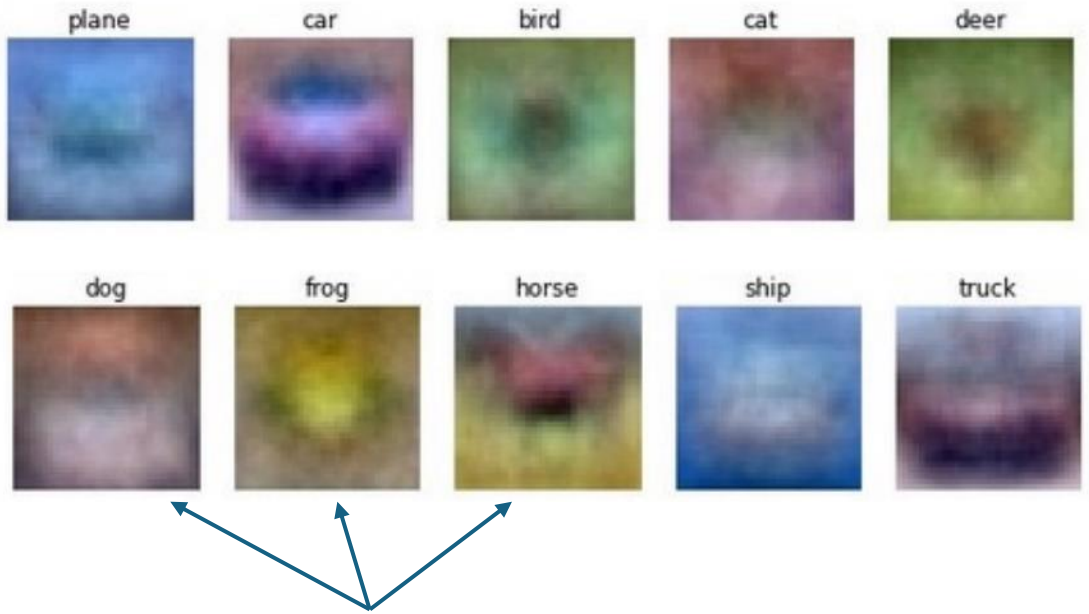
Simplified Matrix Multiplication: Bias Trick



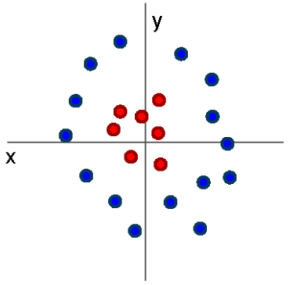
geometric interpretation:
separating hyperplanes



matching interpretation:
generic class templates



different rows in matrix W



Cannot separate red and blue points with linear classifier

raw-pixel images not linearly separable
→ linear model has not enough representational power

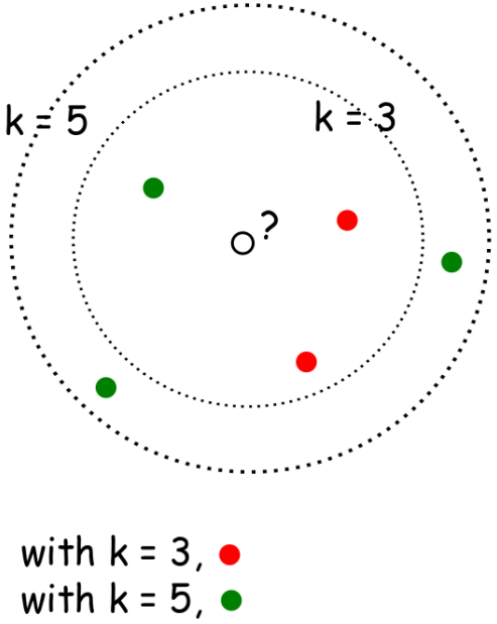
k-Nearest Neighbors (kNN)

local method, instance-based learning
non-parametric
distance defined by metric on $\boldsymbol{x}$ (e.g., Euclidean)

regression:

$$\hat{f}(\boldsymbol{x}_0) = \frac{1}{k}\sum_{j=1}^{k} y_j$$

with j running over
k nearest neighbors of $\boldsymbol{x}_0$



low k: low bias but high variance
high k: low variance but high bias

$$bias = f(\boldsymbol{x}) - \frac{1}{k}\sum_{j=1}^{k} y_j$$

$$var = \frac{\sigma^2}{k}$$

Image Classification with kNN

choose an image distance, e.g., L1 distance:

$$d(\boldsymbol{I}_1, \boldsymbol{I}_2) = \sum_p |I_1(p) - I_2(p)|$$

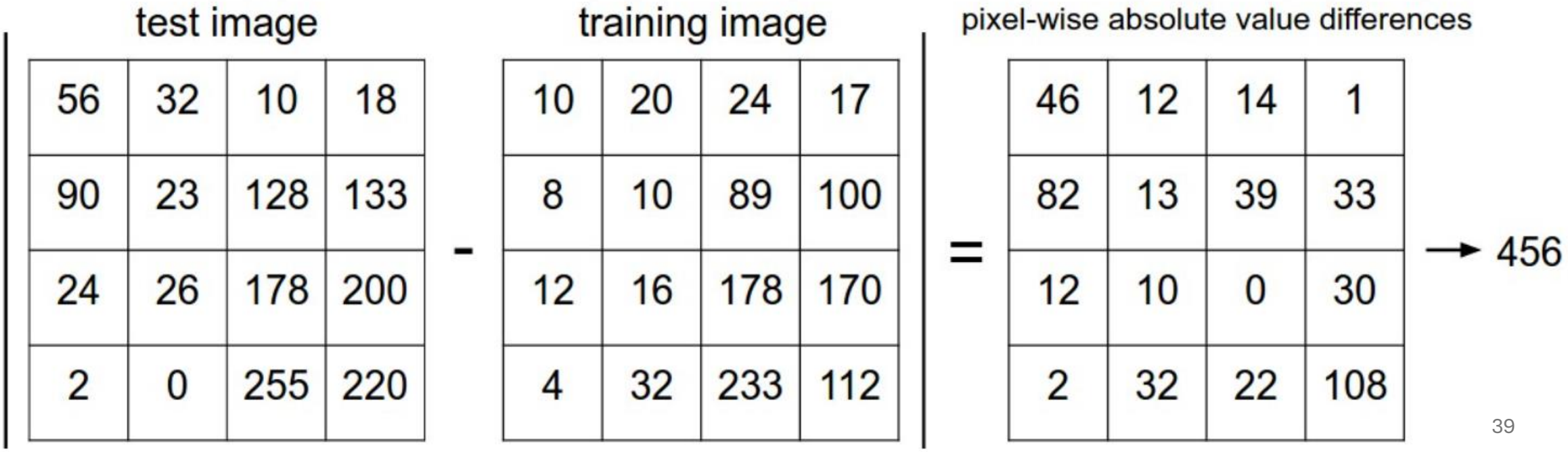
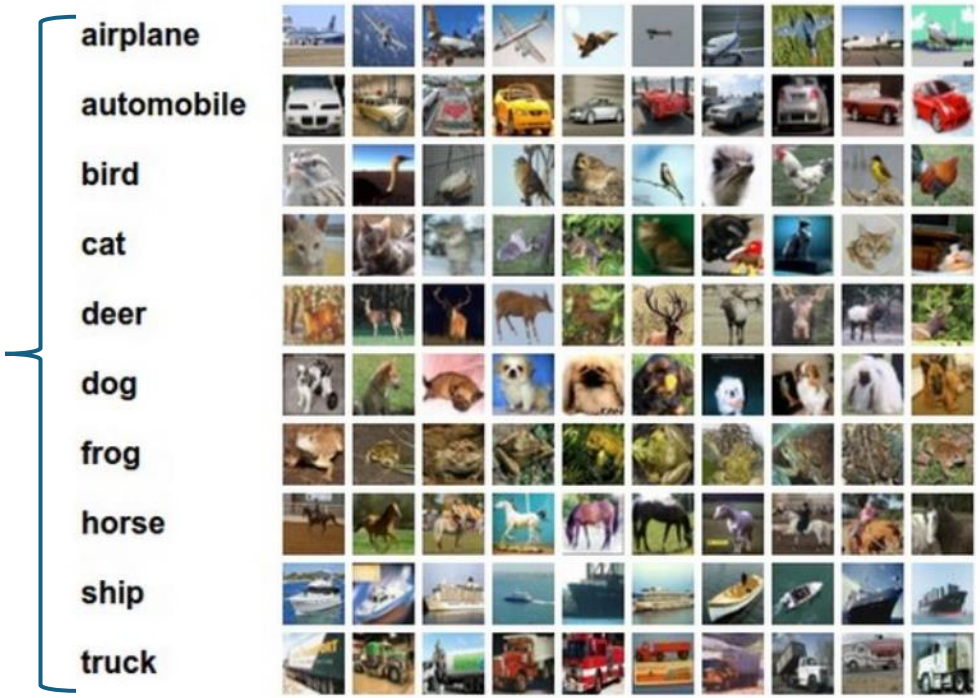


Image Classification with kNN

training examples CIFAR-10 data set

10 classes



10 nearest neighbors to some test images



better than random guessing, but also not very impressive

Image Classification: Lots of Categories



Which Features for Image Classification?

linear regression & kNN (same for tree-based methods):

learning directly from raw pixel intensities does not work great

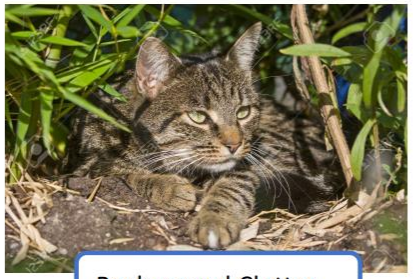
→ try learning from pre-extracted features, such as HOG or SIFT

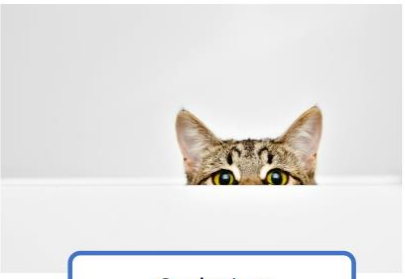
challenges:


Viewpoint Variation


Lighting Variation


Deformation

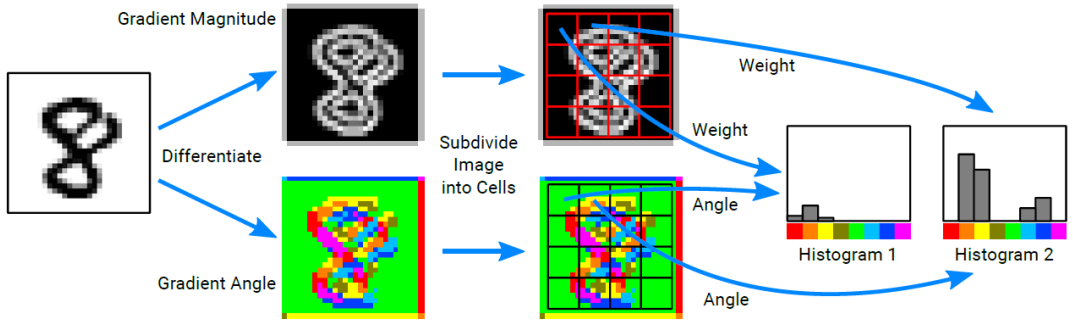

Background Clutter


Occlusion

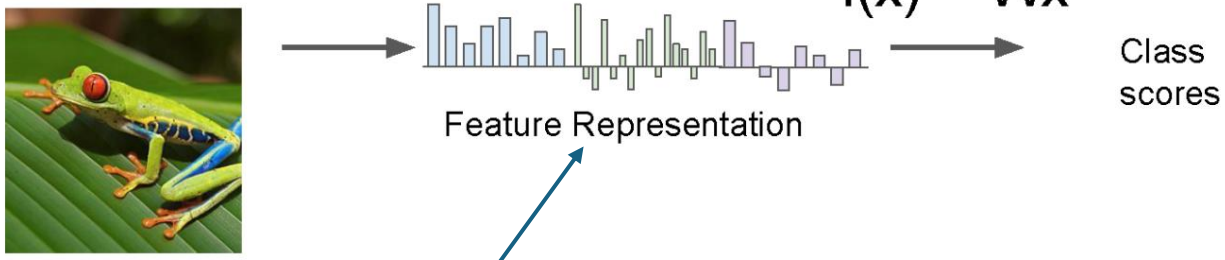
Global Features in Classic ML Method

features covering entire image
(instead of only local patches)

often used: HOG



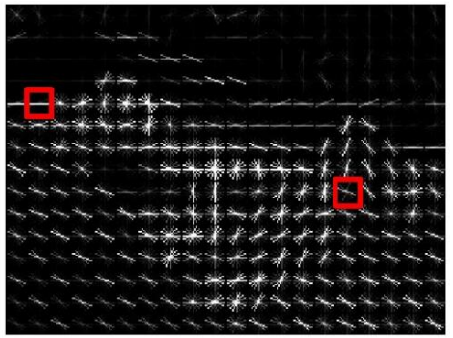
or random forest, support-vector machine, ...



easier to separate than raw pixels



Divide image into 8x8 pixel regions
Within each region quantize edge direction into 9 bins



Example: 320x240 image gets divided into 40x30 bins; in each bin there are 9 numbers so feature vector has 30*40*9 = 10,800 numbers

Lowe, "Object recognition from local scale-invariant features", ICCV 1999
Dalal and Triggs, "Histograms of oriented gradients for human detection," CVPR 2005

important disadvantage: not translation invariant (due to ordering of patches)

Local Features in Classic ML Method

local features like SIFT do not cover the entire image

→ need for some processing to identify features in different images (to be able to compare different images with ML)

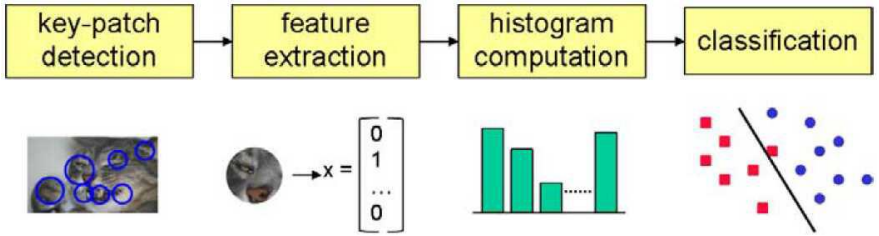
in HOG this is done by ordering of patches making up the complete image

one popular approach, called bag-of-words model (as it is orderless):

1. learn clustering of SIFT vectors (e.g., with K-means)
2. quantization of different clusters into visual “vocabulary” (see embeddings in language models)
3. create (sparse) histogram of visual “word” occurrences
4. train ML model (e.g., random forest) with histogram bins as features







important disadvantage: ignores spatial relationships among different patches

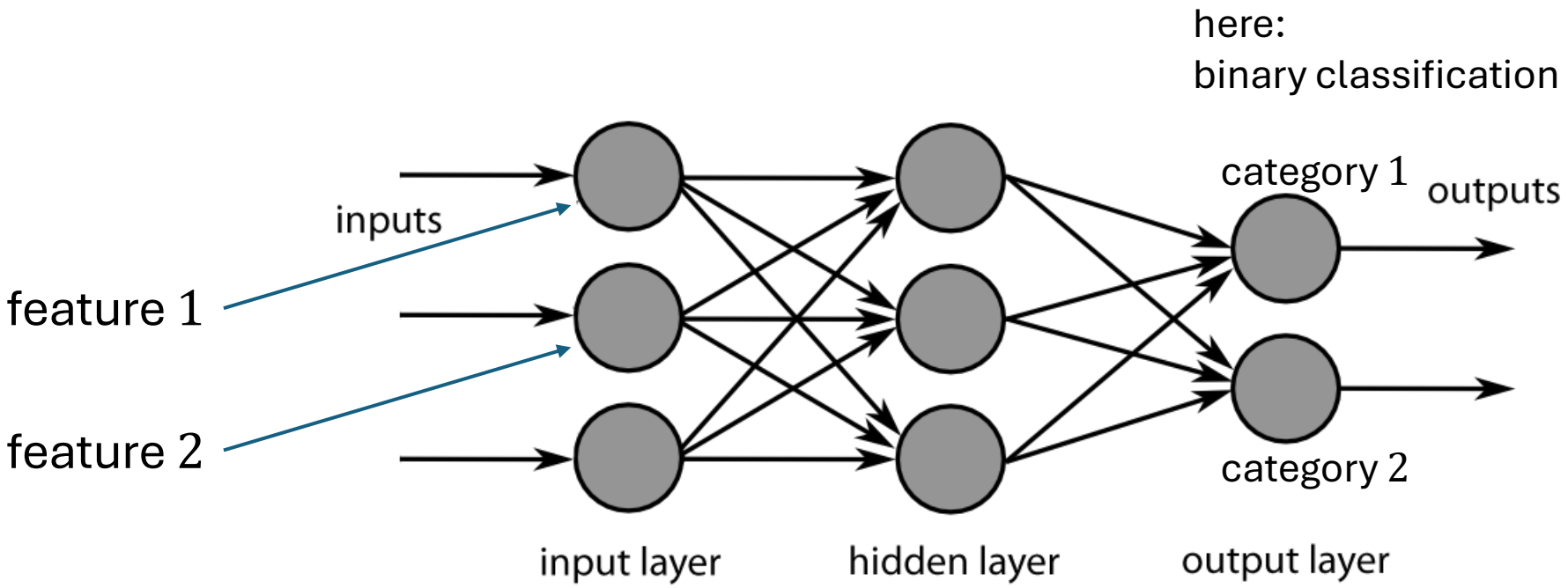
Need for Feature Learning

many hand-designed components in approach using pre-extracted features → poor generalization

Is there maybe some way after all to learn features end-to-end from raw pixel intensities?

Deep Learning

Neural Networks



each node exchanges information only with directly connected nodes
→ parallel computation

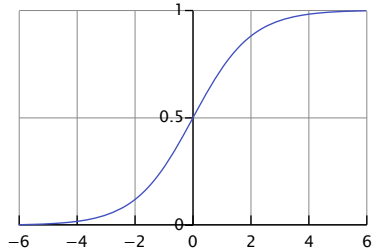
Artificial Neuron

linear model with parameters called weights $\boldsymbol{w}$ (including bias, not shown here for simplicity)

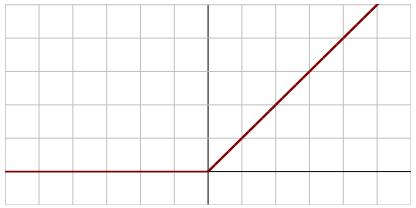
non-linear via (differentiable) activation function on top of linear model

examples:

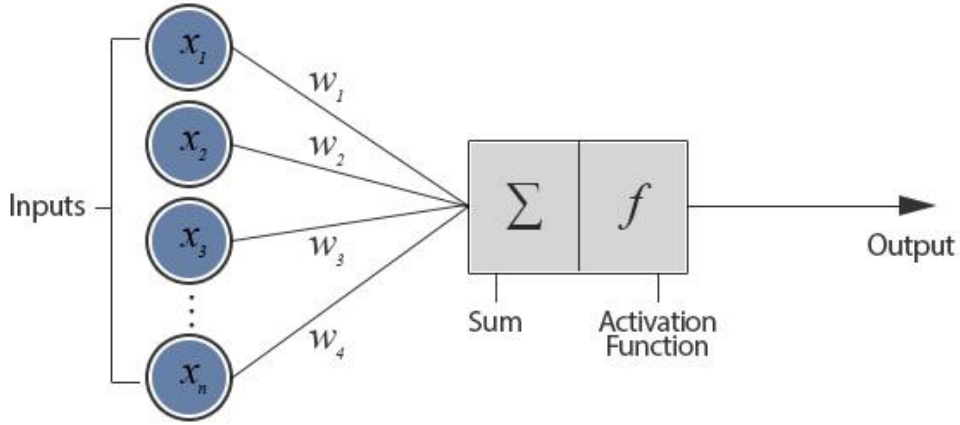
- sigmoid



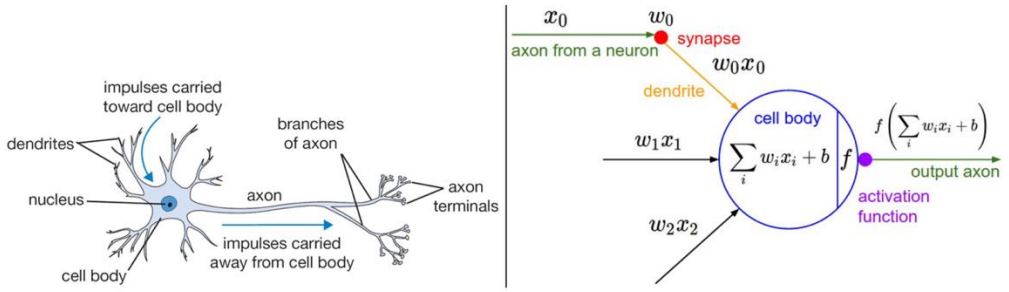
- ReLU (Rectified Linear Unit)



artificial neuron (node in neural network):

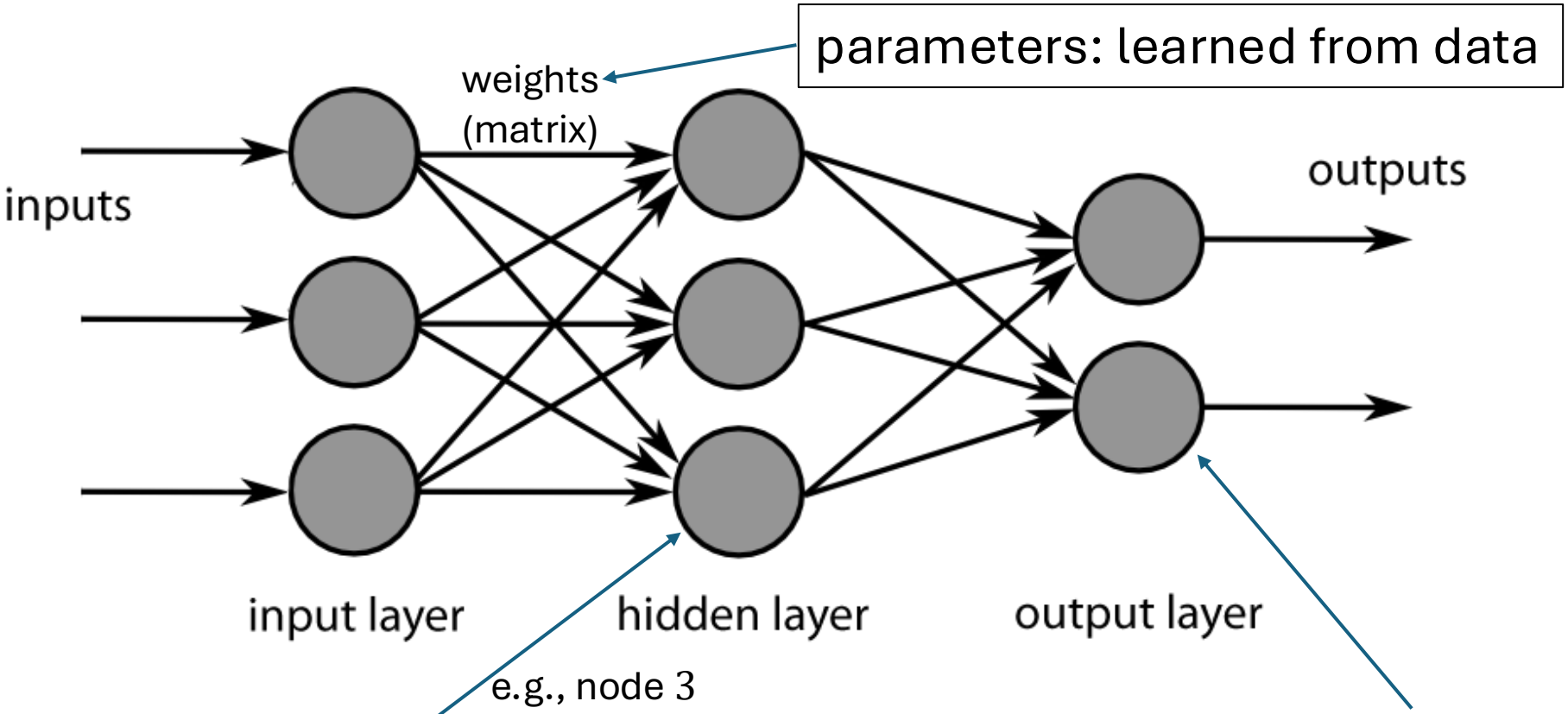


inspired from biological neurons:



source

Neural Network as Nonlinear Function

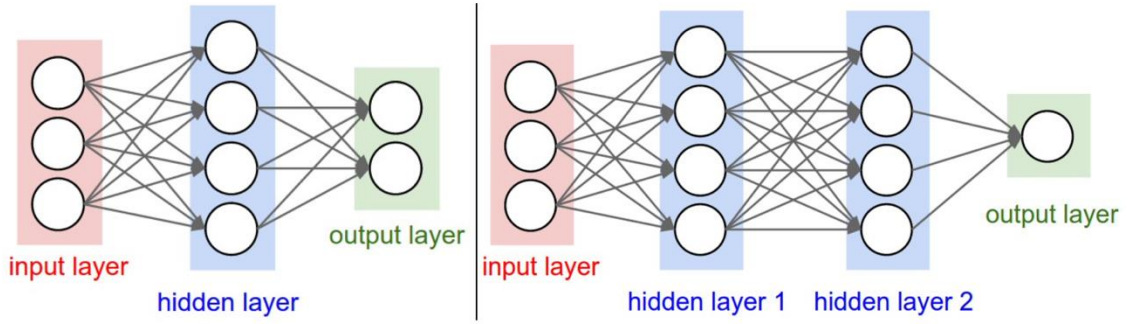


linear transformation with nonlinear activation functions

Multi-Layer Perceptron (MLP)

fully-connected feed-forward network with at least one hidden layer (universal function approximator)

binary classification



regression

toward deep learning: add hidden layers

more layers (depth) more efficient than just more nodes (width): less parameters needed for same function complexity

Classification Networks

cross-entropy loss (one output node for each class k):

$$L(y, \hat{y}) = -\sum_{k=1}^{K} y_k \log \hat{y}_k$$

using softmax on output nodes:

$$\hat{y}_k = \frac{e^{o_k}}{\sum_{l=1}^{K} e^{o_l}}$$

→ prediction of probabilities

Regression Networks

squared error loss (just one output node):

$$L(y, \hat{y}) = (y - \hat{y})^2$$

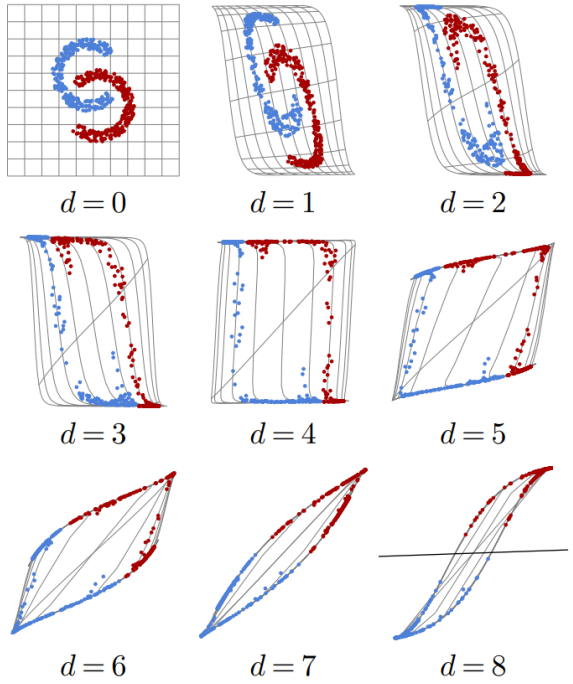
no function (identity function) on output

→ prediction of real numbers

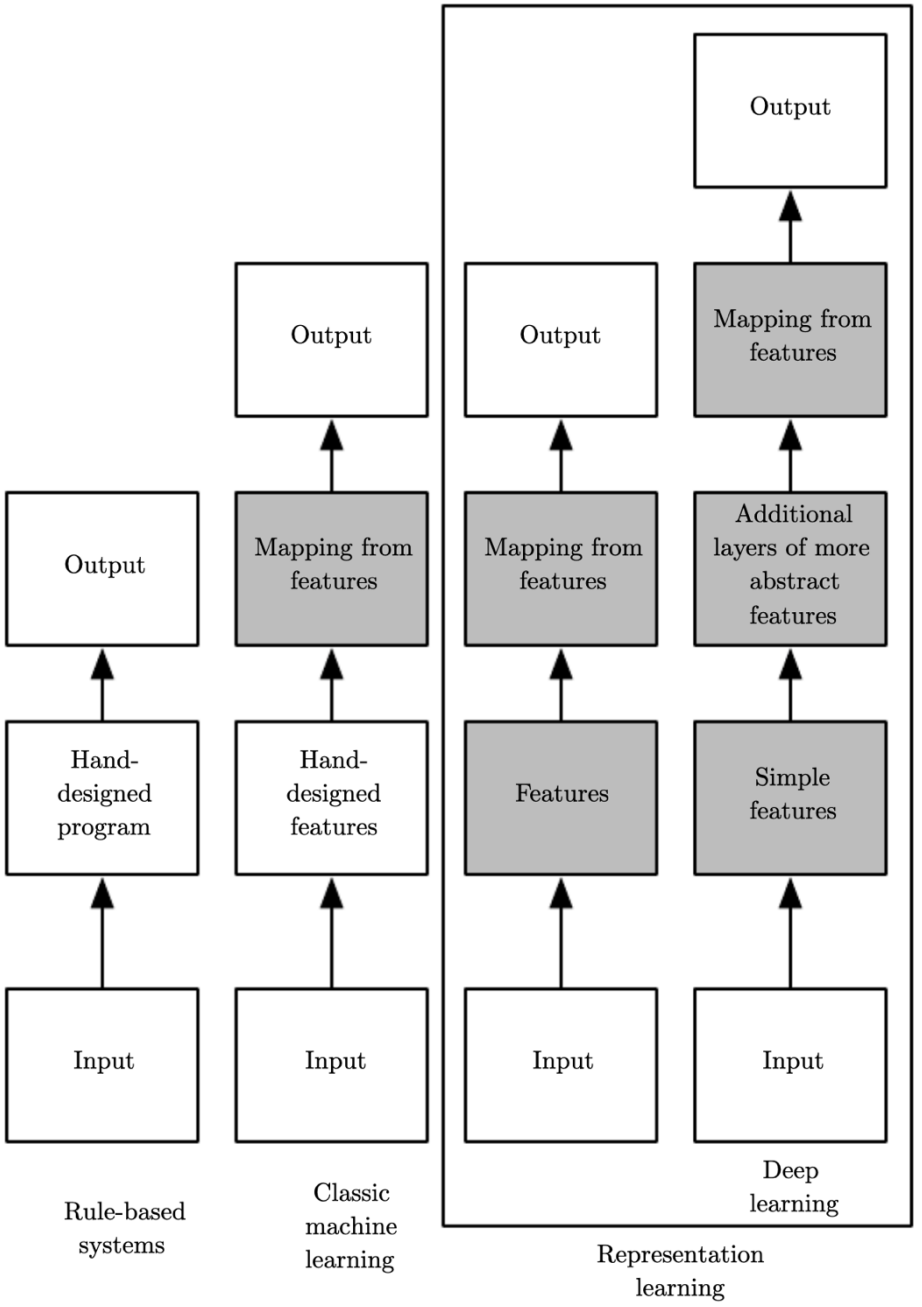
Ladder of Generalization

classic ML: feature engineering

deep learning: feature learning



source

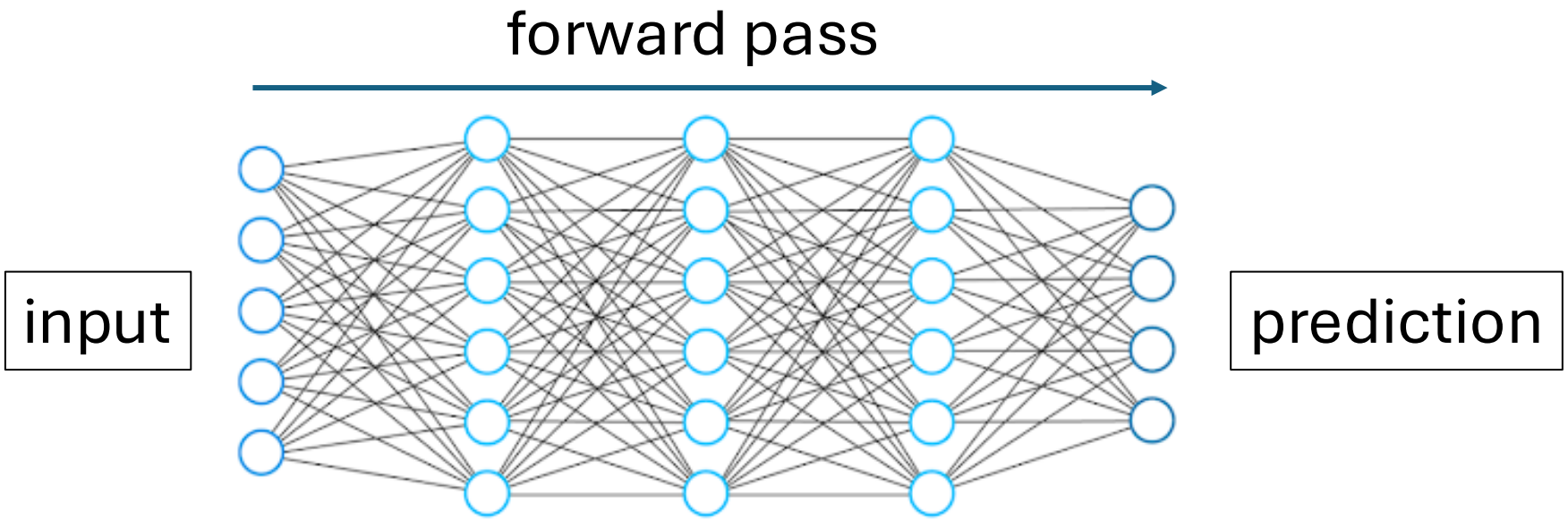


source

Forward Pass

for each training example:

- fix current weights
- compute prediction



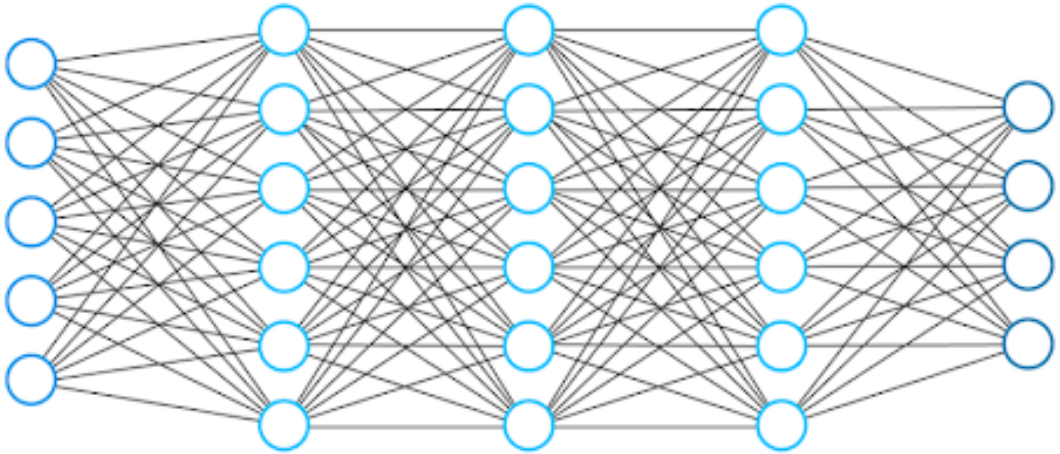
Backward Pass

cost function: $J(\boldsymbol{w}) = \frac{1}{n}\sum_{i=1}^{n} L_i(\boldsymbol{w})$

gradient: $\nabla_{\boldsymbol{w}} J(\boldsymbol{w})$

for all training examples (in considered batch):

- compute individual errors (losses)
- calculate gradient: backpropagation via chain rule of calculus



backward pass

predictions

= ? ← target

error measurement

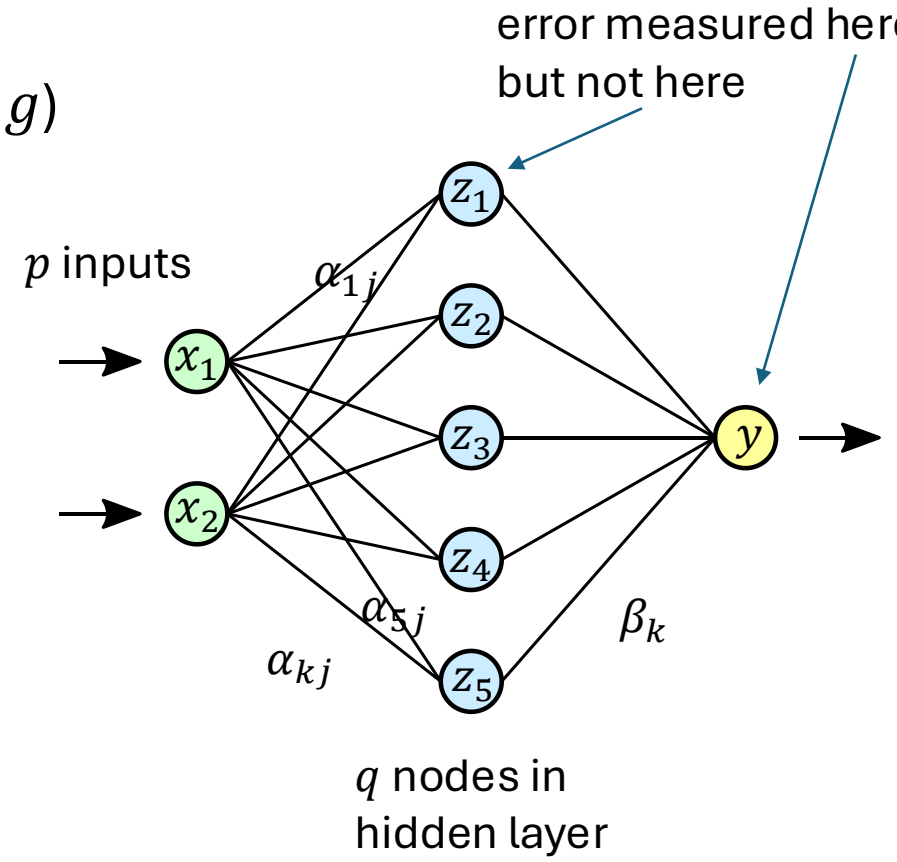
Example WOLOG

- regression (squared error loss, identity output function g)
- with one hidden layer ($\boldsymbol{w}$: $\boldsymbol{\alpha}, \boldsymbol{\beta}$)

$$\hat{y}_i = f(\boldsymbol{x}_i; \boldsymbol{w}) = g(\boldsymbol{z}_i; \boldsymbol{\beta}) = \sum_{k=0}^{q} \beta_k z_{ik}$$

$$z_{ik} = h(\boldsymbol{x}_i; \boldsymbol{\alpha}_k) = h\left(\sum_{j=0}^{p} \alpha_{kj} x_{ij}\right)$$

activation function



cost function:

$$J(\boldsymbol{w}) = \sum_{i=1}^{n} L_i(y_i, \hat{y}_i; \boldsymbol{w}) = \sum_{i=1}^{n} \left(y_i - f(\boldsymbol{x}_i;\ \boldsymbol{\alpha}, \boldsymbol{\beta})\right)^2$$

Example WOLOG

gradients:

$$\frac{\partial L_i}{\partial \beta_k} = -2\,(y_i - \hat{y}_i) z_{ik} = \delta_i z_{ik}$$

$$\frac{\partial L_i}{\partial \alpha_{kj}} = -2\,(y_i - \hat{y}_i)\beta_k h_k' \left(\sum_{j=0}^{p} \alpha_{kj} x_{ij} \right) x_{ij} = s_{ik} x_{ij}$$

backpropagation equations: $s_{ik} = h_k' \left(\sum_{j=0}^{p} \alpha_{kj} x_{ij} \right) \beta_k \delta_i$

use errors of later layers to calculate errors of earlier ones (avoiding redundant calculations of intermediate terms)

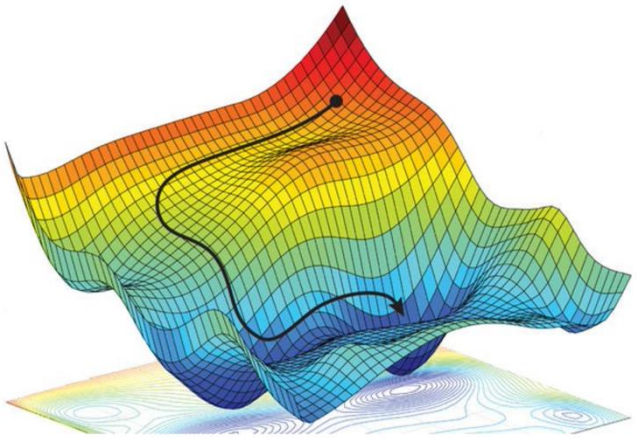
Using Gradients for Iterative Learning

use gradients found via backpropagation to iteratively update weights (gradient descent):

$$\beta_k^{(r+1)} = \beta_k^{(r)} - \eta_r \sum_{i=1}^{n} \frac{\partial L_i}{\partial \beta_k^{(r)}} \qquad \alpha_{kj}^{(r+1)} = \alpha_{kj}^{(r)} - \eta_r \sum_{i=1}^{n} \frac{\partial L_i}{\partial \alpha_{kj}^{(r)}}$$

- adaptive learning rate η_r: learning rate often adjusted per iteration
- weight initialization: choose small random weights as starting values to break symmetry (typically using some heuristics)

Stochastic Gradient Descent (SGD)



step size

cost function

weight updates: $\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta \nabla_{\boldsymbol{w}} \mathcal{L}$

all training examples

options:

- $\mathcal{L} = \frac{1}{n} \sum_{i=1}^{n} L_i$ batch gradient descent (whole training dataset)
- $\mathcal{L} = L_i$ stochastic gradient descent (single example)
- $\mathcal{L} = \frac{1}{m} \sum_{i=1}^{m} L_i$ mini-batch stochastic gradient descent

mini-batch size → hyperparamter: tradeoffs between runtime & memory, convergence & generalization

random fluctuations of SGD help to escape saddle points

Training: Iterative Adjustment of Weights

```
for epoch in range(epochs):
    for X, y in mini_batches:
        pred = model(X)
        cost = loss(pred, y)
        cost.backward()
        optimizer.step()
```

forward pass

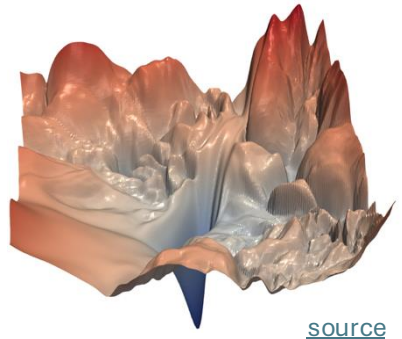
quantification of error

backpropagation

gradient descent: one step in direction of steepest descent

How to Train Deep Neural Networks Effectively?

typical loss surface:



source

optimization and regularization difficult

- local vs global minima, saddle points, easily overfitting
- many hyperparameters to tune

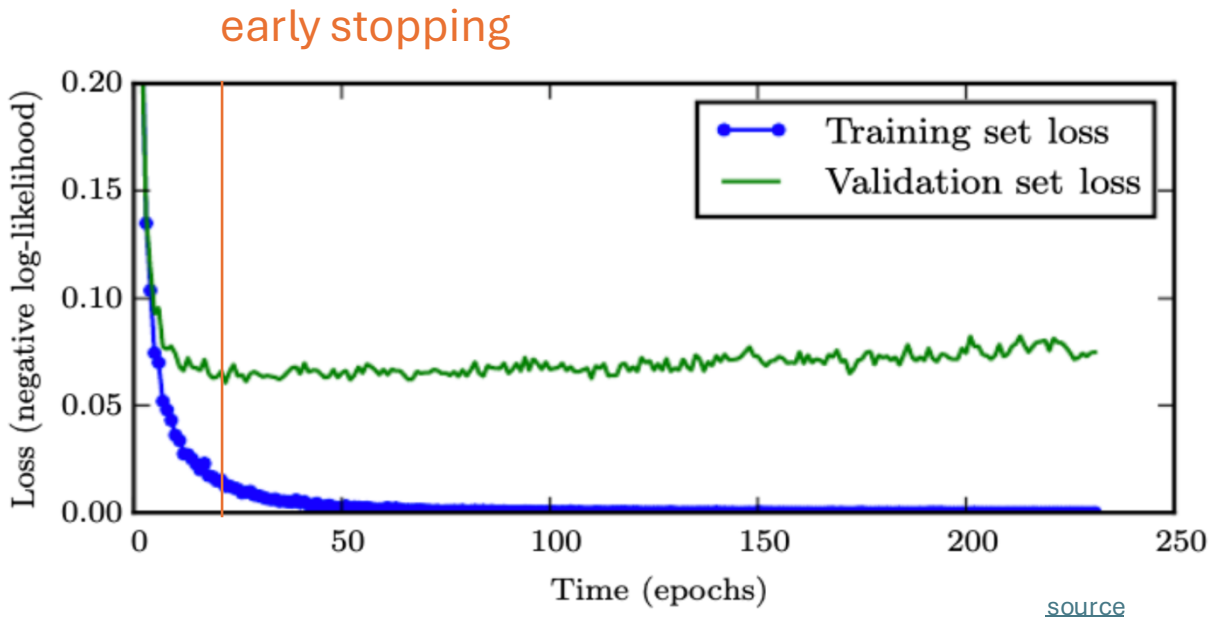
but many methods to get it working in practice (despite partly patchy theoretical understanding)

- optimization: activation and loss functions, weight initialization, adaptive learning rate (e.g., Adam), learning rate schedulers
- explicit regularization: weight decay, dropout, data augmentation
- implicit regularization: early stopping, batch/layer normalization, SGD

Early Stopping

loss independently measured on validation set

halting training when overfitting begins to occur



source

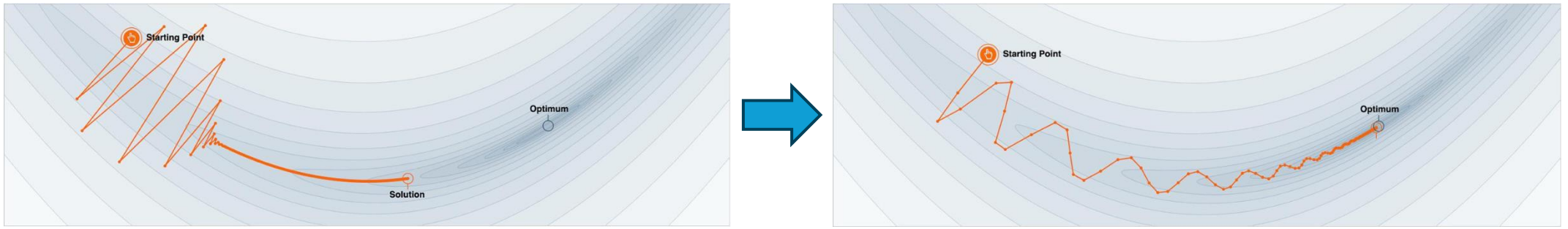
Gradient Descent with Momentum

algorithm:

- estimate gradient: $\boldsymbol{g} = \nabla_{\boldsymbol{w}} \mathcal{L}$
- compute velocity update: $\boldsymbol{v} \leftarrow \alpha\, \boldsymbol{v} - \eta\, \boldsymbol{g}$
- apply weight update: $\boldsymbol{w} \leftarrow \boldsymbol{w} + \boldsymbol{v}$

hyperparameter ($0 < \alpha < 1$) specifying exponential decay

→ no bouncing around of weights, escape from local minima and saddle points



Adaptive Learning Rate

better convergence by adapting learning rate for each weight per iteration: lower/higher learning rates for weights with large/small updates

popular methods (g_w denotes component of gradient for **individual weight** w):

- Adagrad: $w \leftarrow w - \frac{\eta}{\sqrt{\sum_{\tau=1}^{t} g_{w,\tau}^2}} g_w$ with t, τ denoting current and past iterations

  (issue: sum in denominator grows with more iterations → can get stuck)
- RMSProp: $w \leftarrow w - \frac{\eta}{\sqrt{v(w)}} g_w$ with $v(w) \leftarrow \gamma\, v(w) + (1-\gamma) g_w^2$
- Adam (Adaptive Moment Optimization): combines RMSProp with momentum

Learning Rate Schedulers

learning rate η in gradient descent typically set to small value (e.g., 0.001)

use momentum and adaptive learning rates to improve optimization

learning rate scheduling as further alternative (can be used on top):

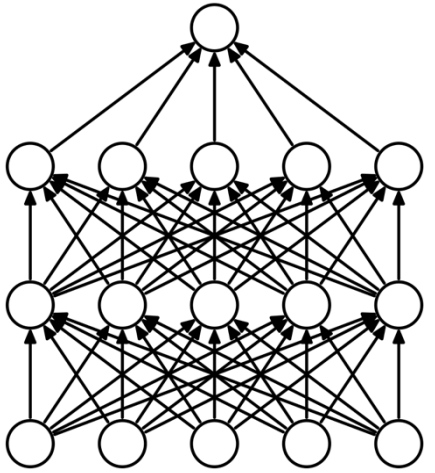
- decay by a certain factor every given number of epochs
- reduce when a metric has stopped improving
- smooth decay by cosine annealing (optionally cyclic with restarts)
- …

Dropout in Neural Networks

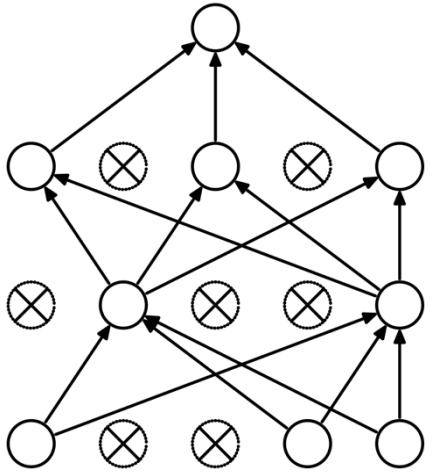
goal: prevent overfitting of large neural networks

randomly drop non-output nodes (along with their connections) during training (not prediction)

→ adaptability: regularizing each hidden node to perform well regardless of which other hidden nodes are in the model



(a) Standard Neural Net



(b) After applying dropout.

source

- for each mini-batch, randomly sample independent binary masks for the nodes
- destroying extracted features rather than input values

Batch Normalization

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;
Parameters to be learned: γ, β
Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

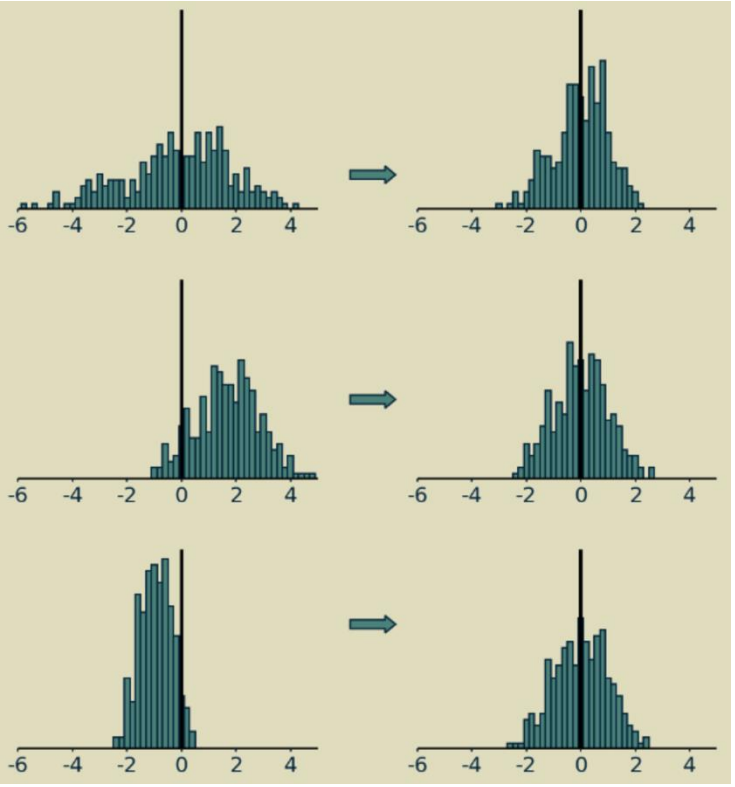
$$\sigma^2_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma \widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

source



source

adaptive reparameterization of inputs to network layer (before or after activation)

independently for each feature

(implicit) regularization effect

Benefits from Batch Normalization

- allows higher learning rates
- reduces importance of weight initialization
- alleviates vanishing/exploding gradients
- (implicit) regularization effect: introducing noise

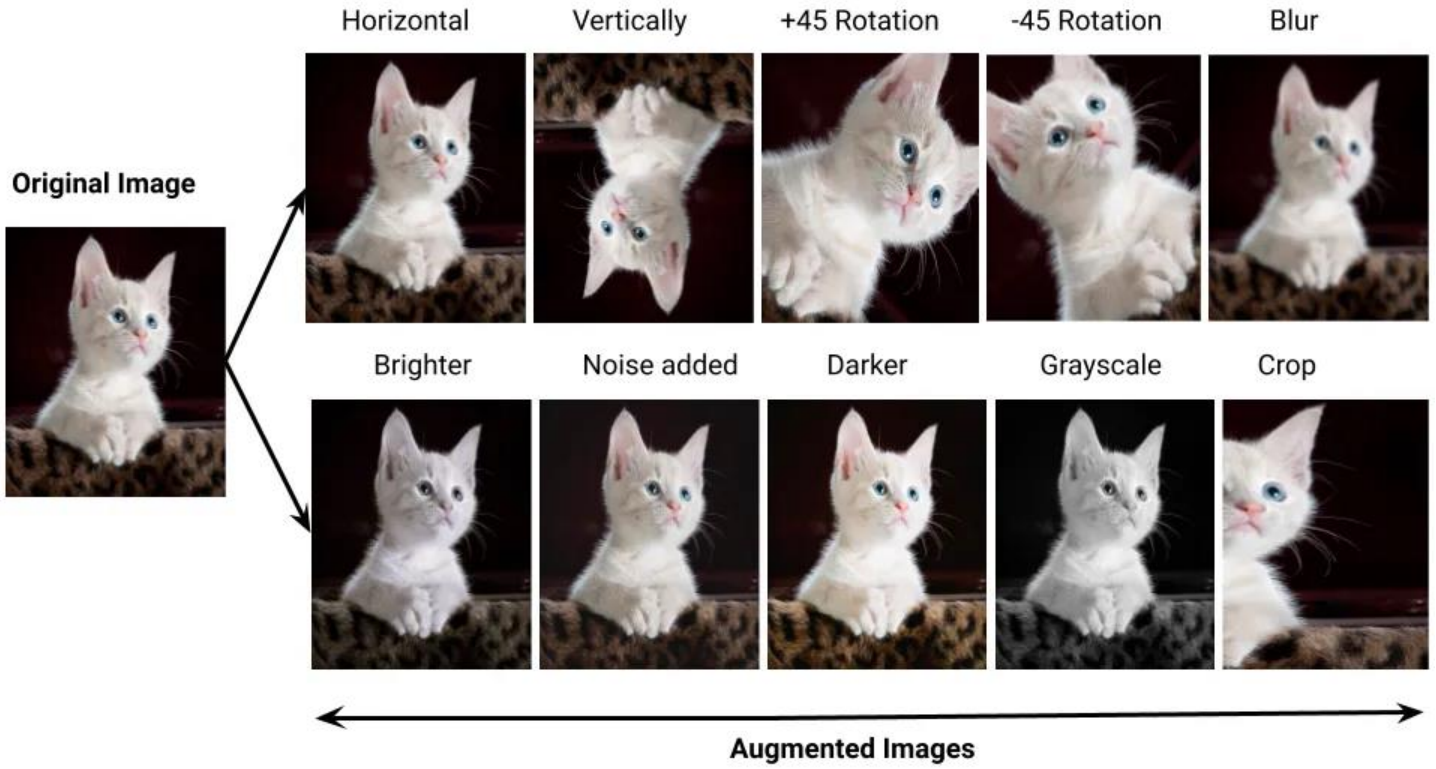
reason why batch normalization improves optimization still controversial

most plausible explanation: smoothening of loss landscape

Data Augmentation

adding different variations of input data to training

idea: supporting the model in maintaining desired invariances



Convolutional Neural Networks (CNN or ConvNet)

From Permutation Invariance to Structured Deep Learning

fully-connected networks treat input features as unordered

(e.g., the different pixel intensities in an image)

but real data is often structured → local correlations between features

- images: nearby pixels are highly correlated
- text / speech: words and sounds depend on previous context
- time series: observations depend on earlier time steps

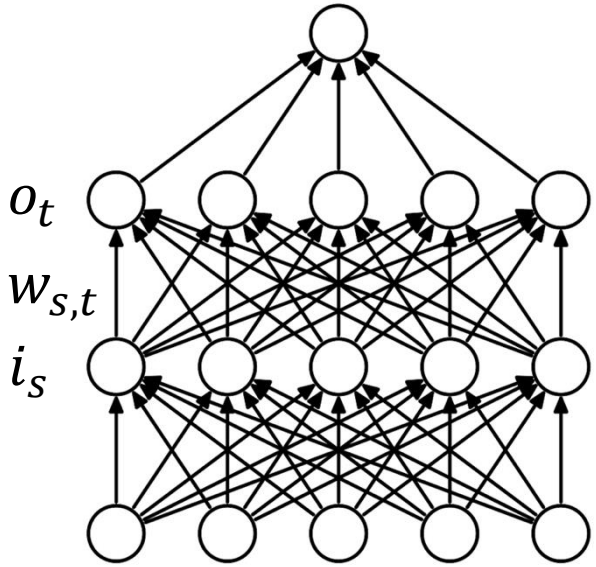
ignoring these structures leads to suboptimal models → CNNs, RNNs

Structure of Fully-Connected Neural Networks

computation in fully-connected network:

matrix multiplication of scalar inputs and weights

$$o_t = \sum_s w_{s,t}\, i_s$$



dropped dimension of different training observations (in mini-batch) in this view

Inductive Bias

plain feed-forward networks very inefficient for image classification:

make no use of spatial structure (locality of objects)

→ need to learn it from scratch

better way: introduce it right in the architecture of the ML method (called inductive bias)

→ convolution with spatial filters (**learned** rather than handcrafted)

Grid-Like Data

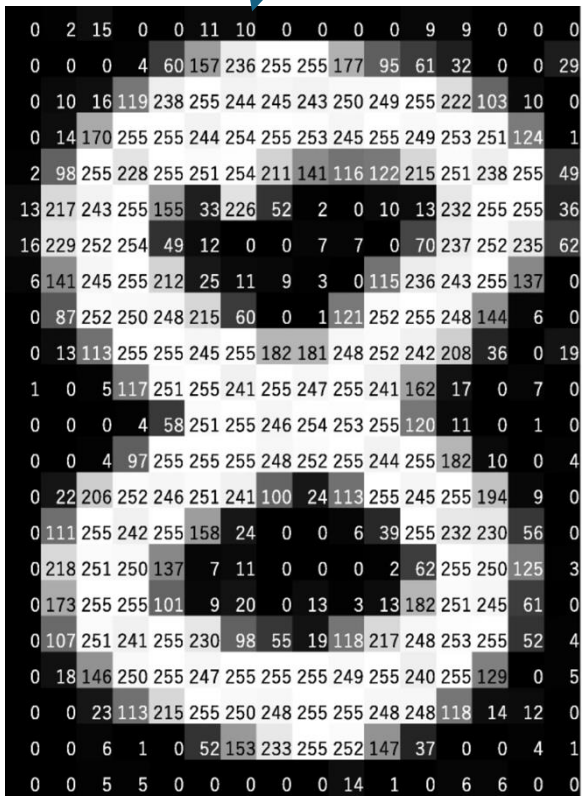
image data: 2-D grid of pixels (spatial structure)

convolutional networks:

neural networks using learned convolution kernels as weights (in place of general matrix multiplications)

→ highly regularized feed-forward networks

scalar values (like in usual feed-forward network)



source

Convolution Operation

actually, correlation (convolution would have – here):

$$o_{x,y} = \sum_{s,t} w_{s,t}\, i_{x+s,y+t}$$

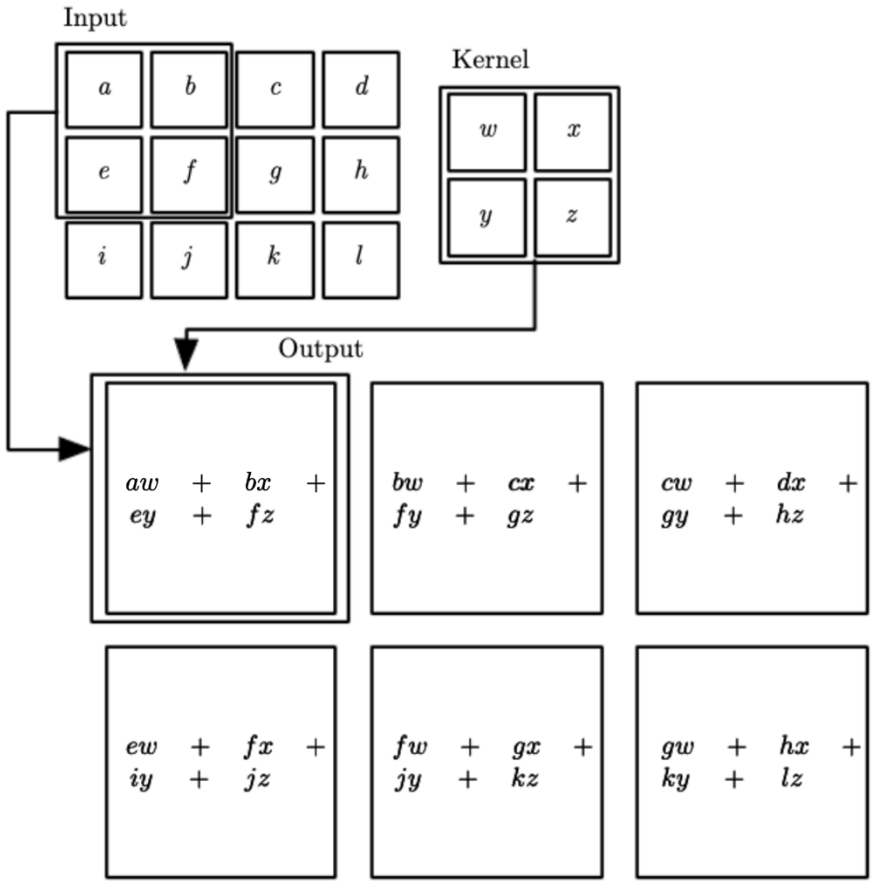
feature map
(transformed image)

kernel
(learned)

image
(input or feature map)

(again, dropping dimension of different training observations)

2D convolution

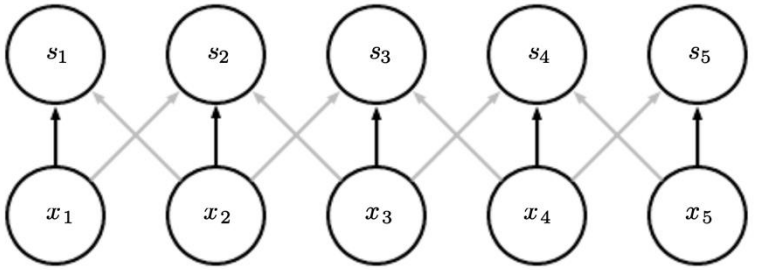


source

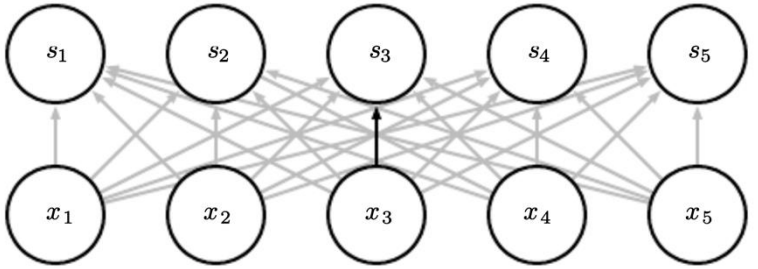
Regularization Effects

- sparse interactions: much less weights
- parameter sharing: use same weights for different connections

convolutional layer:



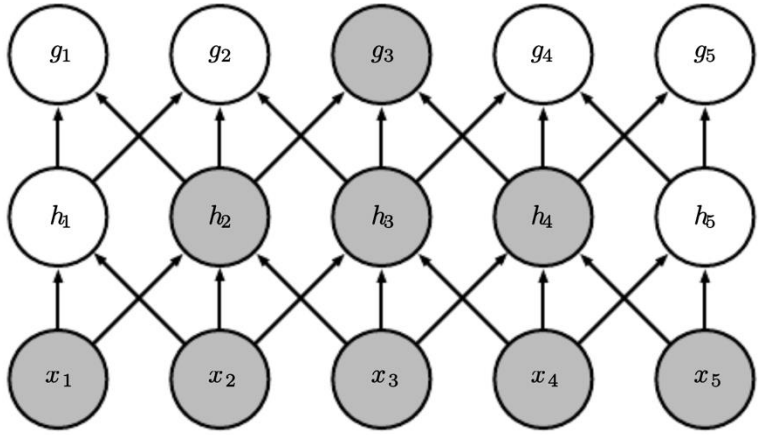
fully-connected layer:



effect of receptive field over several layers:

- consider only locally restricted number of input values from previous layer
- grows for earlier layers (indirect interactions)

→ hierarchical patterns from simple building blocks (many aspects of nature hierarchical)

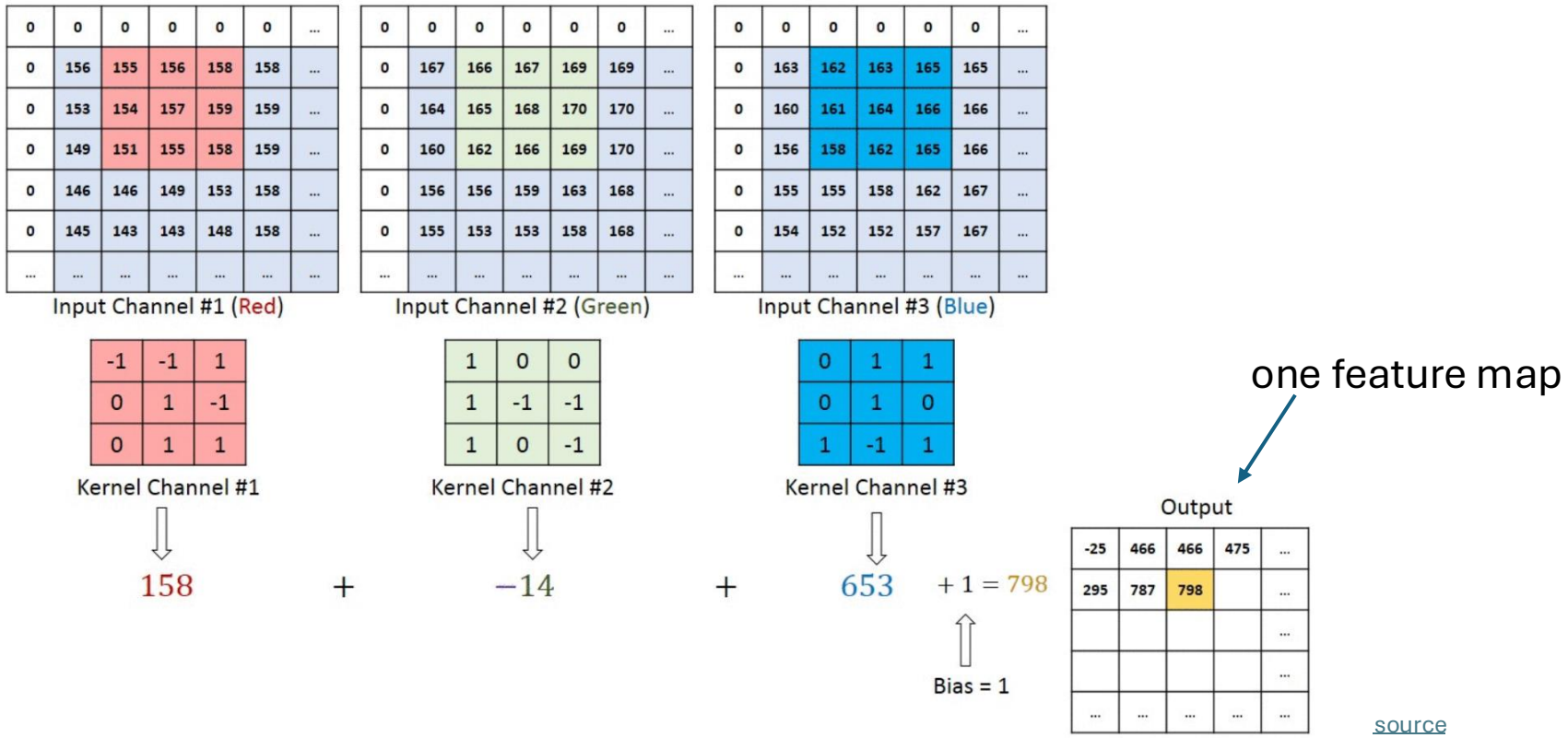


source

Channel Mixing

several input channels c (RGB) and several output channels f (different feature maps):

$$o_{f,x,y} = \sum_{c,s,t} w_{f,c,s,t}\ i_{c,x+s,y+t}$$



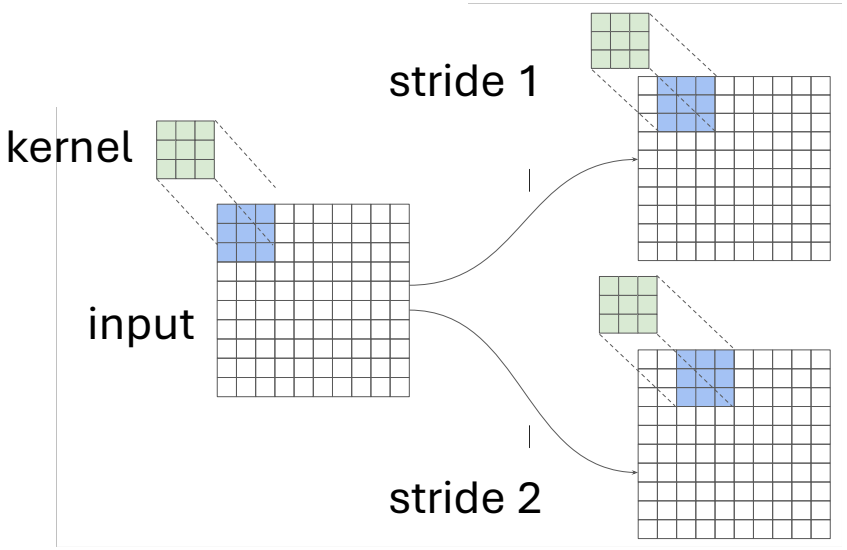
source

Striding and Padding

need to define how to stride over image

stride > 1 corresponds to down-sampling

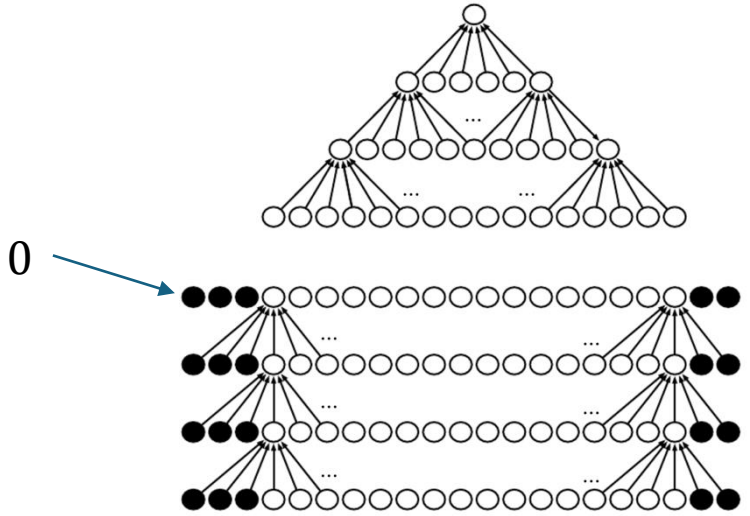
→ fewer nodes after convolutional layer



zero-padding of input to make it wider:

otherwise shrinking of represenation with each layer (depending on kernel size)

→ needed for large kernels and several layers



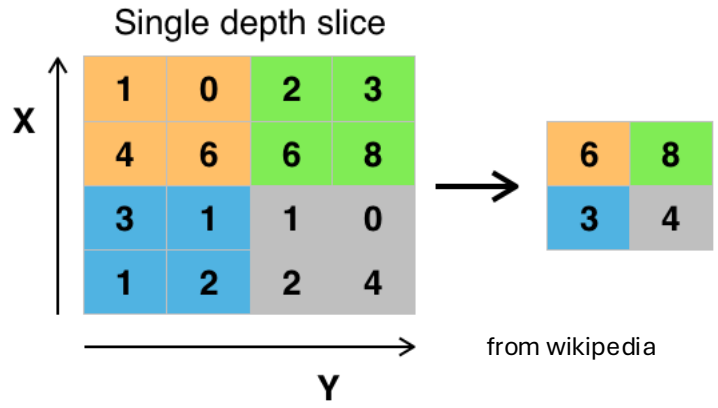
source

Another CNN Ingredient: Pooling

replacing outputs of neighboring nodes with summary statistic (e.g., maximum or average value of nodes)

→ non-linear downsampling

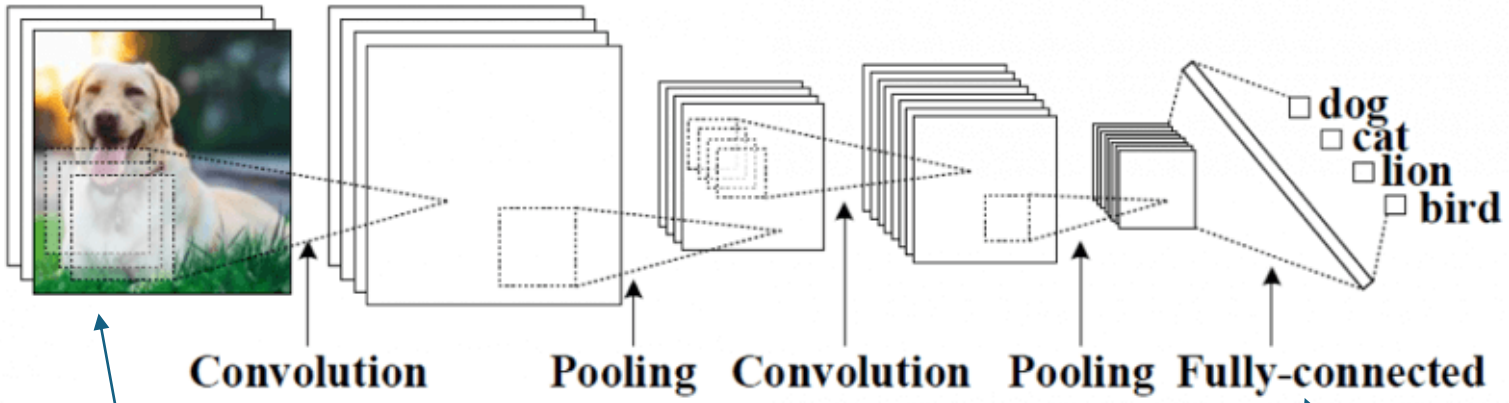
max pooling:



pooling is translation invariant: no interest in exact position of, e.g., maximum value

Putting It All Together

convolutional neural networks in short:
local connections, shared weights, pooling, many layers

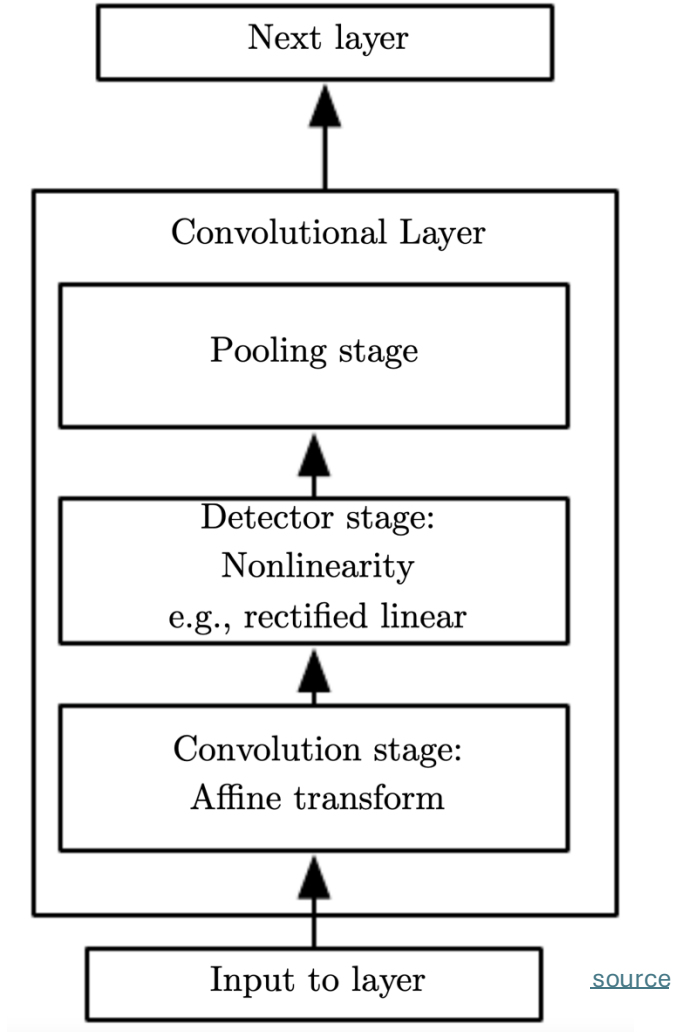


many images (training examples), potentially with several channels

several kernels producing several feature maps

downsampling

flatten dimensions for final classification



source

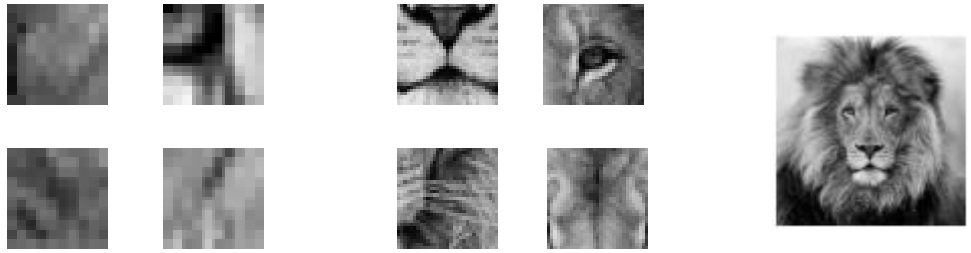
Hierarchical Representation

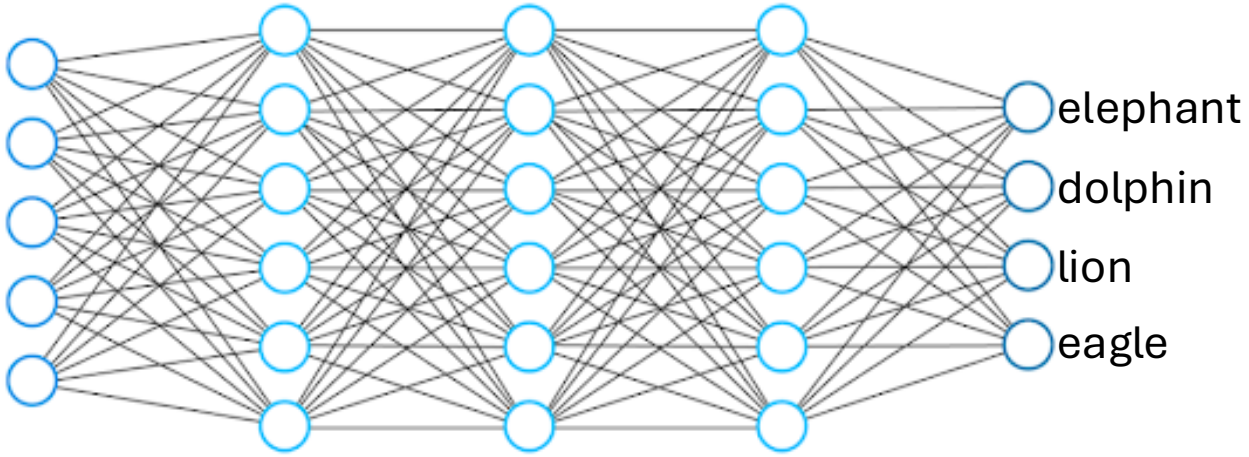
deep learning:
several hidden layers

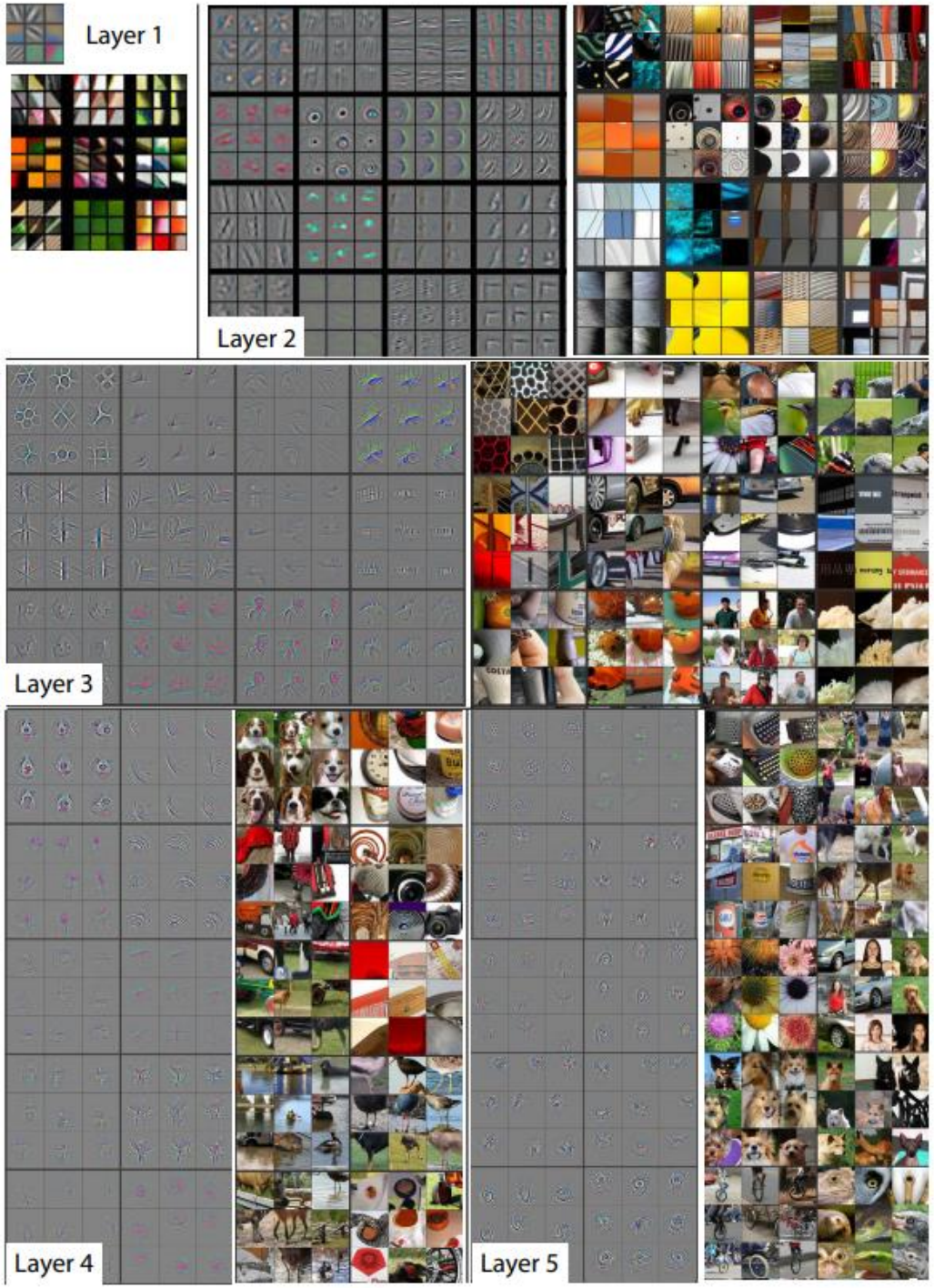
hierarchy of concepts learned from raw data in deep graph with many layers



increasing abstraction







Hierarchical Learning

shown here: top 9 activations of some random feature maps on test images, together with corresponding image patches
(projected down to pixel space using deconvolutional network)

some insights:

- strong groupings in feature maps, with exaggeration of discriminative image parts
- more global activations at higher layers
- greater invariance at higher layers

source



t-SNE Visualization

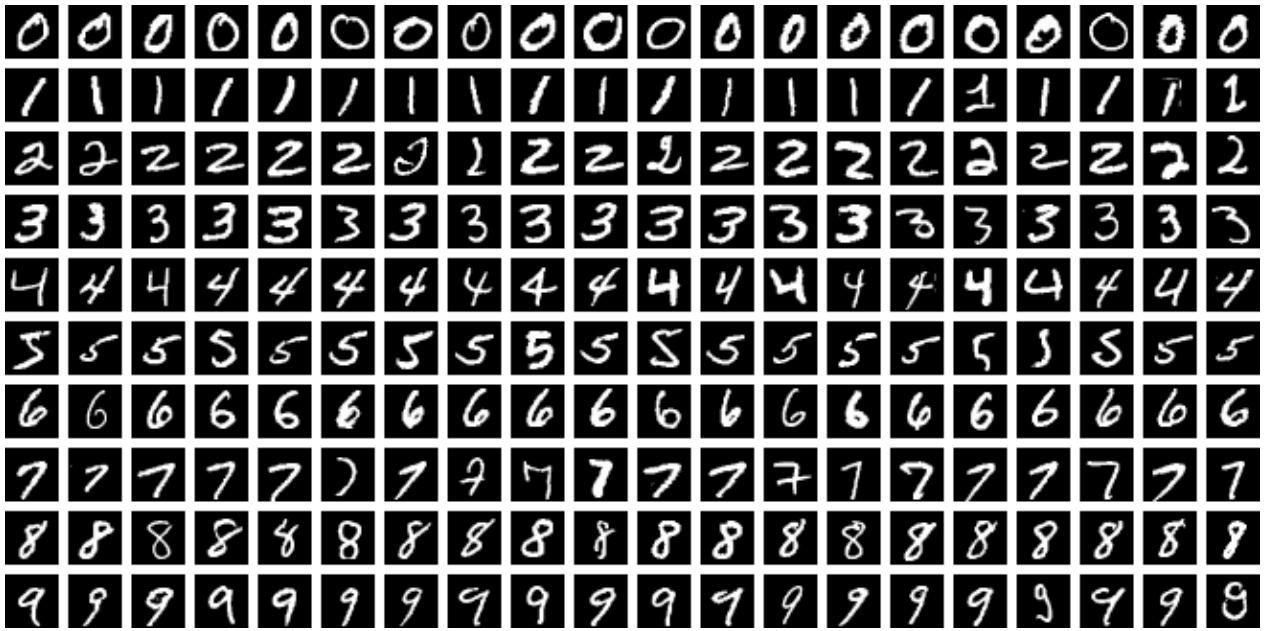
approach:
for every image, embed high-dimensional vector of last hidden layer's activations (right before final fully-connected classification layer) into two-dimensional vector (while preserving distances)

→ semantic similarities

source

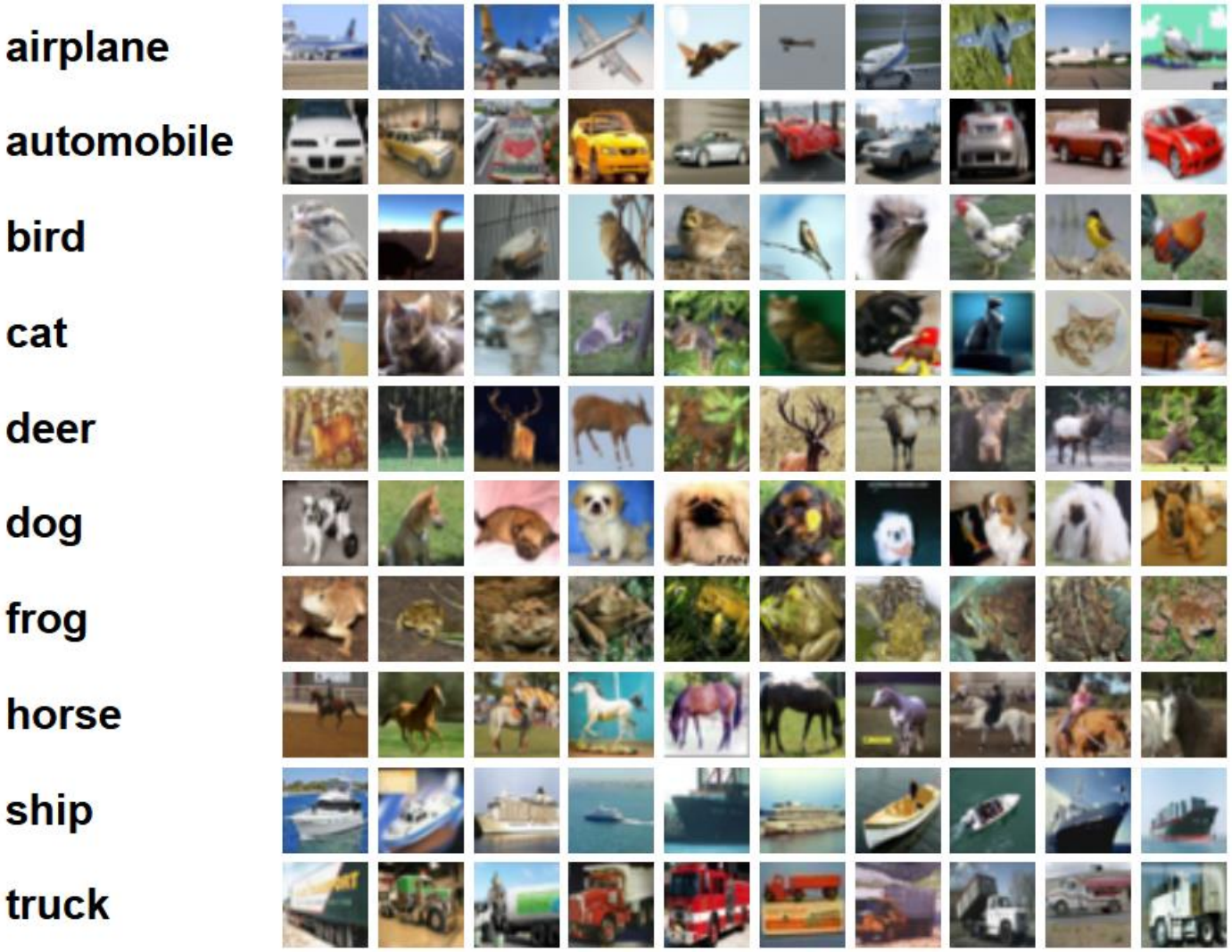
MNIST

- Modified National Institute of Standards and Technology
- handwritten digits (10 classes)
- black and white images
- 28×28 pixels
- 60k training and 10k test images



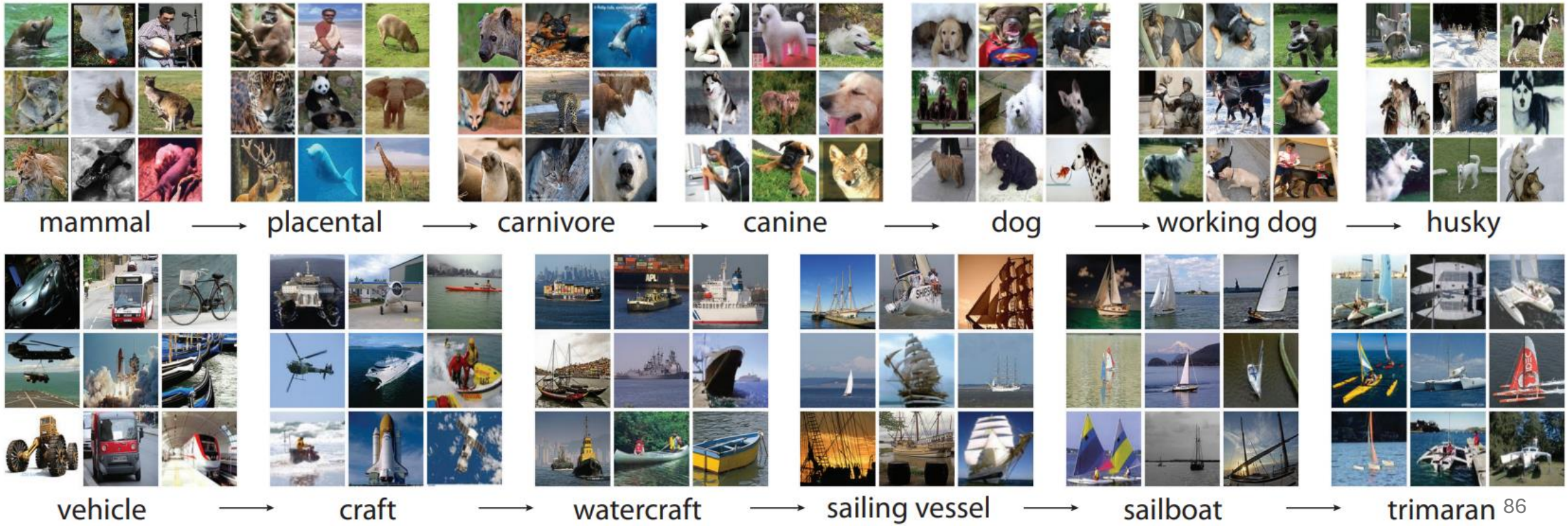
CIFAR-10

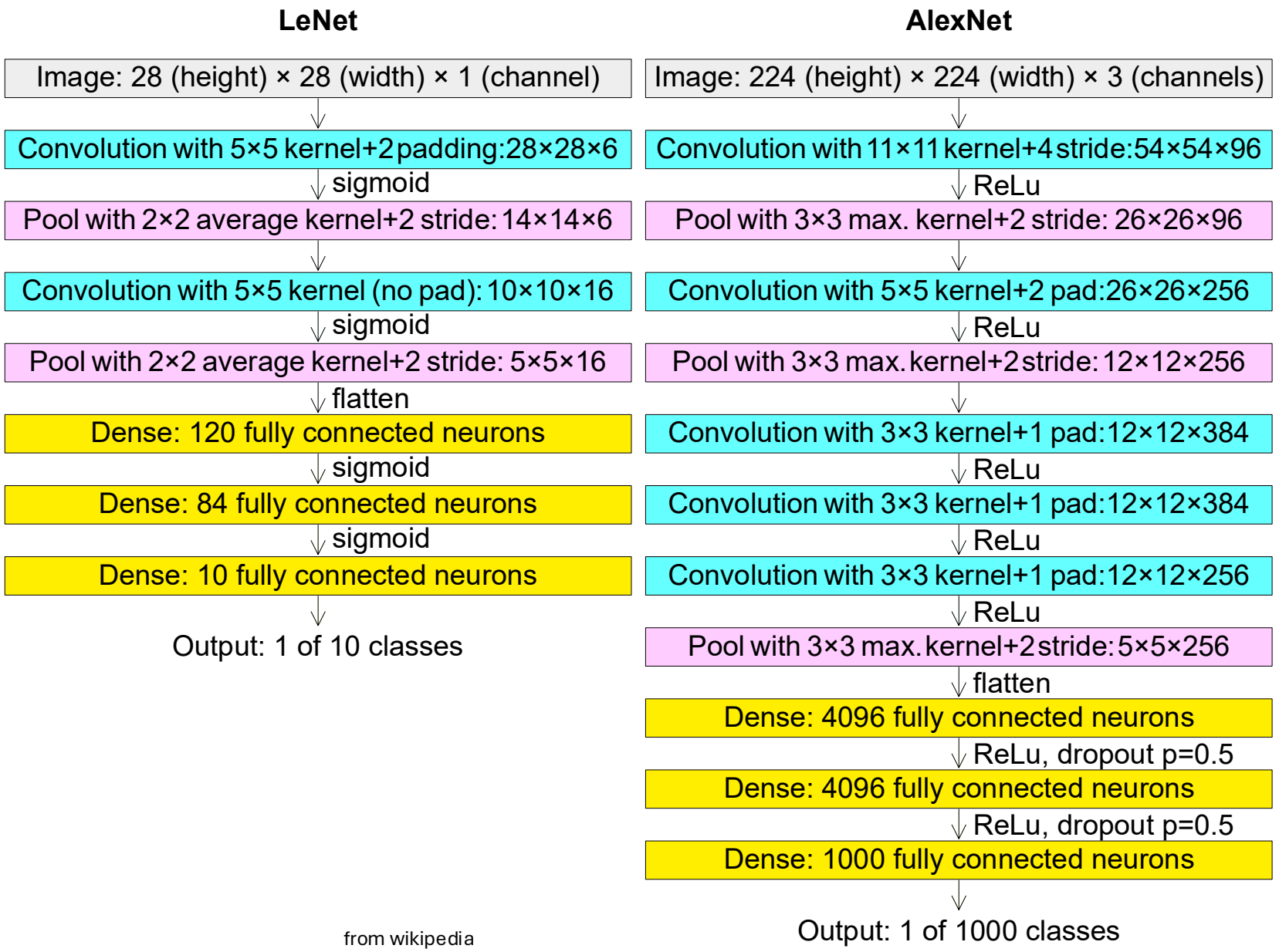
- Canadian Institute for Advanced Research
- 10 different labeled object classes
- color images
- 32×32 pixels
- 50k training and 10k test images



ImageNet

- more than 14 million color images with varying sizes
- more than 20k labeled categories





from wikipedia

AlexNet finally started the deep learning hype.

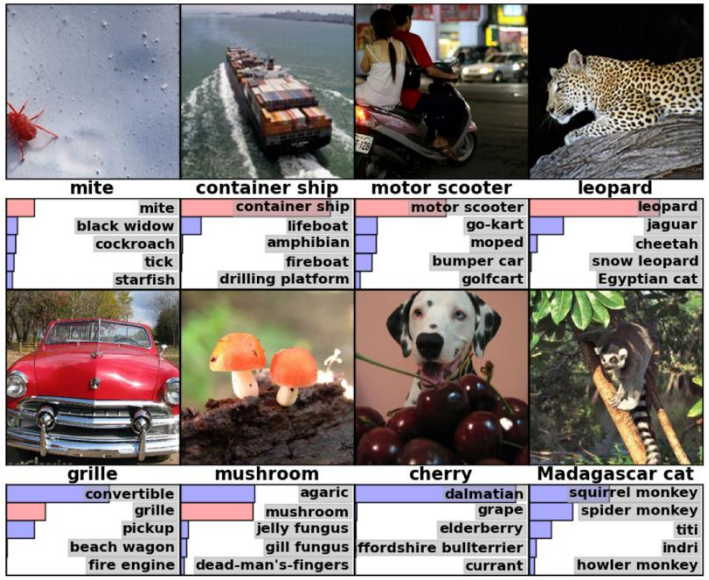
(winning the ImageNet challenge in 2012)

Rise of Deep Learning

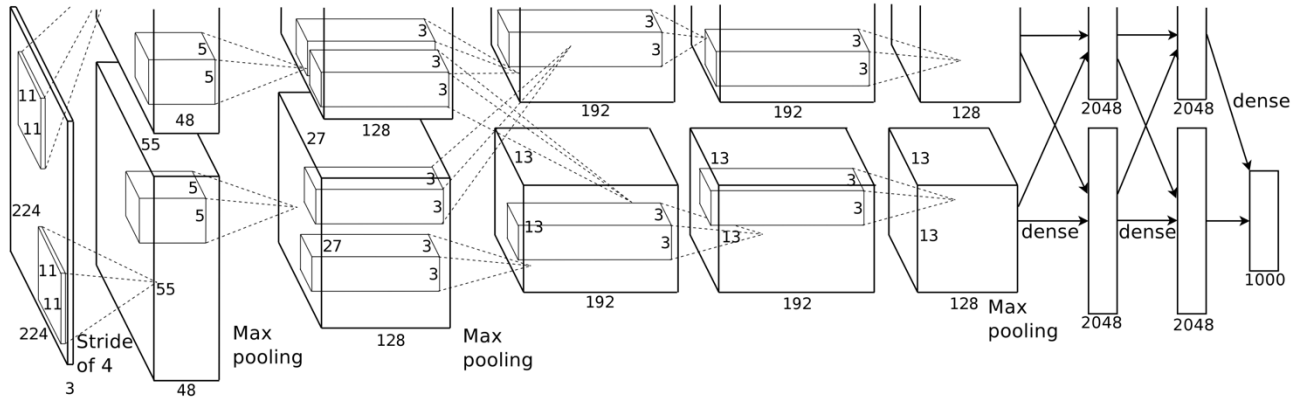
a little bit oversimplified:

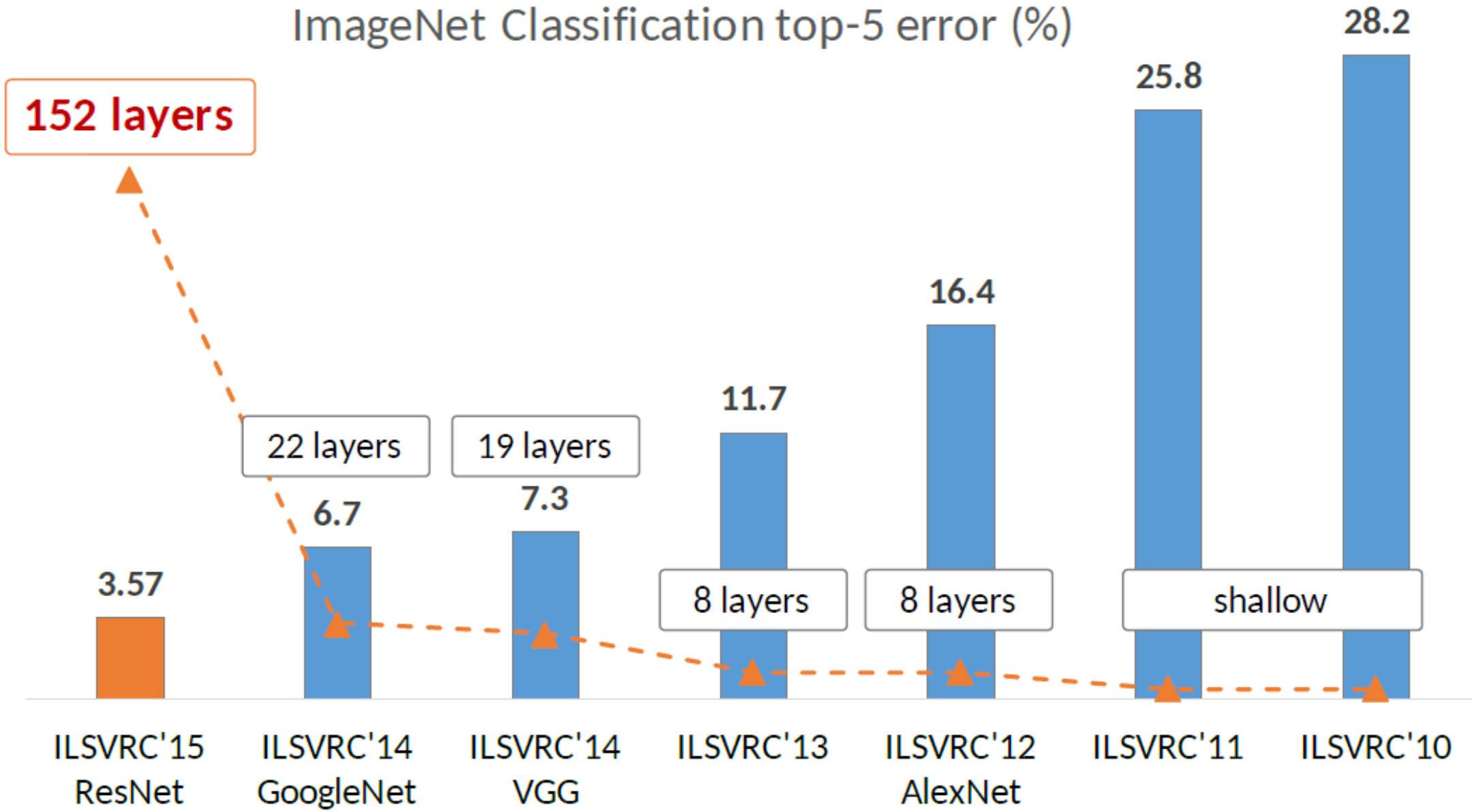
deep learning = lots of training data + parallel computation + smart algorithms

AlexNet: ImageNet (with data augmentation) + GPUs + ReLU, dropout, SGD

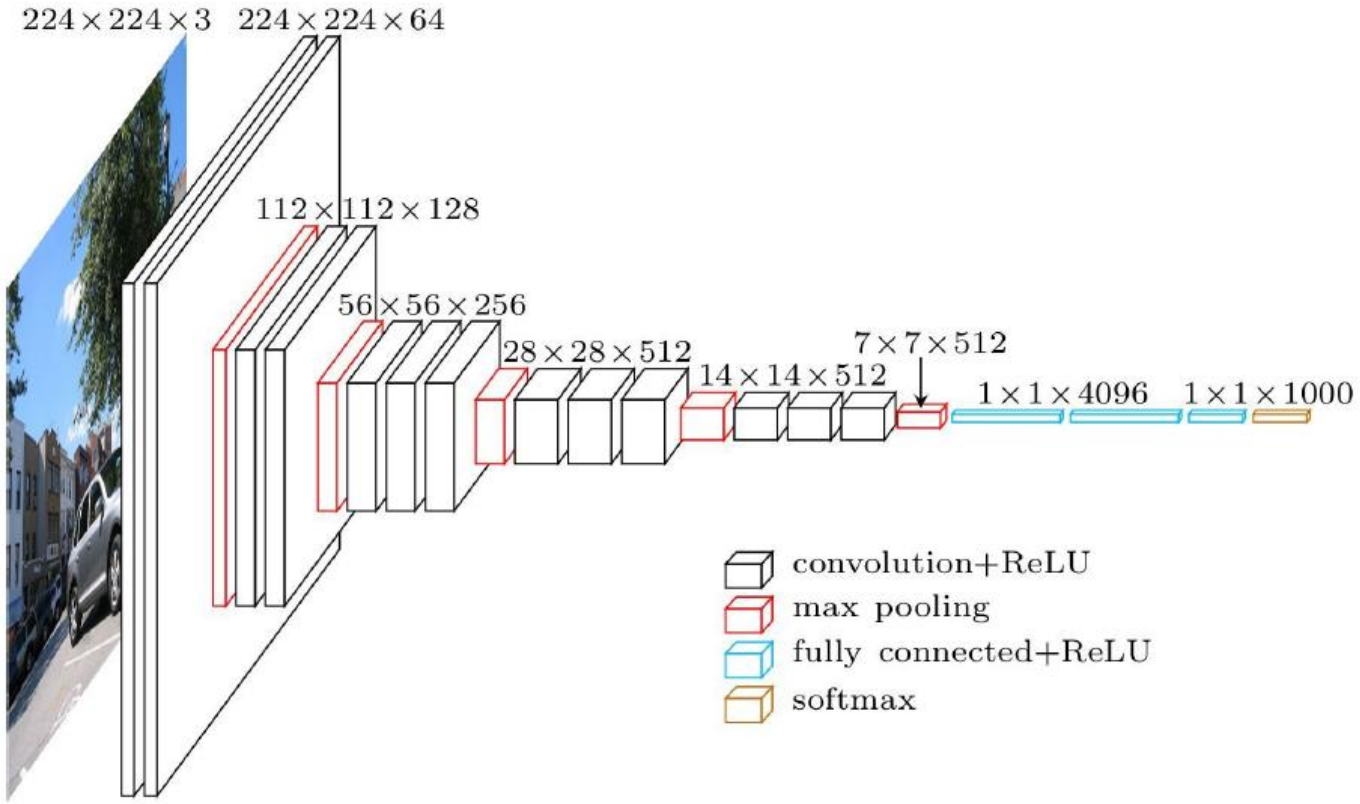


source

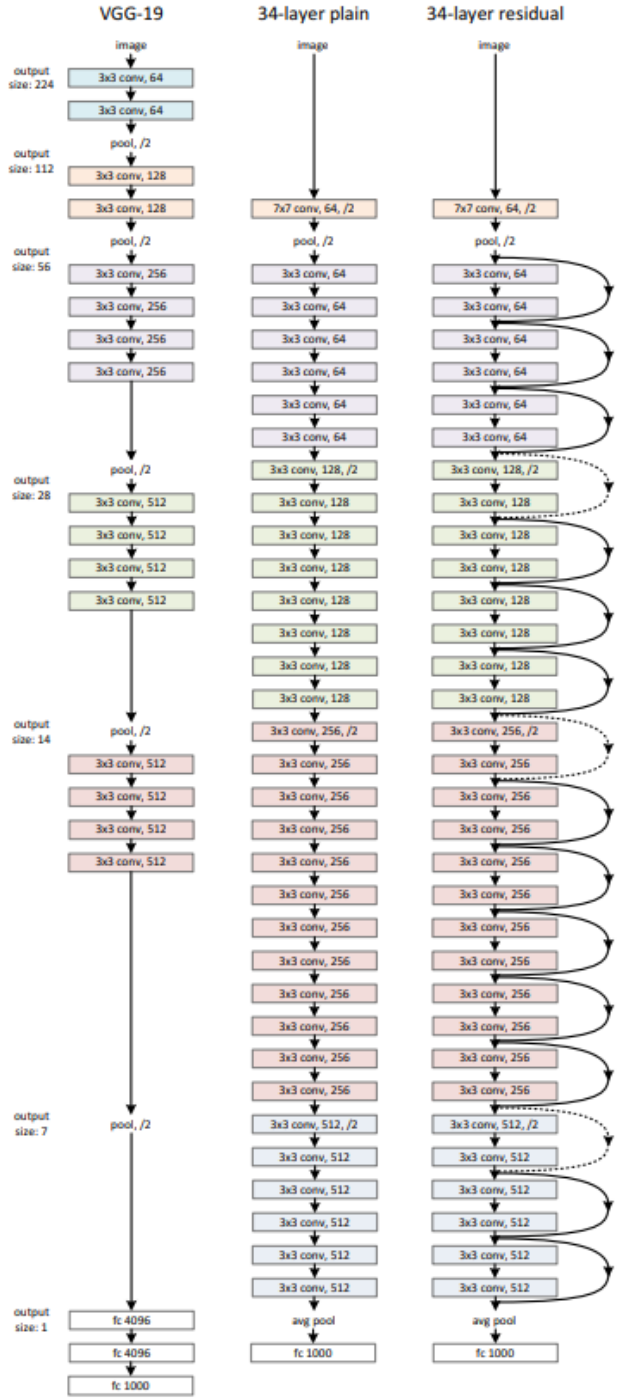



ImageNet Classification top-5 error (%)
152 layers
22 layers
19 layers
8 layers
8 layers
shallow
3.57
6.7
7.3
11.7
16.4
25.8
28.2
ILSVRC'15 ResNet
ILSVRC'14 GoogleNet
ILSVRC'14 VGG
ILSVRC'13
ILSVRC'12 AlexNet
ILSVRC'11
ILSVRC'10

VGG


224 × 224 × 3
224 × 224 × 64
112 × 112 × 128
56 × 56 × 256
28 × 28 × 512
14 × 14 × 512
7 × 7 × 512
1 × 1 × 4096
1 × 1 × 1000
convolution+ReLU
max pooling
fully connected+ReLU
softmax

ResNet


VGG-19
34-layer plain
34-layer residual
image
output size: 224
3x3 conv, 64
pool, /2
output size: 112
3x3 conv, 128
output size: 56
3x3 conv, 256
output size: 28
3x3 conv, 512
output size: 14
output size: 7
fc 4096
fc 1000
7x7 conv, 64, /2
3x3 conv, 128, /2
3x3 conv, 256, /2
3x3 conv, 512, /2
avg pool

Skip Connections

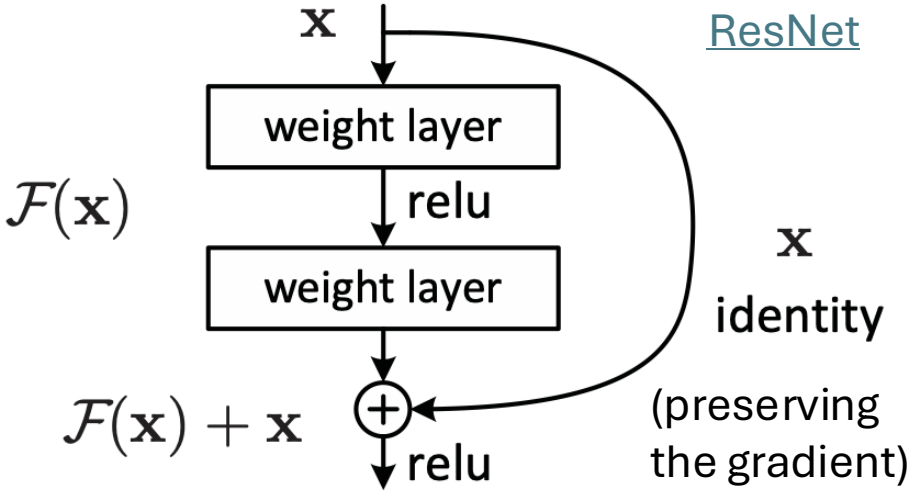
issue: degradation of training and test errors when adding more and more layers → not due to overfitting (but reason controversial)

solution: learning of residuals by means of skip connections (resulting in combination of different paths through computational graph)

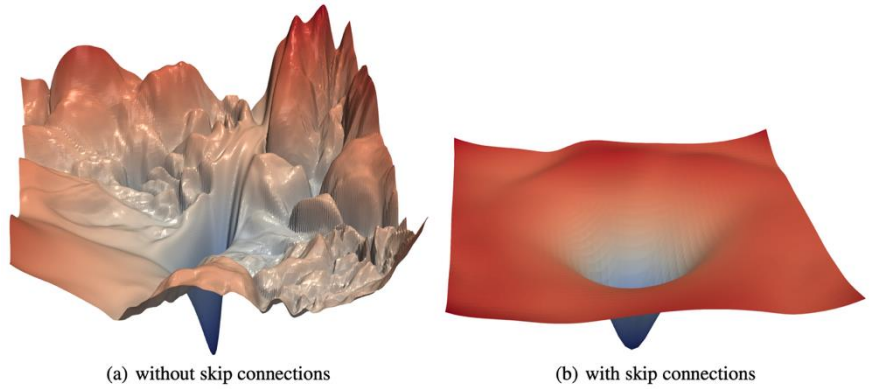
→ produces loss functions that train easier

together with batch normalization (avoiding exploding gradients), skip connections enable extremely deep networks (>1000 layers) without degradation

residual mapping (special kind of skip/shortcut connections):



loss surface:



source

MobileNet

standard convolution: computes spatial filtering and channel mixing (color combinations) simultaneously

example: input image with 10x10x3 → 25 filter blocks of size 3x3x3 → output: 25 channels

→ total weights: 25 × (3 × 3 × 3) = 675 weights

depthwise separable convolution (MobileNet approach): splits the operation into two dedicated, lightweight steps

1. depthwise convolution (spatial filtering)
   - each input channel is filtered independently (no mixing)
   - example: 3 Channels → 3 flat filters of size 3x3 → 3 filtered channels
2. pointwise convolution (channel combination)
   - 1x1 convolution mixes the filtered channels into the final output
   - example: 3 filtered channels → 25 filters of size 1x1x3 → output: 25 channels

   → total weights: 27 (step 1) + 75 (step 2) = 102 weights

massive resource savings (fewer computations): approaching 90% in deep networks

MobileNet

coloring book analogy:

- standard conv: 25 artists drawing and coloring 25 unique images entirely from scratch
- MobileNet: one artist draws the outlines (depthwise), then 25 assistants color them differently (pointwise)

why it works: spatial forms (shapes) and channel information (colors) largely independent in real-world data

slight drop in accuracy can be compensated by deeper networks (layers are cheap)

Transfer Learning

Foundation Models

problem with ConvNets trained from scratch (in fact, with all ML methods): only suited for domain of training data

idea of transfer learning:

pretrain a big model (called foundation model) on a broad data set, and then use these learnings for subsequent trainings on specific (typically, narrow) data

two forms of transfer learning: feature extraction and finetuning

Feature Extraction

approach:

- get a pretrained ConvNet as foundation model
- remove final fully-connected layer (its outputs are the class scores from the pretraining task)
- pass training data (for the task at hand) through the network
- for each image, extract vector of last hidden layer‘s activations (e.g., 4096-dimensional for AlexNet)
- use the extracted values as features to train another model (to be adjusted to the task at hand)

(same idea as used before for SIFT features)

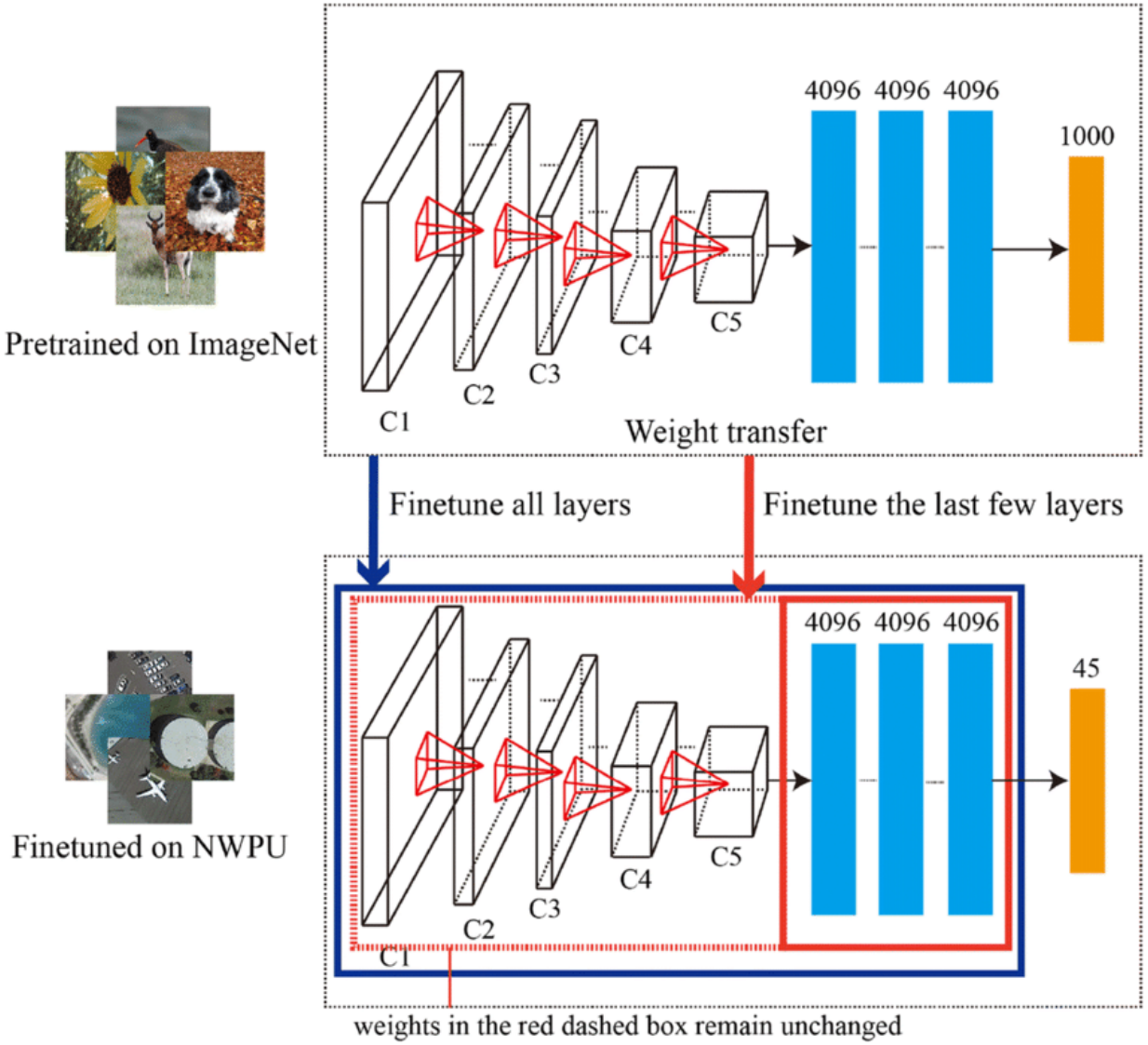
Finetuning

approach:

- get pretrained ConvNet as foundation model
- continue (starting with weights after pretraining) training with data for task at hand

typically, need to change architecture of final layer: different number of classes

options: finetune all weights or just some



source

Finetuning Scenarios

finetuning gives better adjustment to new task than feature extraction (important if new task differs a lot from pretraining task)

→ coming along with danger of overfitting

ConvNet features more generic in early layers (e.g., edges) and more specific to the data set of pretraining in later layers

→ for small finetuning data sets, typically keep weights of early layers fixed to avoid overfitting

for large finetuning data sets (less danger of overfitting), finetuning all layers can be beneficial

Brief Recap of Different Image Classification Techniques

- Hough transform: looking for predefined shapes such as lines or circles (feature-based recognition)
- feature description & matching: identify specific object instances (typically used for instance rather than class recognition)
- image features as input to classic ML method: generalization from data
- plain feed-forward network on raw image data: even more generalization without handcrafted components
- convolutional neural networks: use of spatial structure (inductive bias)
- pretraining & finetuning (or feature extraction): transfer learning

Transformer

Large Language Models (LLM)

recent hype around LLMs

fully started with ChatGPT release end of 2022

technical backbone: transformer architecture

transformers also applicable to computer vision (alternative to convolutional neural networks)

Tokenization

tokenization: *breaking text in chunks*

- word tokens: different forms, spellings, etc → undefined and vast vocabulary
- character tokens: not enough semantic content (longer sequences)
- popular compromise: byte-pair encoding

Byte-Pair Encoding

data compression method used for encoding text as sequence of tokens (sub-words)

- merging token pairs (starting with characters) with maximum frequency
- continue merging until defined fixed vocabulary size (hyperparameter) is reached

→common words encoded as single token

→rare words encoded as sequence of tokens (representing word parts)

alternative: direct operation on bytes (e.g., ByT5)

aaabdaaabac

ZabdZabac
Z=aa

ZYdZYac
Y=ab
Z=aa

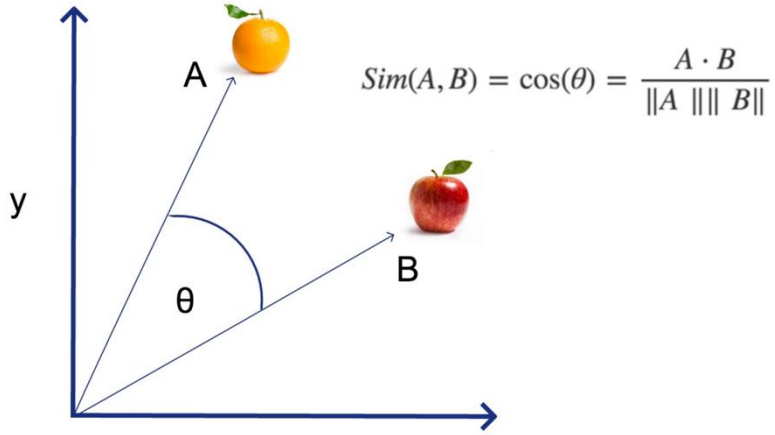
XdXac
X=ZY
Y=ab
Z=aa

example from wikipedia

Embedding Layers

representation of entities by vectors

→ semantic similarity



most famous application: word embeddings

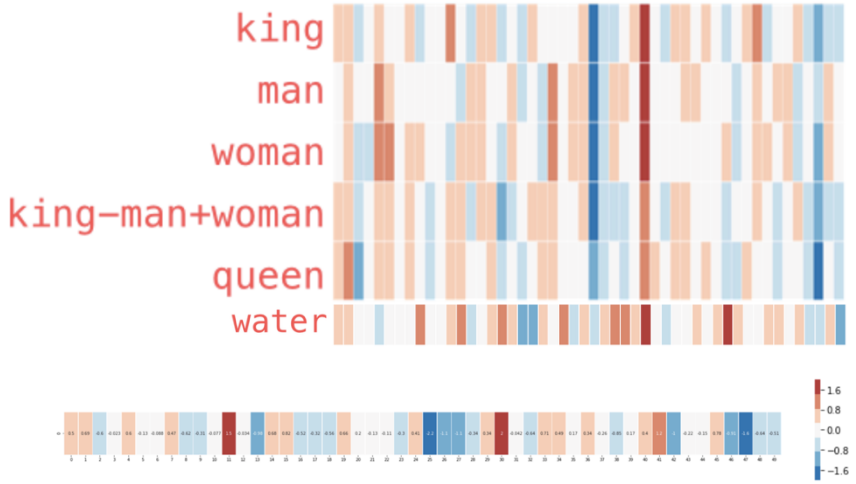
→ associations (natural language processing)

but general concept: embeddings of (categorical) features (e.g., products in recommendation engines)

learned via co-occurrence (e.g., word2vec)

but also direction of difference vectors interesting (analogies):

king − man + woman ~= queen



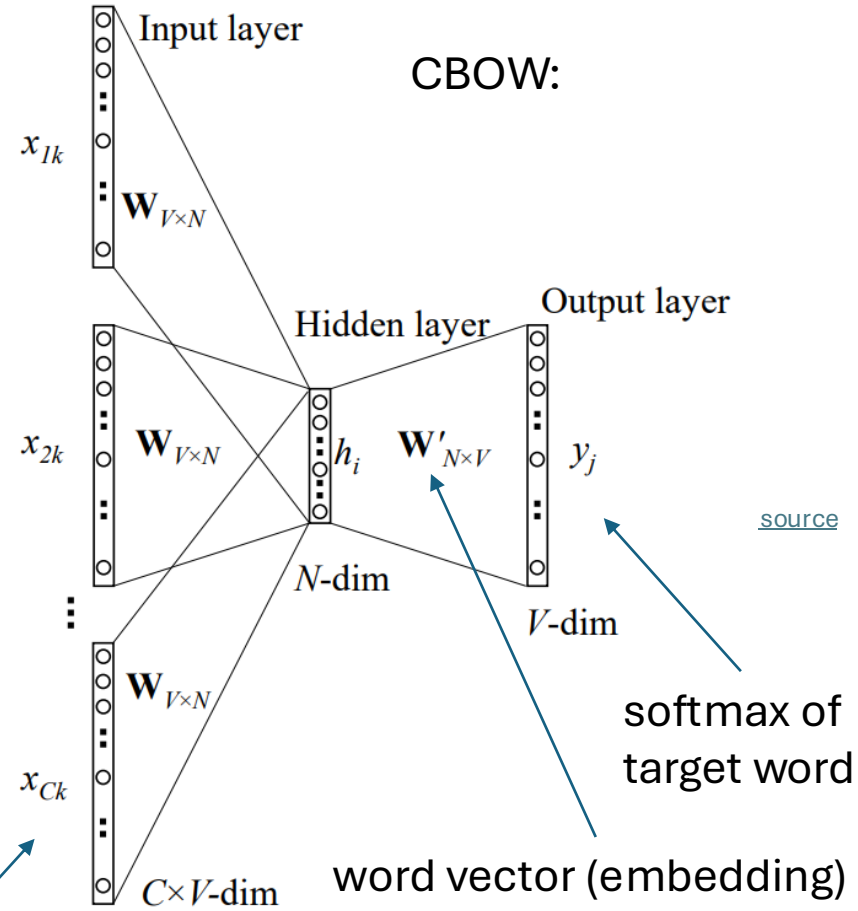
source

Some Thoughts on Word Embeddings

can be implemented as

- neural network with single hidden layer (linear activation)
- using, e.g., bag-of-words approach (predict masked word from its surroundings)

→ not context-aware



Word Embeddings as Part of Language Model

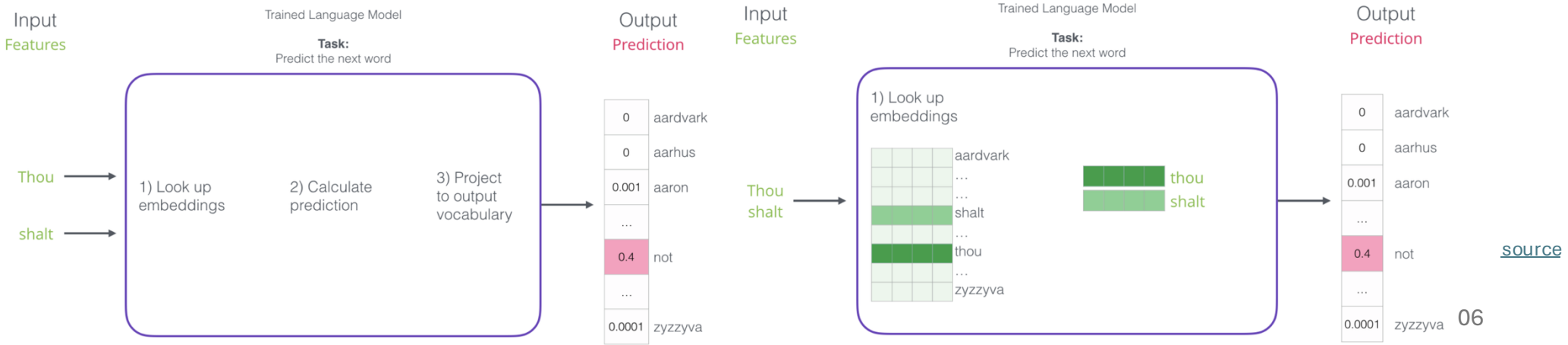
language models contain embedding matrix as part of learned parameters

typically several hundred dimensions for word vectors (to be compared with vocabulary sizes of many thousands)

(can also be extracted and subsequently used as pretrained embeddings for other task)

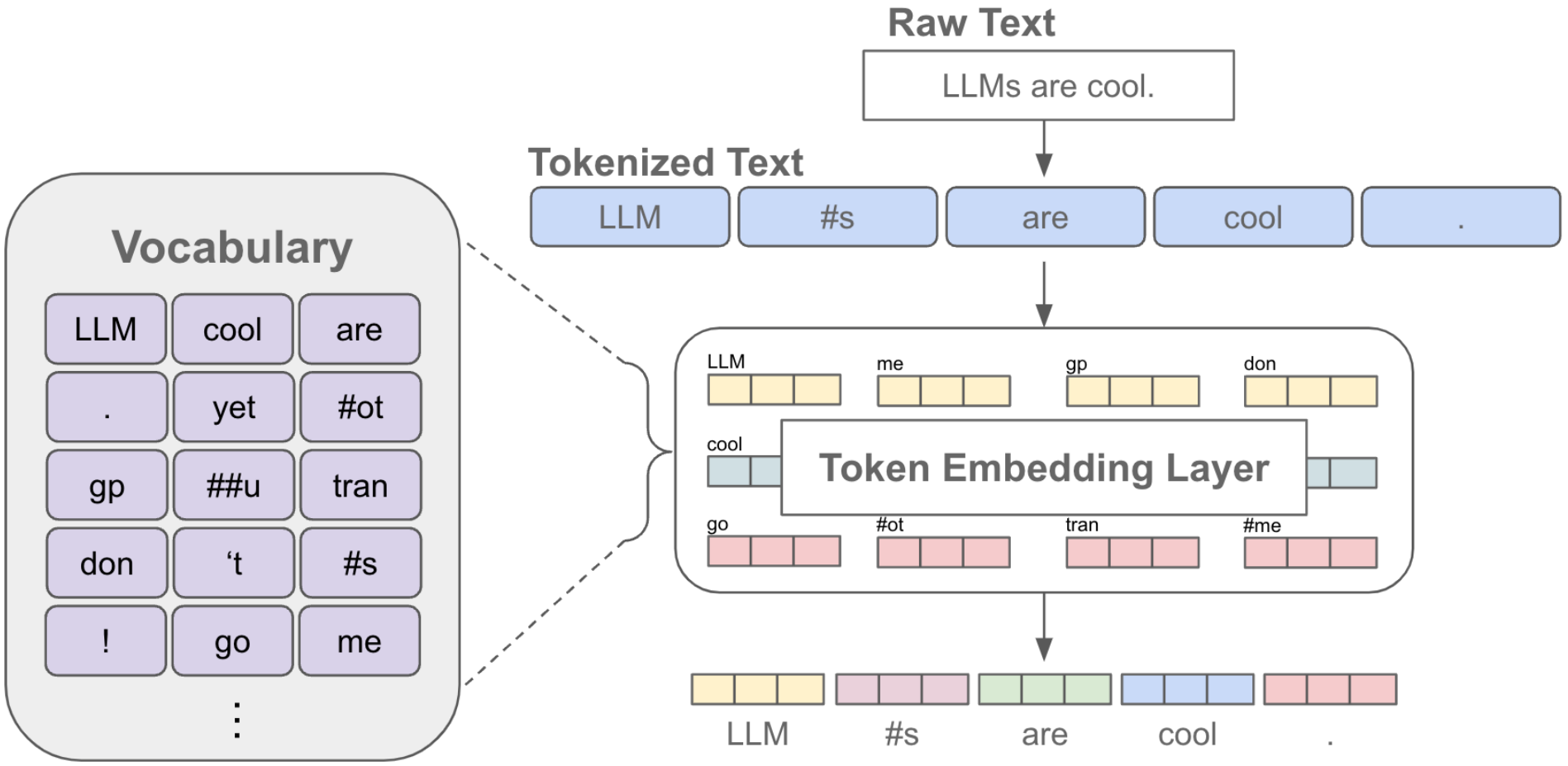


next-word prediction:



source

Tokenization & Embeddings



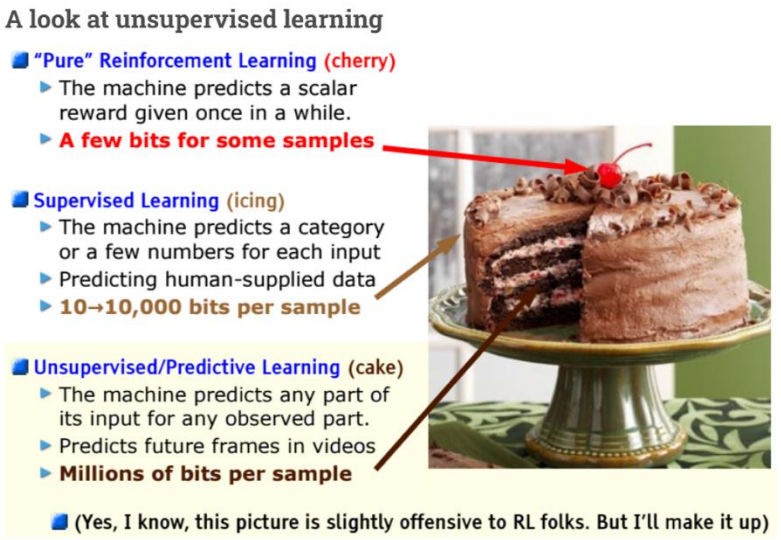
source

Self-Supervised Learning

ML needs lots of training data

unsupervised learning (learning by observation)

no target information → kind of "vague" pattern recognition (but plenty of data)



source

can be cast as **self-supervised learning**:

- input-output mapping like supervised learning
- but generating labels itself from inputs

`The Lord of the Rings is considered one of the greatest` [ ]

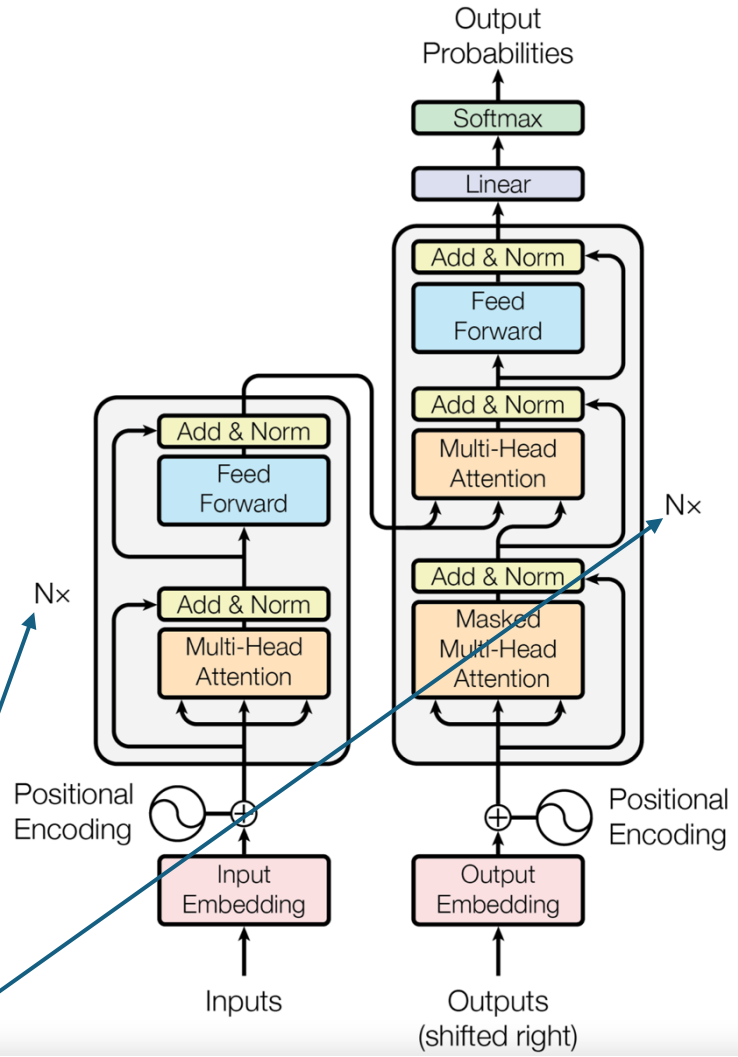
`The Lord of the Rings is considered one of the greatest fantasy` [ ]

⋮

now run this across the entire internet …

Context Awareness: Transformer

original transformer:
sequence-to-sequence model
(e.g., for machine translation)



[Attention Is All You Need]:

completely replaced recurrent neural networks (RNN)

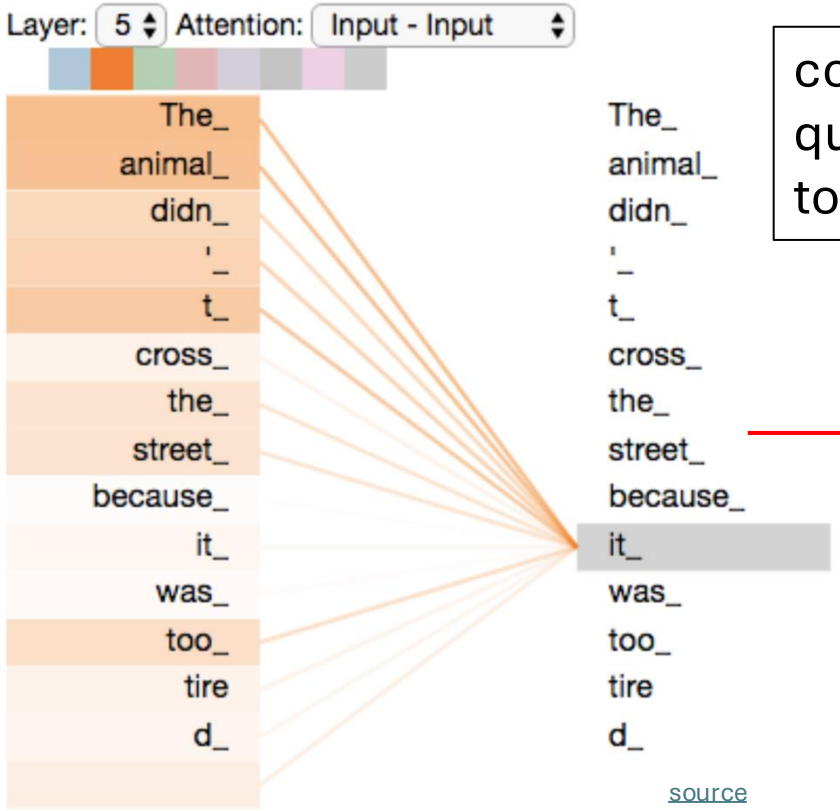
much more parallelization → bigger models

better long-range dependencies thanks to shorter path lengths in network (less sequential operations)

encoder/decoder stacks (depth):
output fed as input to next one

Self-Attention Mechanism

evaluating other input tokens in terms of relevance for encoding of given token

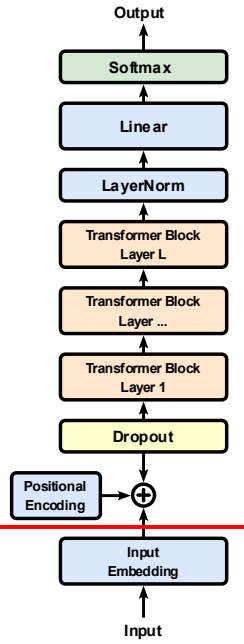


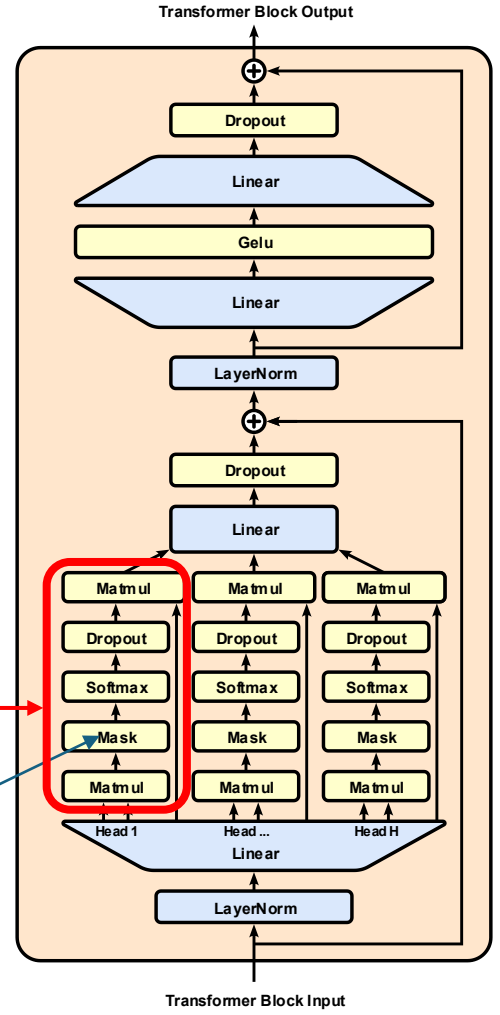
source

computational complexity quadratic in length of input (each token attends to each other token)

only allow to attend to earlier positions in sequence (masking future positions)

GPT (decoder-only):





Scaled Dot-Product Attention

softmax not scale invariant (largest component dominates for large scale) and dot product larger for more vector dimensions

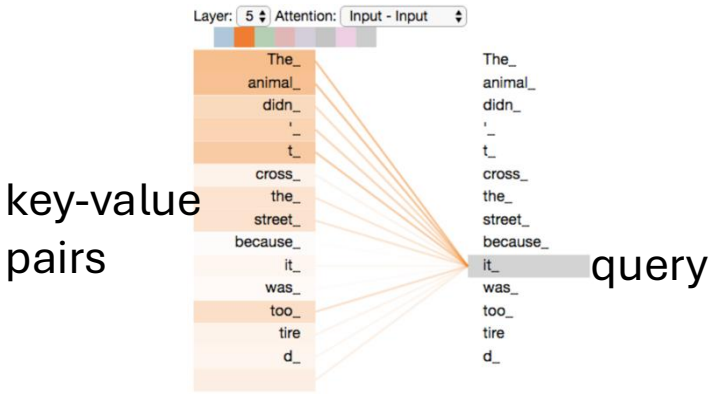
$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

Scaled Dot-Product Attention

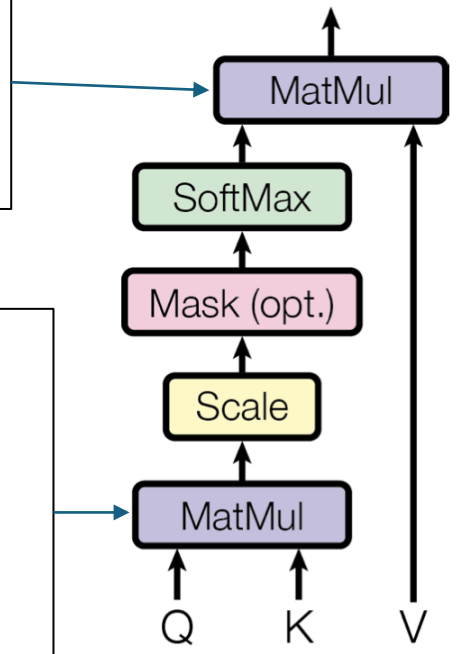
3 matrices created from input embeddings by multiplication with 3 different weight matrices

- query Q
- key K
- value V

filtering: multiplication of attention probabilities with corresponding key-word values



scoring each of the key words (context) with respect to current query word: correlation between inputs (not independent as in conventional neural networks)

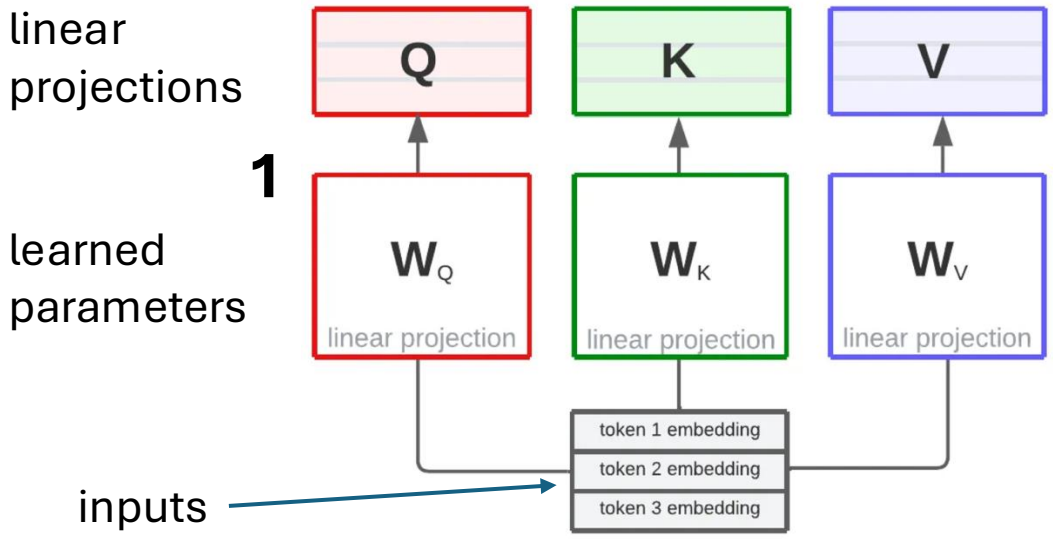


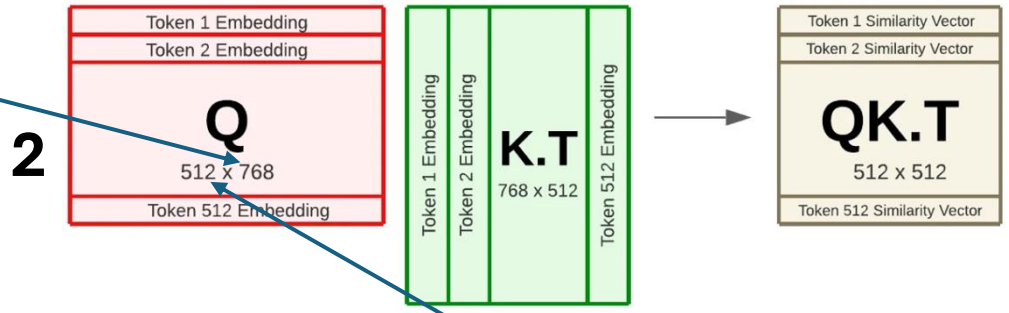
source

Example: Dimensions in BERT

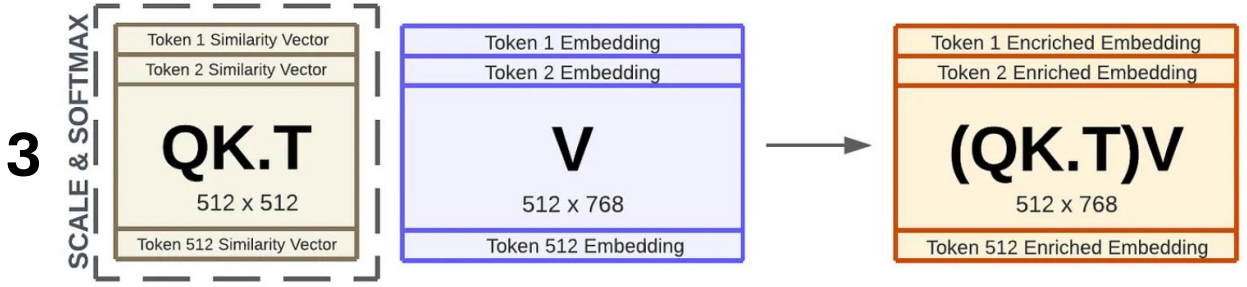
$$W_Q, W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$$

often $d_k = d_v = d_{\text{model}}$
(but not necessarily)



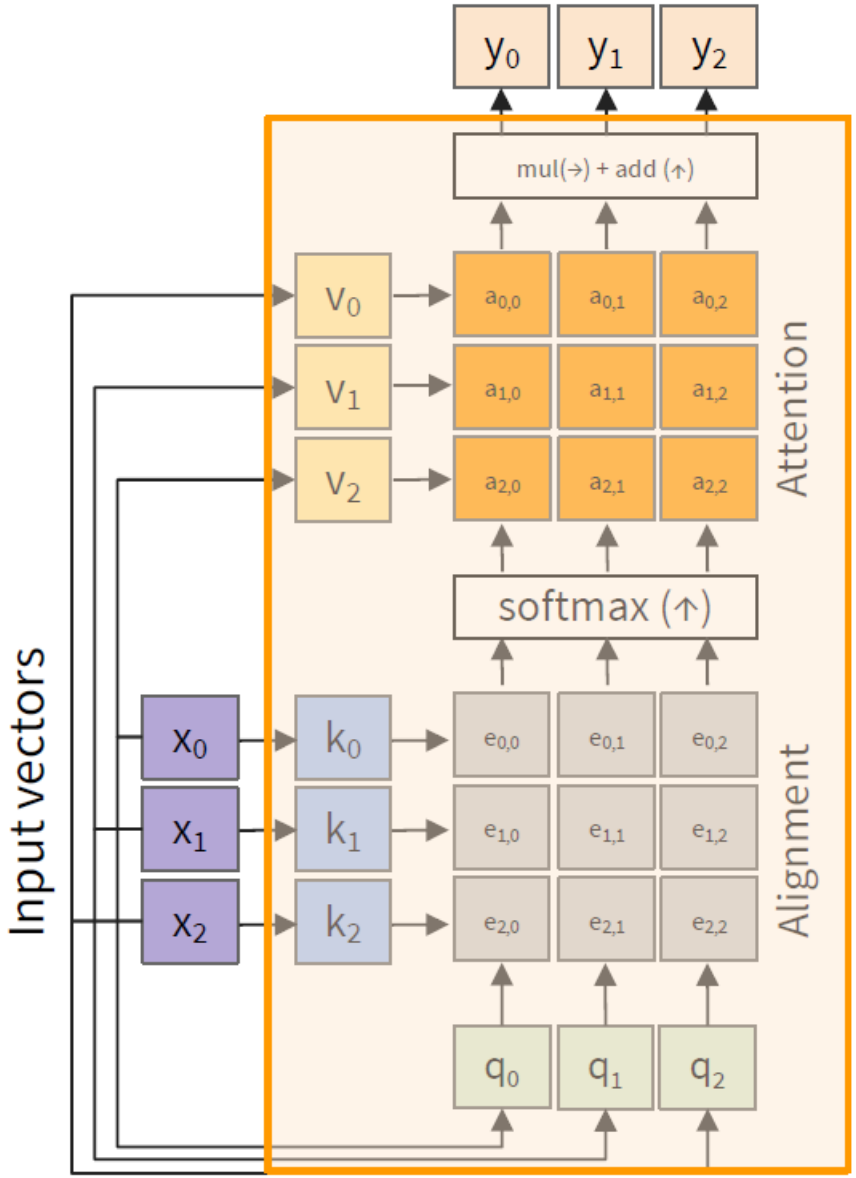


sequence length n
upper bound: context length of model
(design choice)



source

MatMul



Outputs:
context vectors: y (shape: D_v)

d_v

Operations:
Key vectors: $k = xW_k$
Value vectors: $v = xW_v$
Query vectors: $q = xW_q$
Alignment: $e_{i,j} = q_j \cdot k_i / \sqrt{D}$
Attention: $a = \text{softmax}(e)$
Output: $y_j = \sum_i a_{i,j} v_i$

d_k

d_{model}: embedding size

Inputs:
Input vectors: x (shape: N x D)

weighted average: reflecting to what degree a token is paying attention to the other tokens in the sequence

Attention Mask

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V$$

attention mask

causal masking:

dynamically mask future positions in sequence by setting to $-\infty$

$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Padding Mask

goal: (technically) batch together sequences of different lengths

→ include fake tokens to pad shorter ones to the max length in the batch

then add a padding mask to let the model ignore these fake tokens:

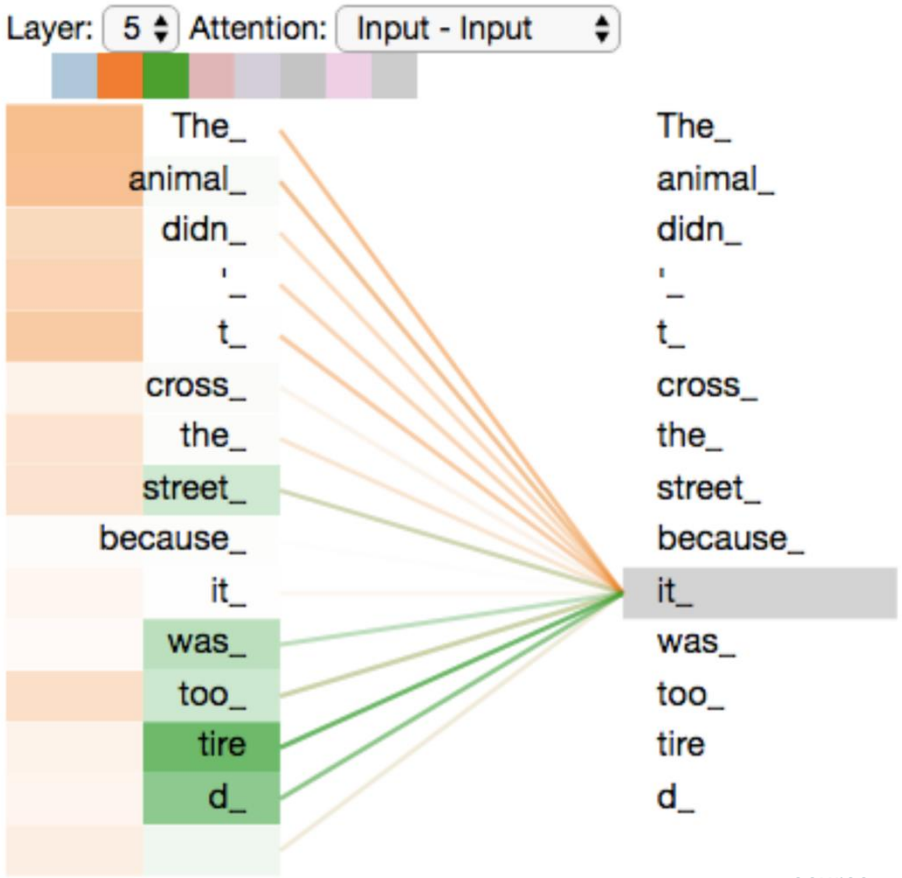
$$M = M_{\text{causal}} + M_{\text{padding}}$$

$$M_{\text{padding}}(i, j) = \begin{cases} 0 & \text{if token } j \text{ is valid} \\ -\infty & \text{if token } j \text{ is padding} \end{cases}$$

Multi-Head Attention

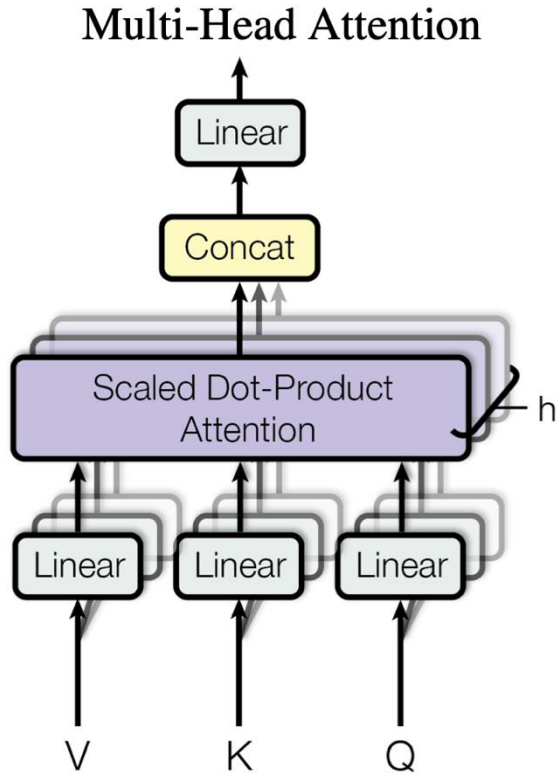
often $d_k = d_v = d_{\text{model}}/h$
and all h heads share the same d_k and d_v

multiple heads: several attention layers running in parallel



source

different heads can pay attention to different aspects of input (multiple representation sub-spaces)



source

Positional Encoding

defines context length

attention permutation invariant → need for positional encoding to learn from order of sequence

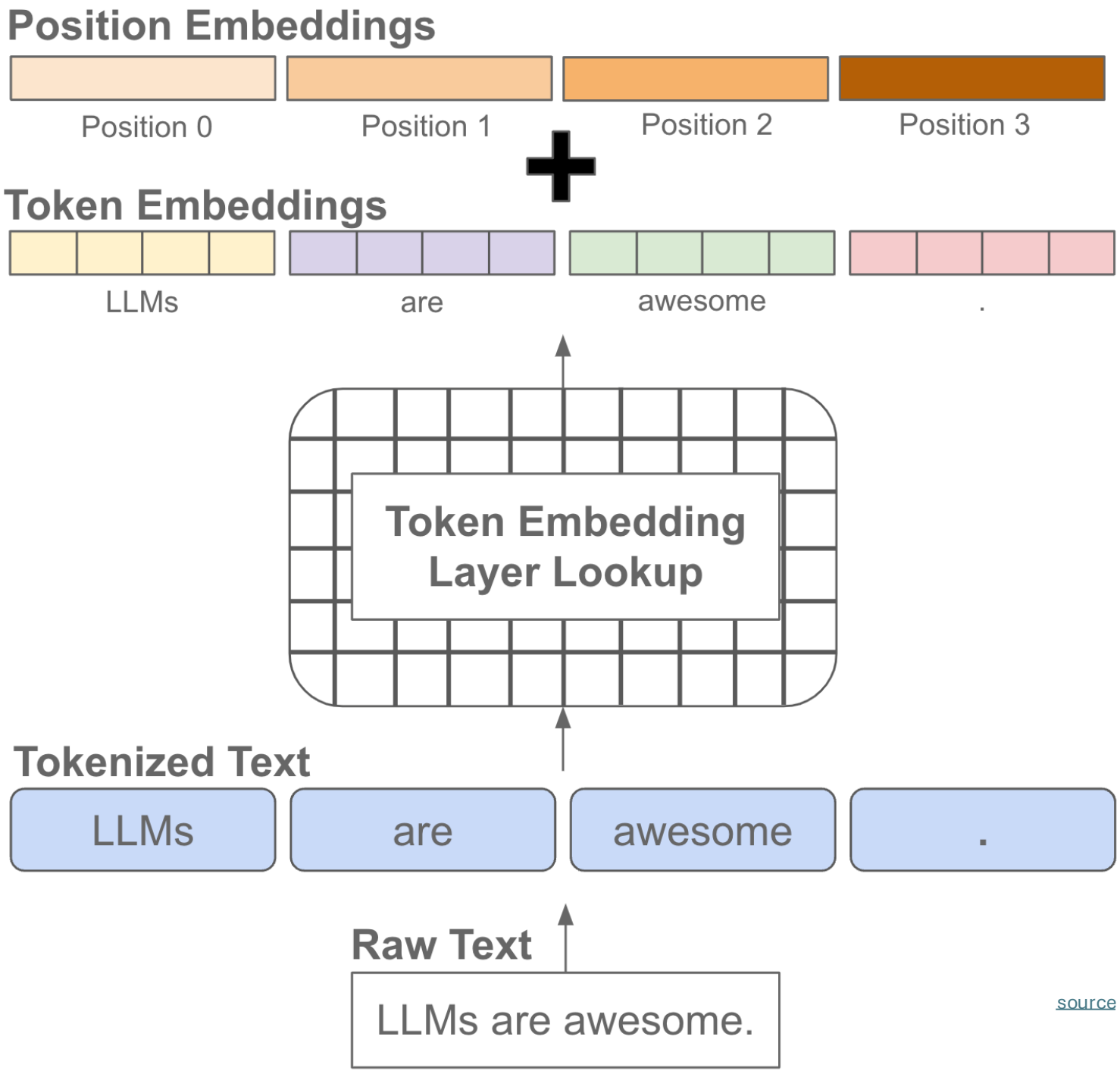
different options:

- for each absolute position (from 1 to maximum sequence length), add a learned vector (of dimension d_{model}) to input embeddings → each positional embedding independent of others
- add fixed sinusoidal functions for each position and dimension i

$$PE_{pos,2i} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right) \qquad PE_{pos,2i+1} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

- rotate input embeddings by multiples (position) of a small angle (RoPE) → captures both relative and absolute positions

$$f_{\{q,k\}}(\boldsymbol{x}_m, m) = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} W^{(11)}_{\{q,k\}} & W^{(12)}_{\{q,k\}} \\ W^{(21)}_{\{q,k\}} & W^{(22)}_{\{q,k\}} \end{pmatrix} \begin{pmatrix} x^{(1)}_m \\ x^{(2)}_m \end{pmatrix}$$


Position Embeddings
Position 0
Position 1
Position 2
Position 3
Token Embeddings
LLMs
are
awesome
.
Token Embedding Layer Lookup
Tokenized Text
LLMs
are
awesome
.
Raw Text
LLMs are awesome.

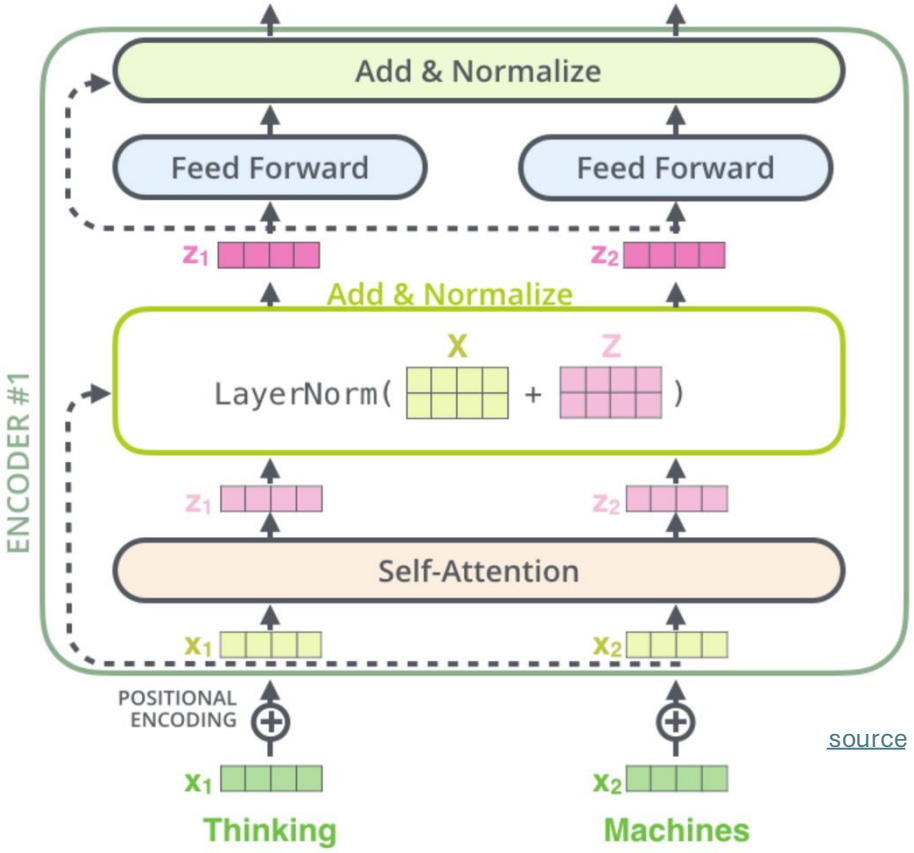
source

Skip Connections and Layer Normalization

skip connections improve robustness by

- preserving original input (attention layers as filters) as well as gradients (mitigate vanishing-gradient problem)
- easier learning of identity functions (useful for disregarding modules that do not improve model performance)

layer normalization improves stability, convergence, and generalization by normalizing activations per input token

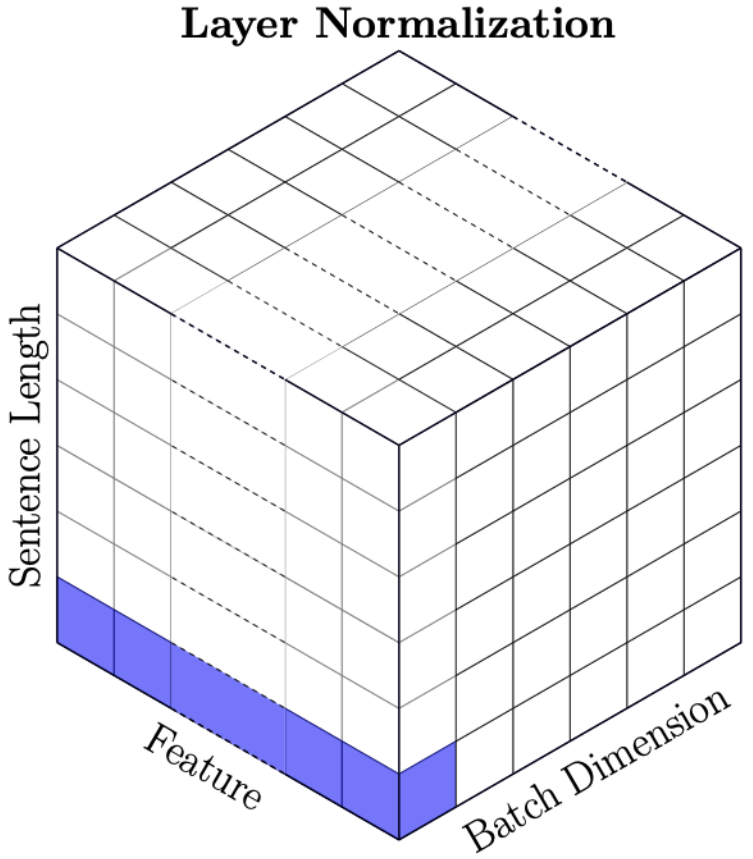


source

Batch vs Layer Norm

batch norm typically in computer vision

layer norm in NLP (variable-sized inputs)

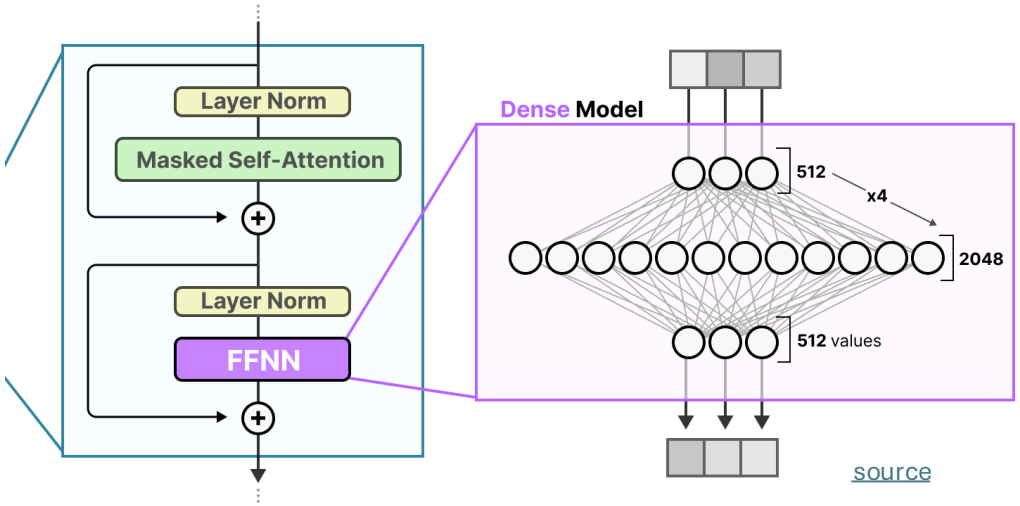


Role of Feed-Forward Neural Networks (FFNN)

position-wise FFNNs:

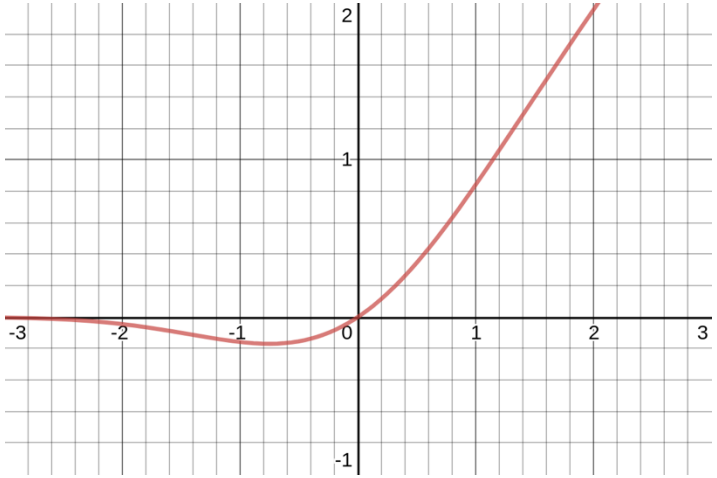
following each attention layer, apply an identical FFNN (with two layers) independently to each embedding vector

(correspond to bulk of overall parameters)



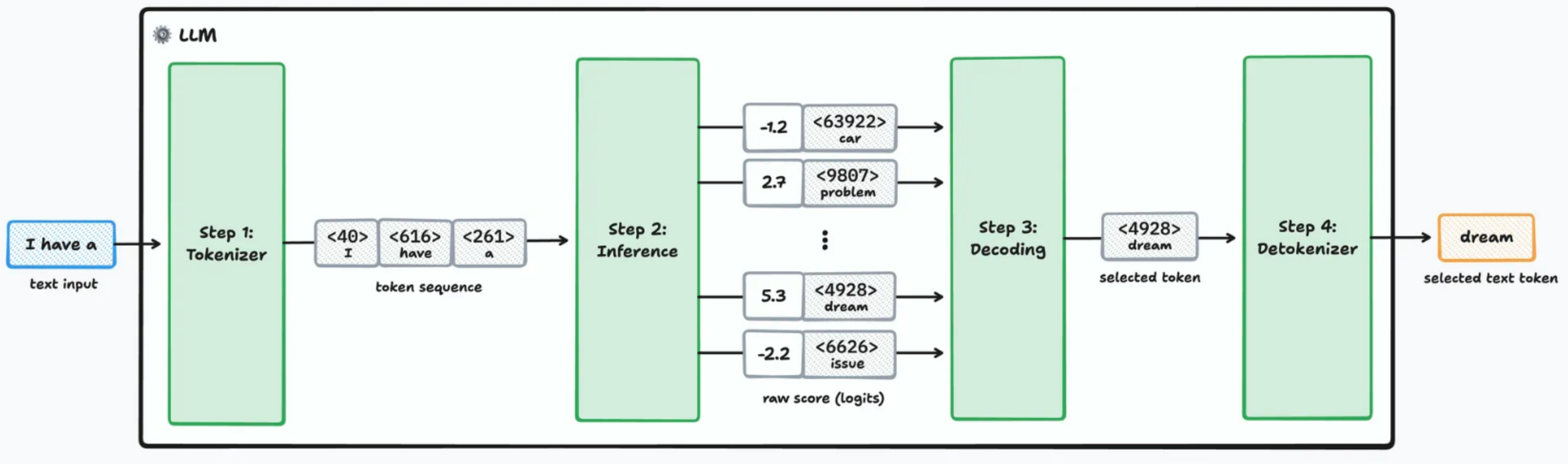
can be interpreted as compute after connect (attention)

attention is just weighted averaging
→ need for non-linearities (often GeLU activations) to capture more complex patterns



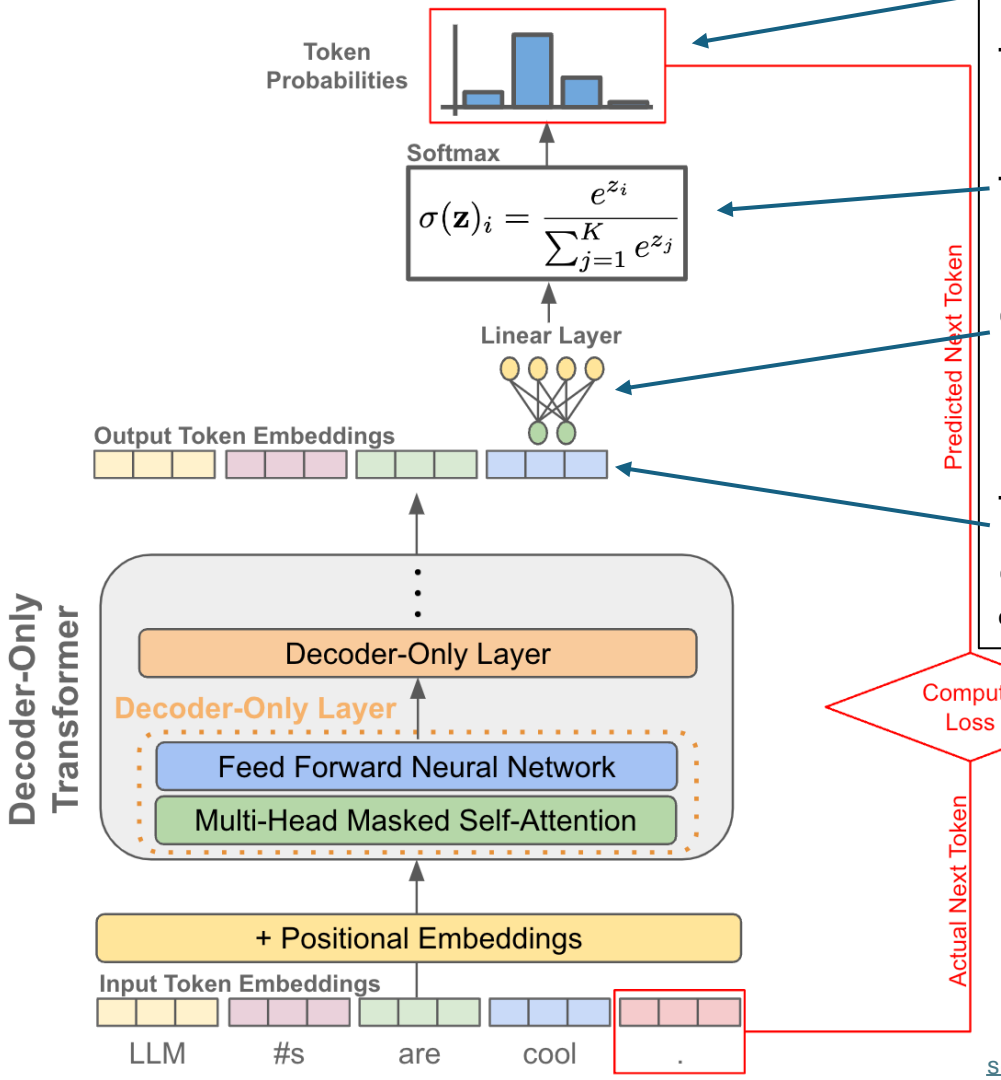
Next-Token Prediction

forward pass:



source

From Embeddings to Probabilities



next-token probabilities for each token in vocabulary

transformation to probabilities

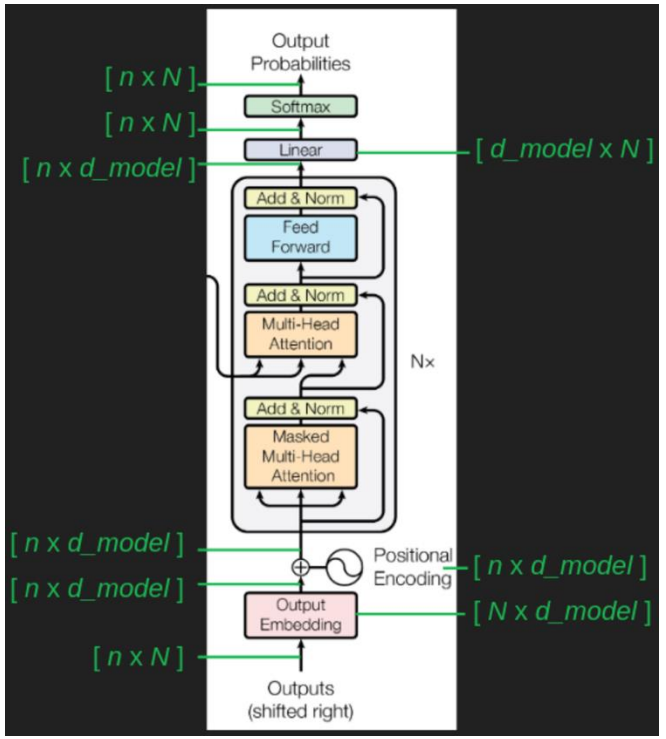
de-embedding to vocabulary: projection to vector of logits

for projection, only use contextual embedding of last token (still need all contextual embeddings due to matrix multiplication in self-attention)

(masked token for encoder-only transformers)

source

n: sequence length
N: vocabulary size
d_model: embedding size



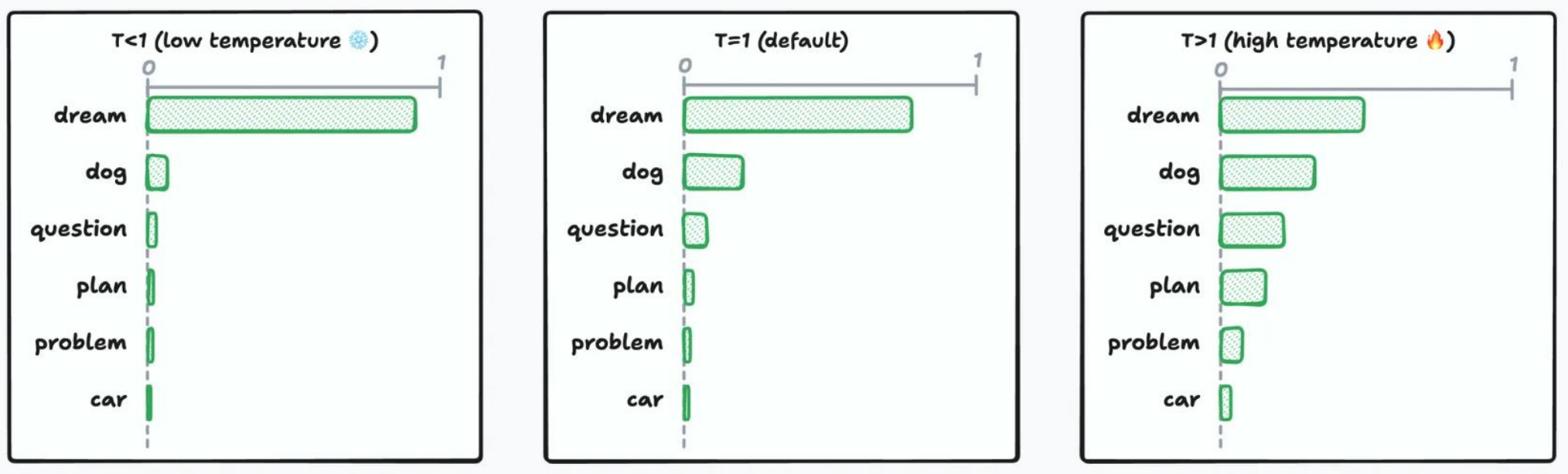
source

Token Selection at Inference

typically, pick according to probabilities

degree of randomness can be controlled by temperature parameter T

$$\text{softmax}(z_i) = \frac{e^{z_i/T}}{\sum_{j=1}^{n} e^{z_j/T}}$$

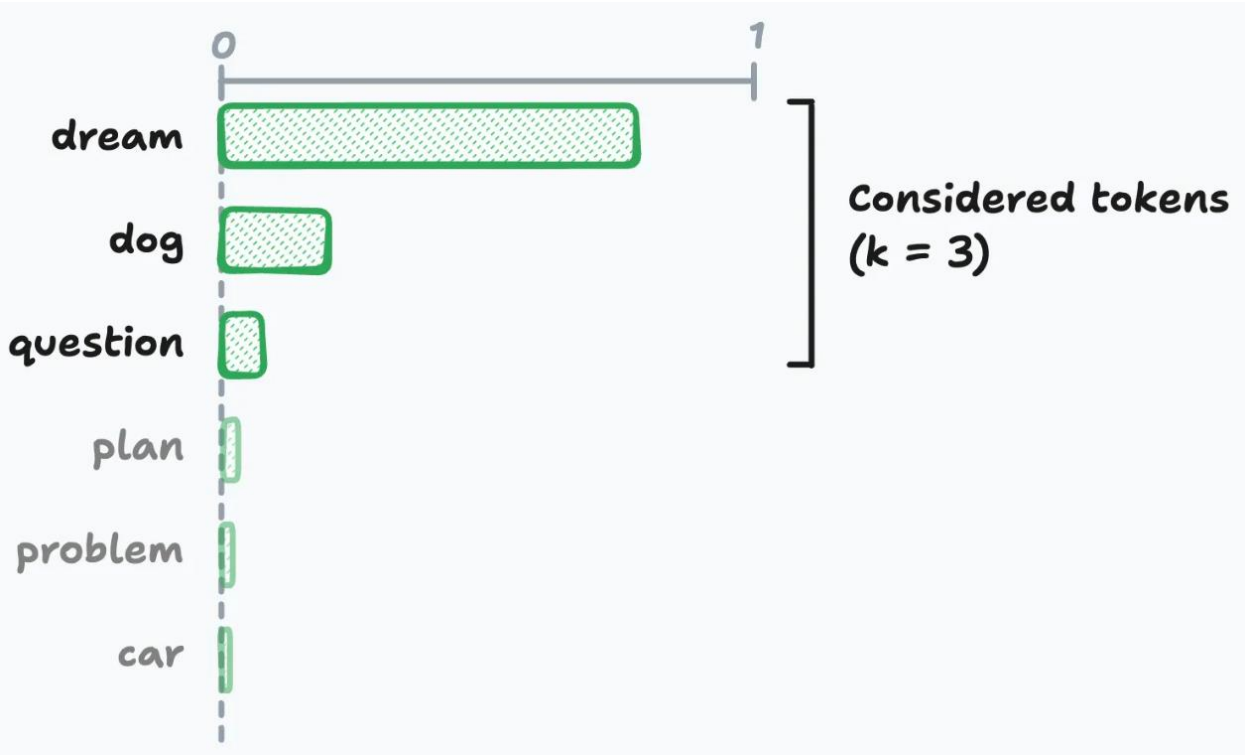


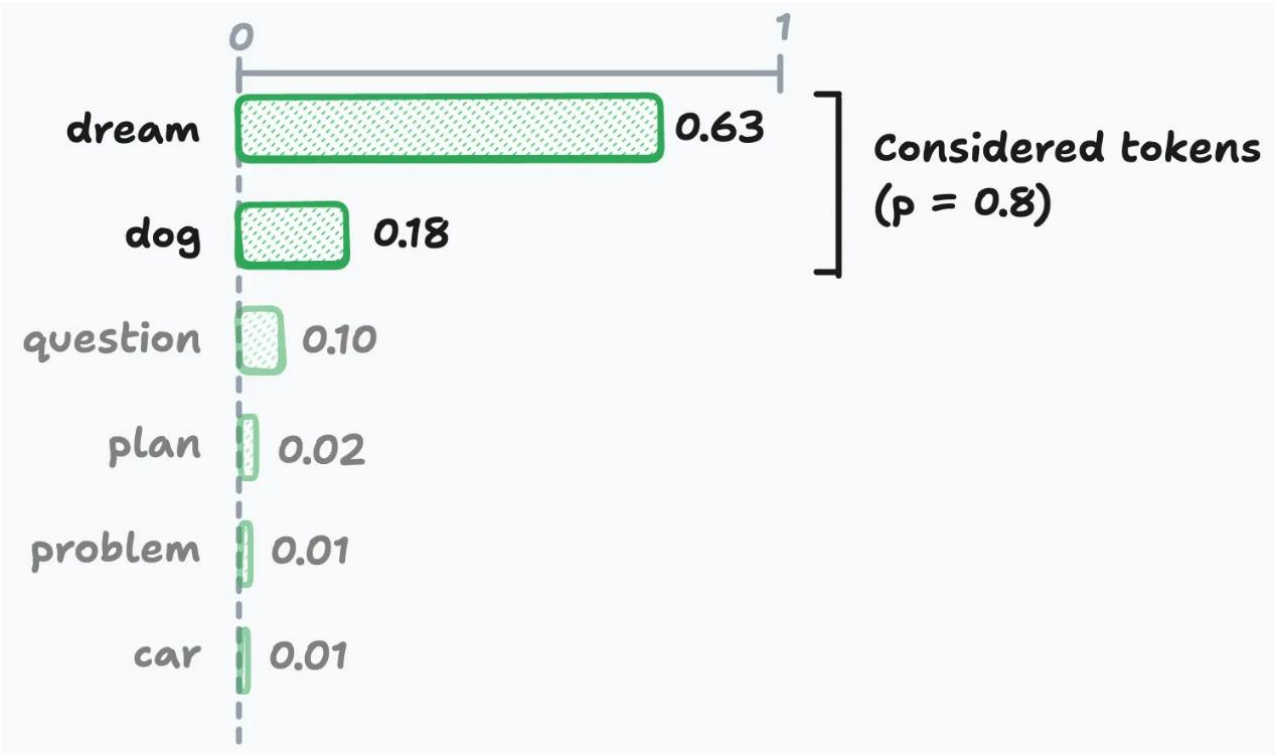
source

$T \to 0$ → greedy

creativity ;)

top-k & top-p Sampling



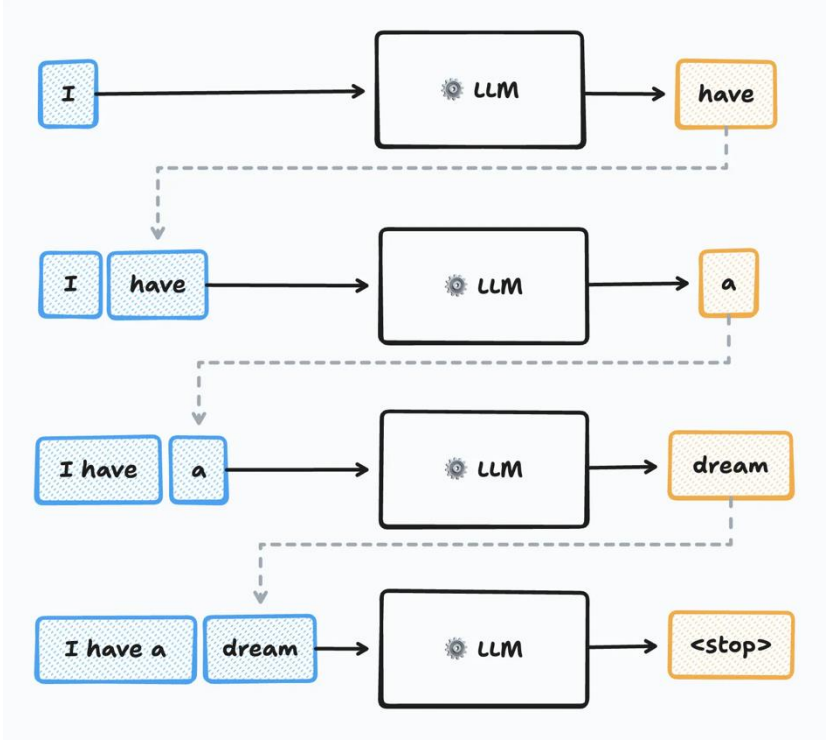


source

Sequence Completion

autoregressive:

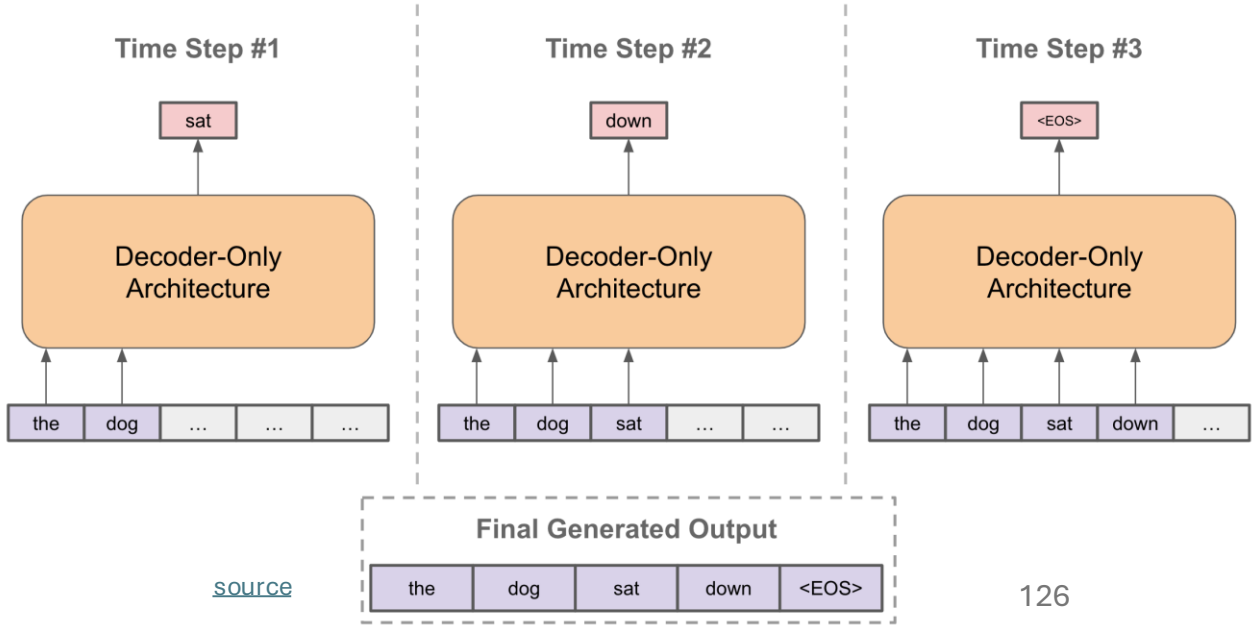
at each step, choose one output token, then add it to decoder input sequence for next step



source

prompt:

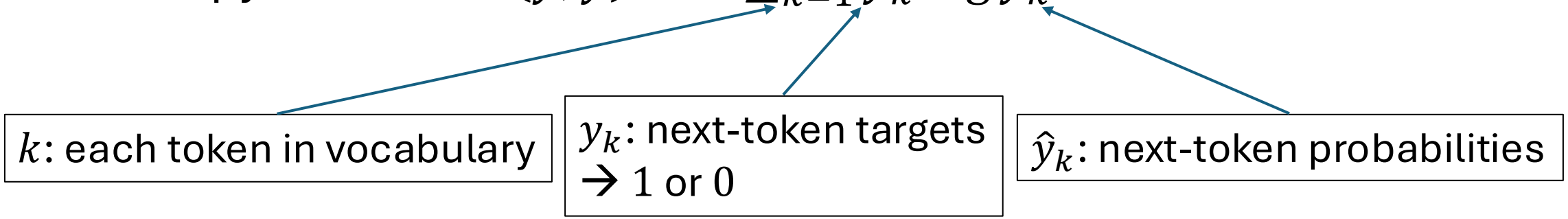
externally given initial sequence for running start and context on which to build rest of sequence



source

Training Loss

cross-entropy loss: $L(y, \hat{y}) = -\sum_{k=1}^{K} y_k \log \hat{y}_k$



during training, a next-token-prediction loss is calculated for every position in input sequence (not just the last one)
→ earlier positions have shorter effective contexts (masking of future positions)

reason: mirroring actual inference case of autoregressive text generation and arbitrary prompts

total loss = sum of individual losses

Teacher Forcing

in training, ground-truth previous tokens are used as inputs for next-token predictions from each position

(instead of own sampled predictions as in inference)

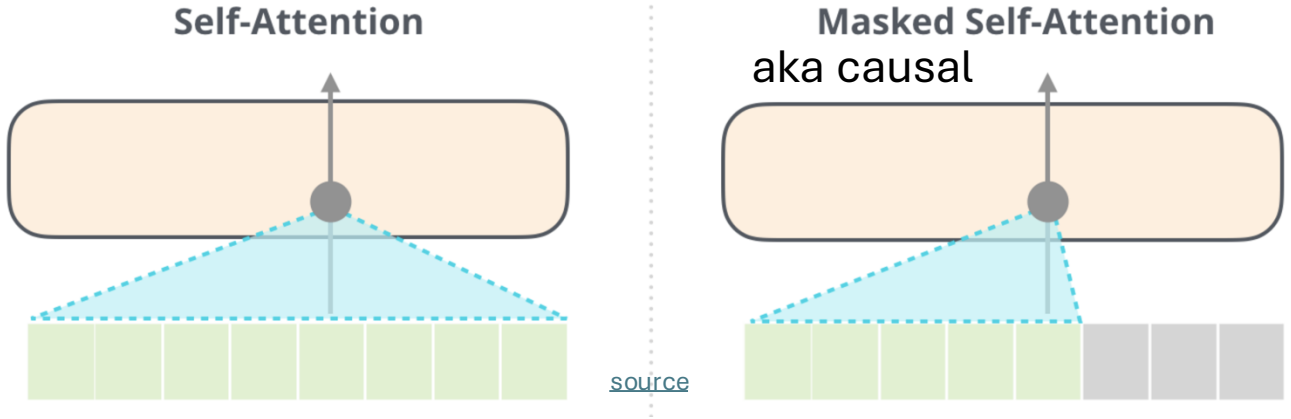
→ reduces variance and allows to compute an individual loss for every token in the sequence in parallel

Transformer Types

encoder-only transformers:

- goal: language understanding
- training: masked-token prediction

prime example:
BERT



source

decoder-only transformers:

- goal: text generation
- training: next-token prediction

prime example:
GPT

encoder-decoder transformers:

- goal: sequence-to-sequence models
- training: next-token prediction

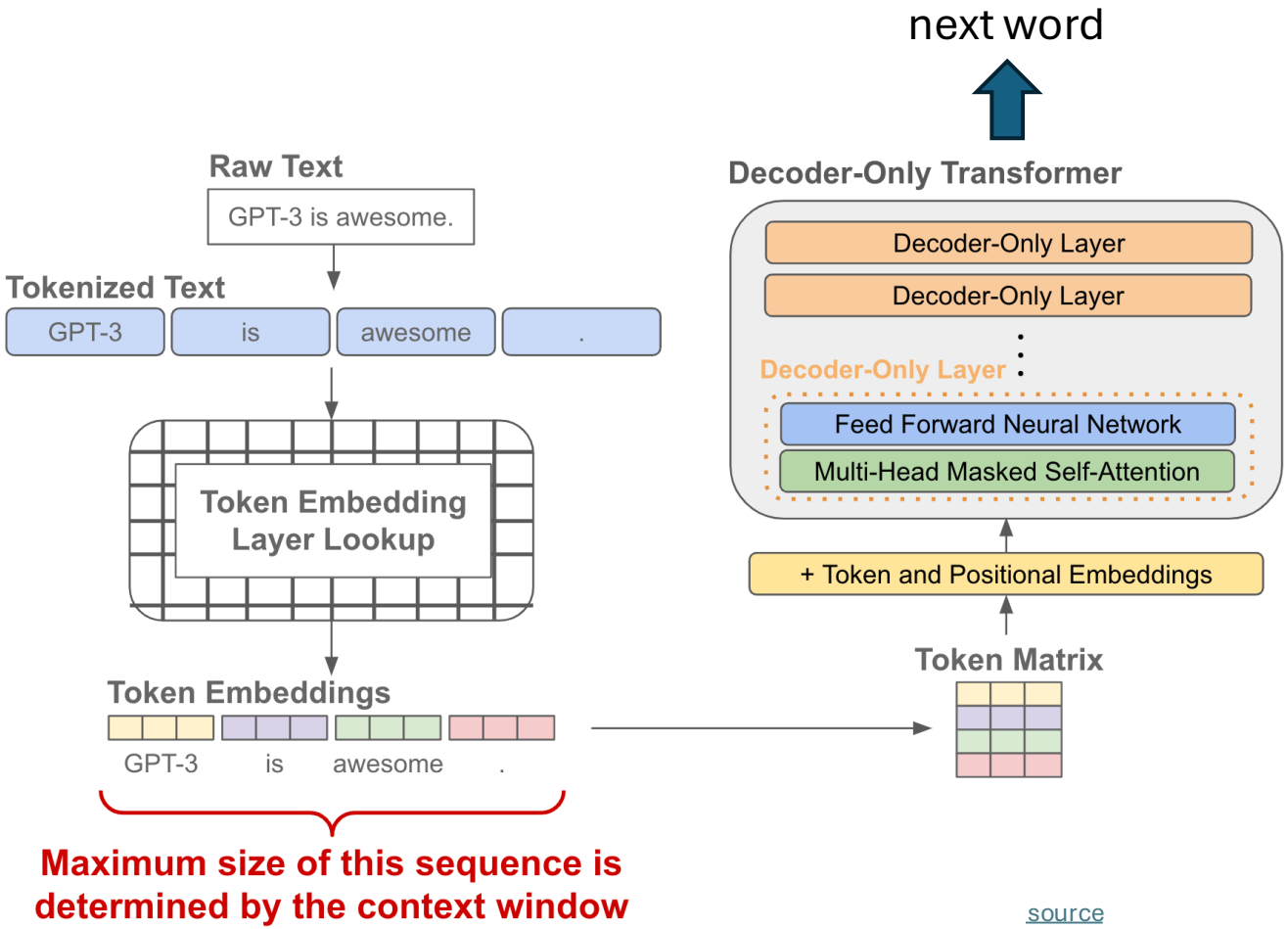
Decoder-Only = Autoregressive Models

today typically synonymous with LLM

stack of transformer decoders

→autoregressively generated sequence of tokens (directly usable as backbone of a chatbot)

training objective: next-token prediction from context



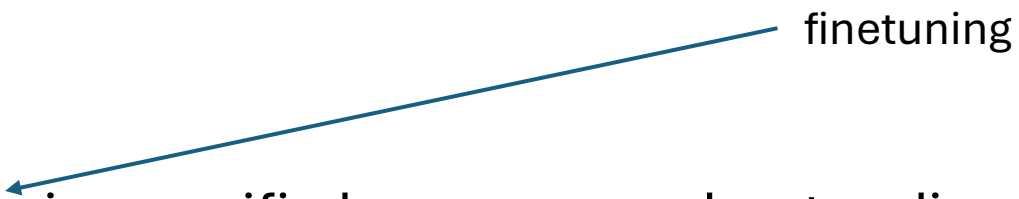
source

Encoder-Only = Bidirectional Models

training objective: masked tokens to be predicted from context

→ aka masked language models (MLM)

bidirectional: jointly conditioning on both left and right context



stack of transformer encoders

→ contextual embeddings (for later use in specific language-understanding tasks like text classification, sentiment analysis, named entity recognition)

still needs a decoder head for training purposes, using the hidden state of the masked token (dedicated [MASK] token) for projection to vector of logits

Generative vs Predictive/Discriminative Models

discriminative models:

predict conditional probability $P(Y|\boldsymbol{X})$

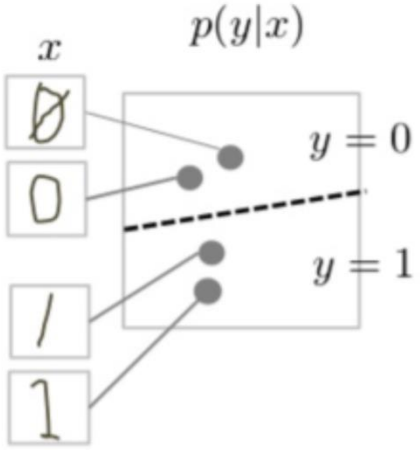
generative models:

predict joint probability $P(Y, \boldsymbol{X})$

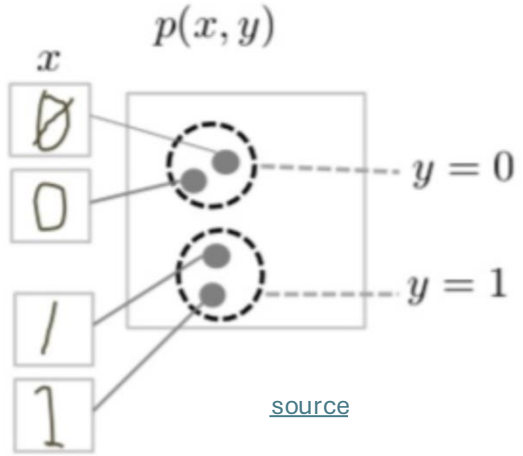
(or just $P(\boldsymbol{X})$ → unsupervised learning)

→ allow to generate new data samples

discriminative model



generative model



source

task of generative models more difficult: need to model full data distribution rather than merely find patterns in inputs to distinguish outputs

Generative models can be used for predictive tasks (Bayes theorem).
But predictive models are usually better at it.

Autoregressive Text Generation

LLMs (autoregressive transformers) generate one output token at a time (according to predicted probabilities)

then add it to the context for the prediction of the next token

→ resulting in a full text, sampled from predicted probabilities

How are LLMs Generative Models then?

at each step, take the sequence of previous tokens $x_{<t}$ (context) and predict a conditional probability distribution over the next token x_t:

$$P(x_t|x_{<t})$$

joint probability distribution over text sequences via chain rule of probability:

$$P(x_1, x_2, \cdots, x_T) = \prod_{t=1}^{T} P(x_t|x_{<t})$$

→ no direct prediction of joint probability in one shot, but implicit calculation through product of conditionals

Prompting

feed information into decoder-only transformer via input prompt

→ attention to context

using all the knowledge acquired during internet-scale pretraining and stored in weights

kind of programmable neural network

→ a new paradigm: **one model for many tasks**

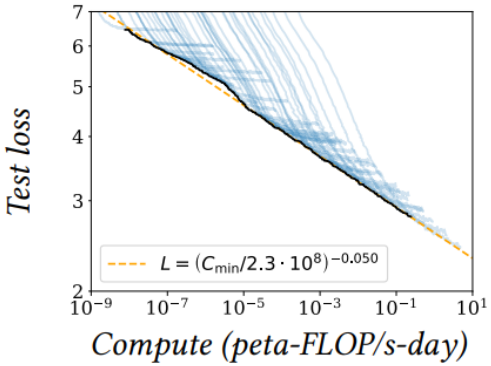
Size Matters: **LARGE** Language Models

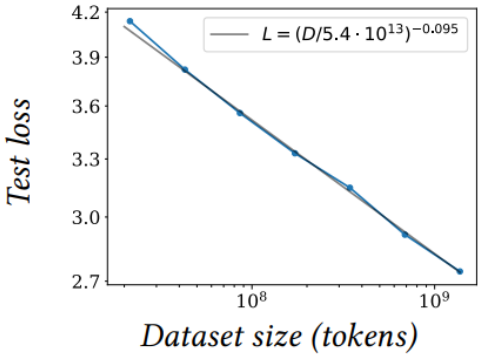
scaling laws, Chinchilla: coupled performance power laws with model size, amount of training data, and compute

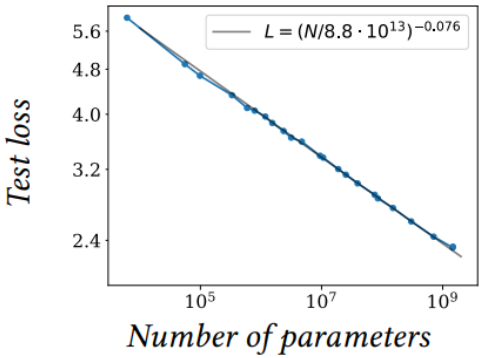
→ era of large-scale models

emergent abilities of LLMs:

- multi-task learning: perform new tasks at test time without task-specific training (via prompting)
- reasoning capabilities







source

Some LLM Numbers

example Kimi K2 (Mixture-of-Experts model):

as usual, comes in base and instruct variants

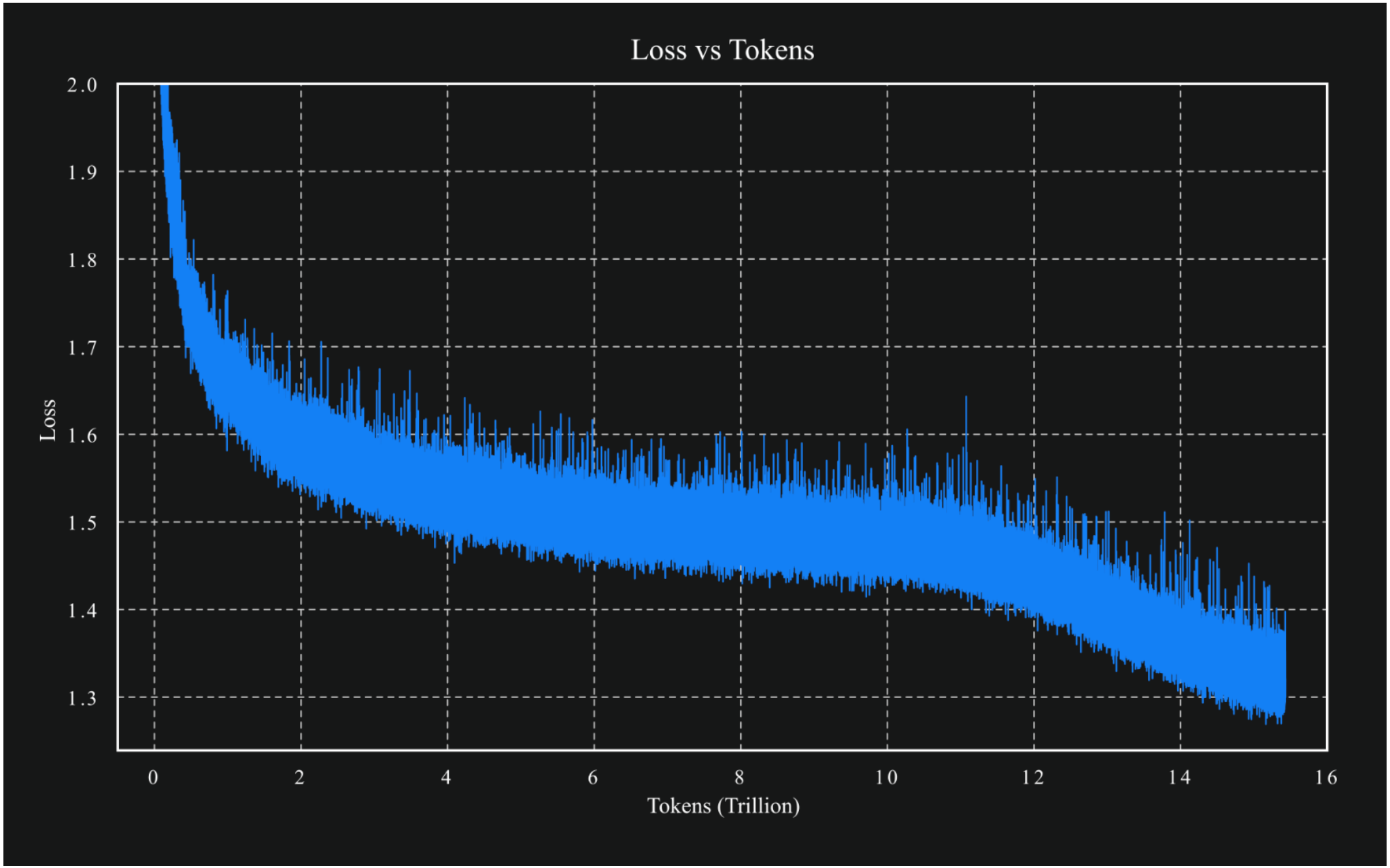
- vocabulary size (tokens): 160K
- embedding dimension: 7,168
- context length (tokens): 128K

- total parameters: 1T
- training tokens: 15.5T

→ a lot of memorizing

compression of the internet

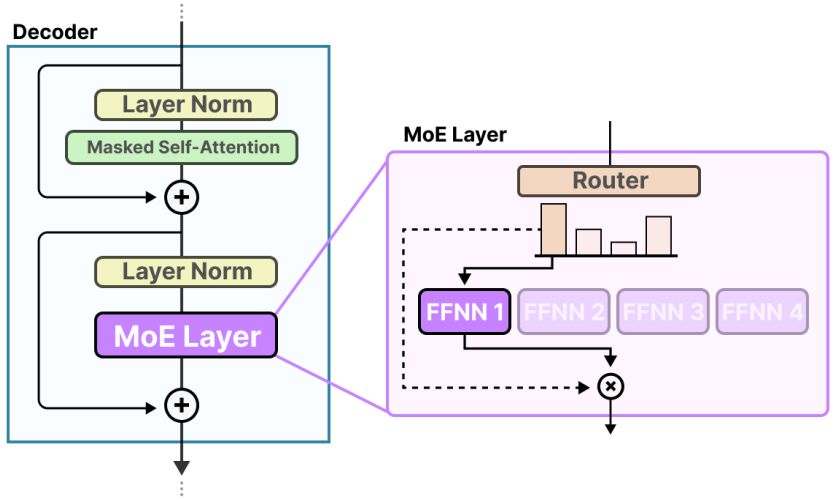
- number of layers: 61
- number of attention heads: 64
- number of experts: 384

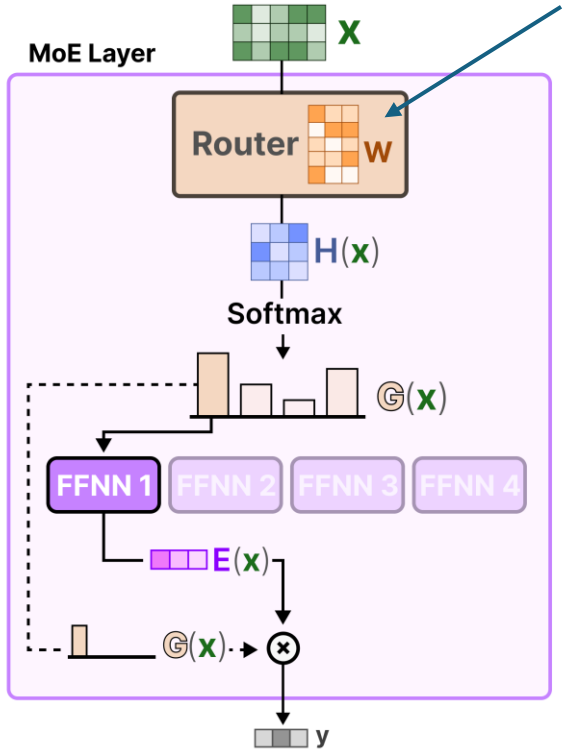

Loss vs Tokens
2.0
1.9
1.8
1.7
1.6
1.5
1.4
1.3
Loss
0
2
4
6
8
10
12
14
16
Tokens (Trillion)

source

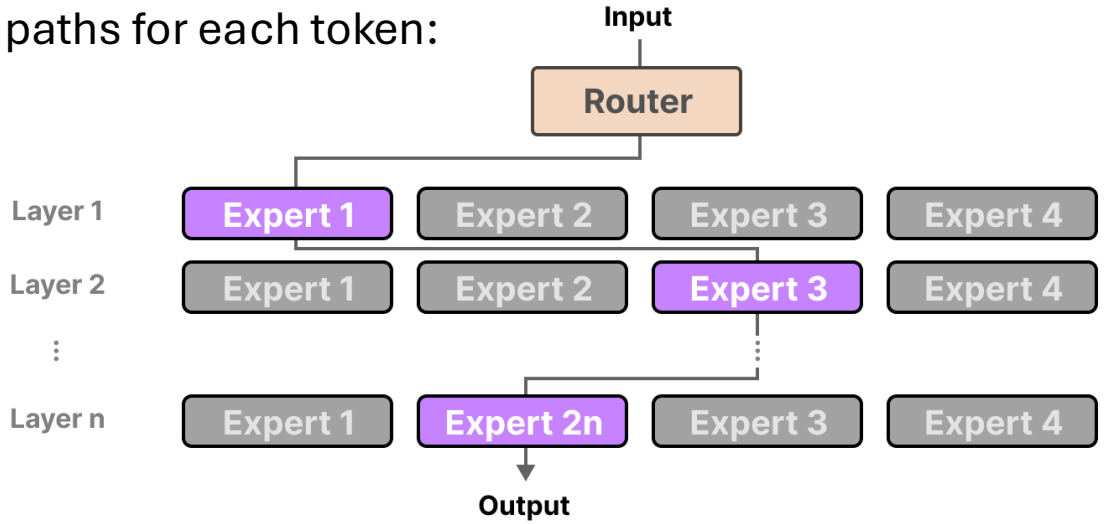
Mixture-of-Experts (MoE)

idea: replace big FFNN with several smaller ones and activate only few
→ less active parameters

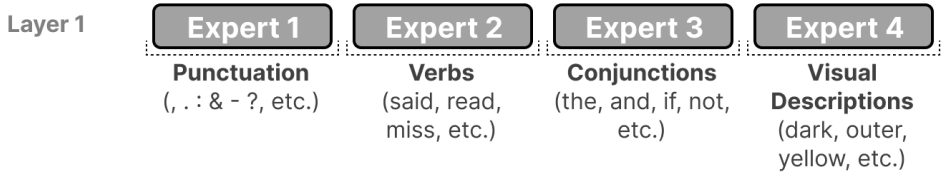


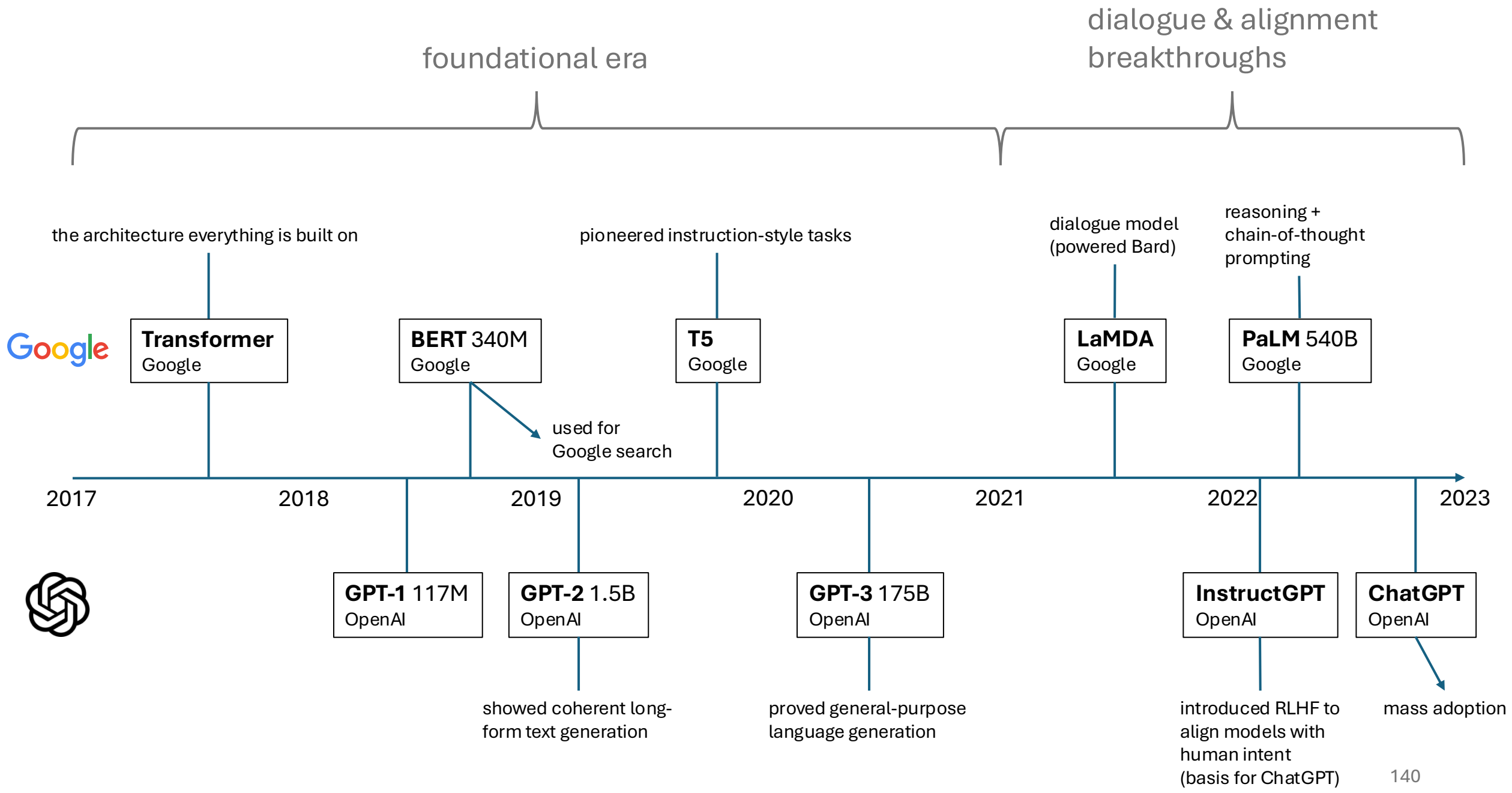


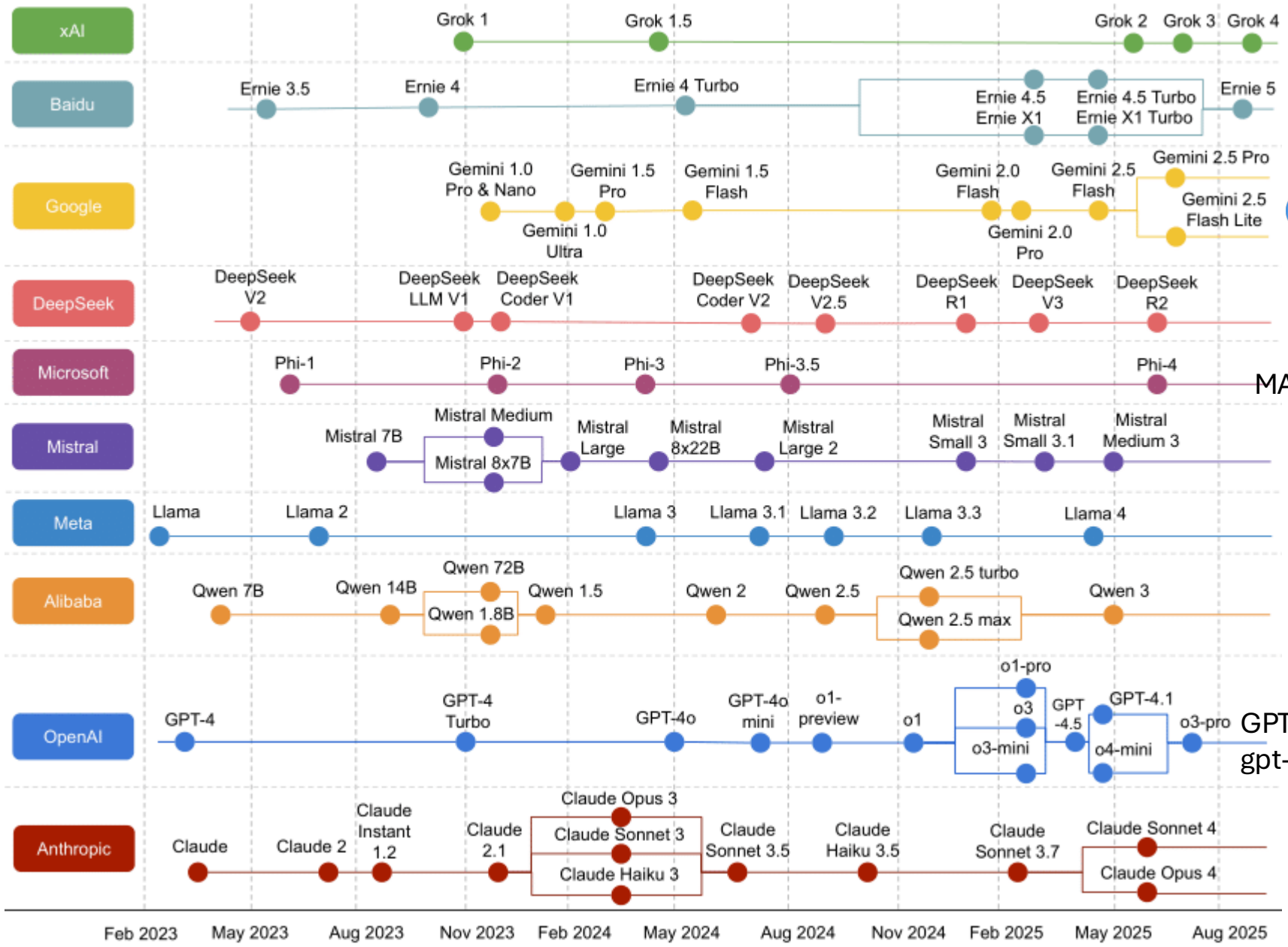
different paths for each token:



example (actually more abstract):




foundational era
dialogue & alignment breakthroughs
the architecture everything is built on
pioneered instruction-style tasks
dialogue model (powered Bard)
reasoning + chain-of-thought prompting
Google
Transformer Google
BERT 340M Google
used for Google search
T5 Google
LaMDA Google
PaLM 540B Google
2017
2018
2019
2020
2021
2022
2023
GPT-1 117M OpenAI
GPT-2 1.5B OpenAI
GPT-3 175B OpenAI
InstructGPT OpenAI
ChatGPT OpenAI
showed coherent long-form text generation
proved general-purpose language generation
introduced RLHF to align models with human intent (basis for ChatGPT)
mass adoption



generative chatbots

Gemini

- evolved from LaMDA & PaLM
- integrated Google ecosystem

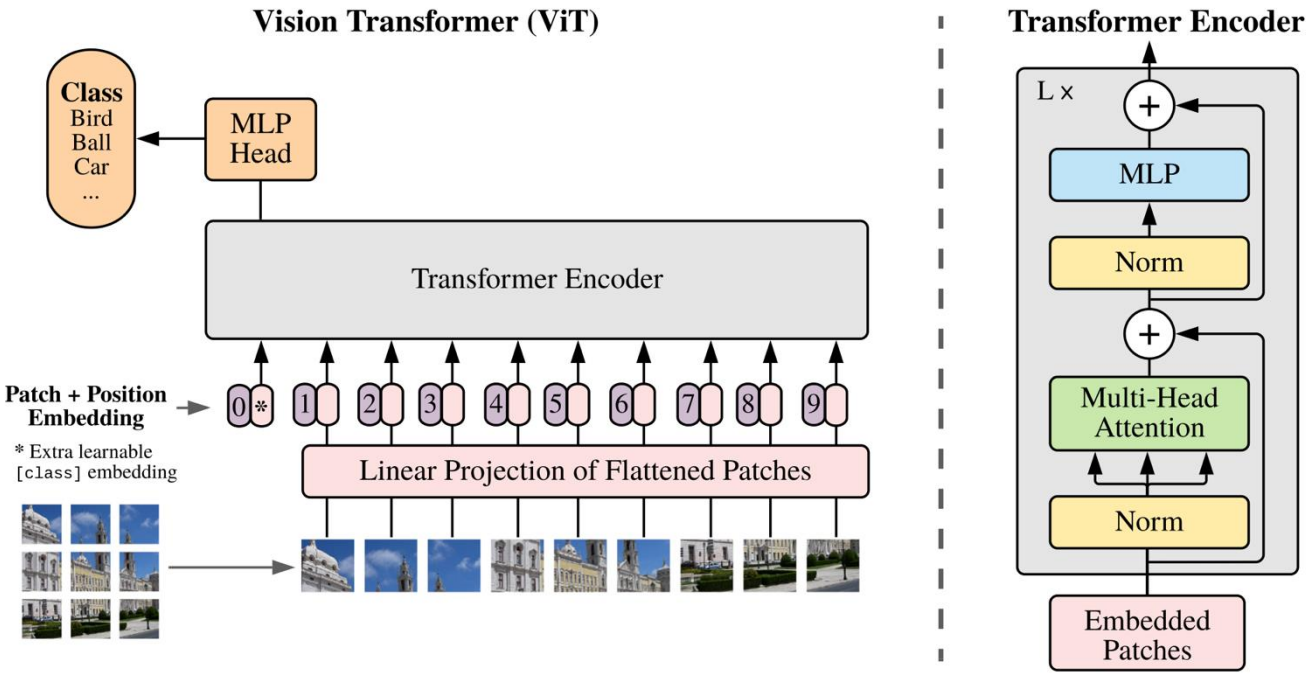
open-source models

ChatGPT

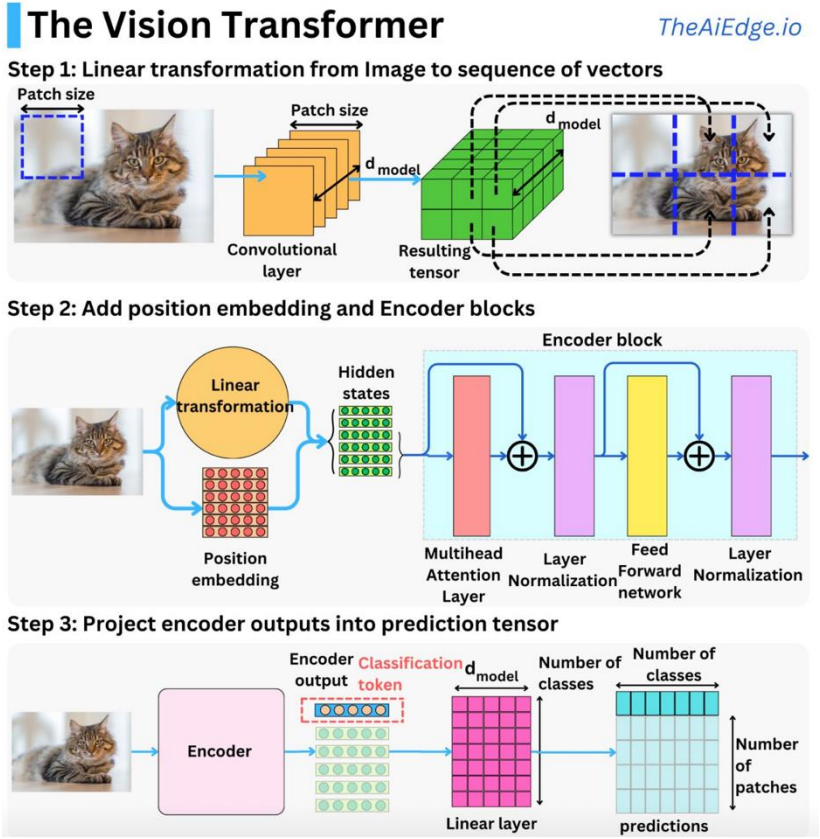
Claude

- different alignment strategy
- tailored for business users

Image Classification with Vision Transformer



source



formulation as sequential problem: split image into patches (tokens) and flatten, add positional embeddings

processing by transformer encoder: pretrain with image labels, fine-tune on specific data set

Attention vs Convolution

fewer inductive biases in ViT than in CNN:

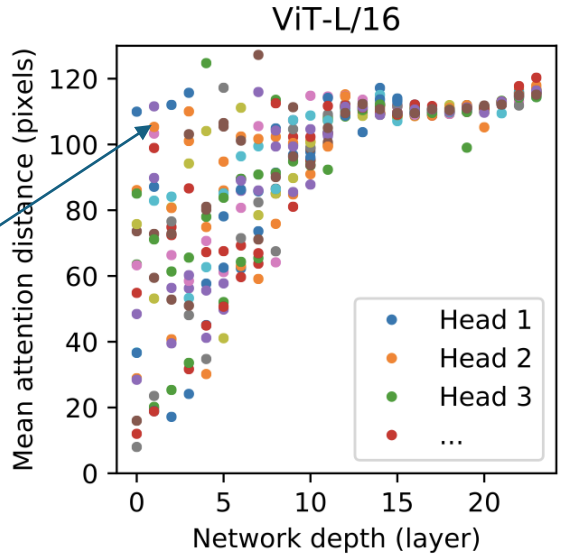
- no translation invariance
- no locally restricted receptive field

Since these are natural for vision tasks, ViTs (conceptionally) learn them from scratch. → ViTs need way more data.

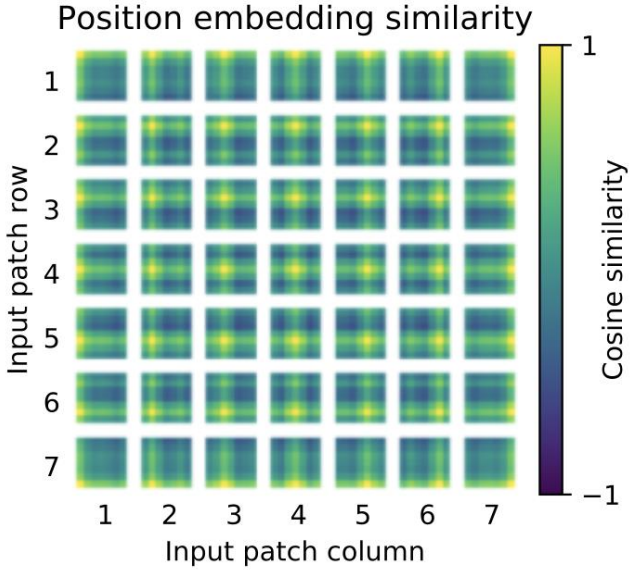
but can lead to beneficial effects (e.g., global attention in lower layers)

see MLP-Mixer results: given enough data, plain multi-layer perceptrons can learn crucial inductive biases

global attention in lower layers (unlike local receptive fields in CNNs)

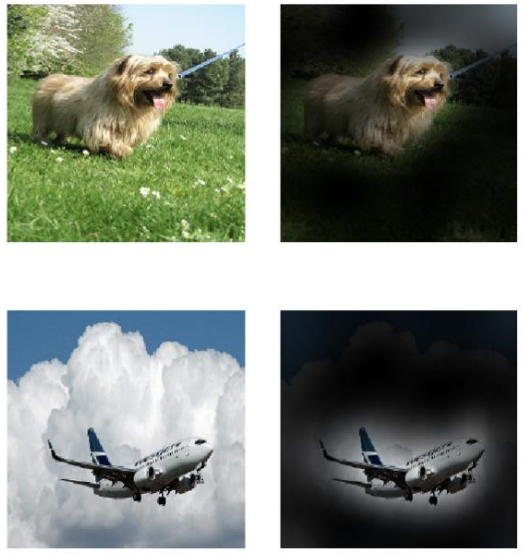


trainable position embedding:



added due to permuation invariance of attention

source



Contrastive Learning

idea: create embedding space in which similar samples are close to each other and dissimilar ones (different images) are far apart

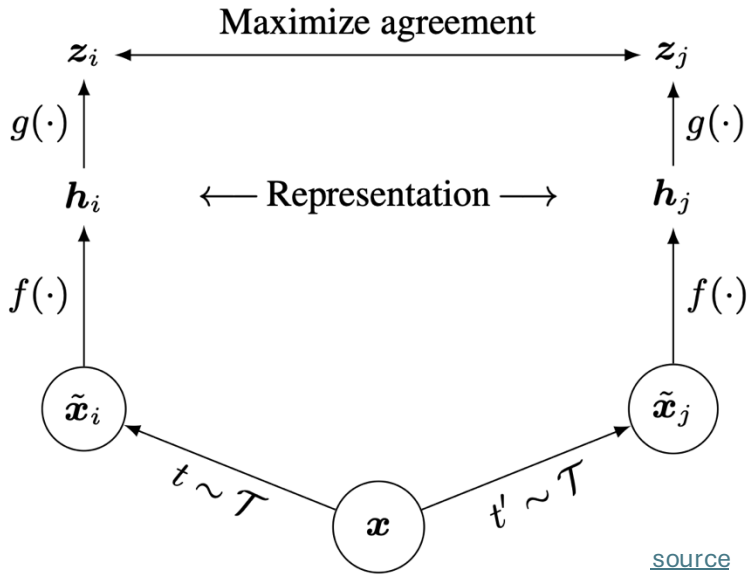
→ learning of image representations in self-supervised way

alternative for supervised setup:

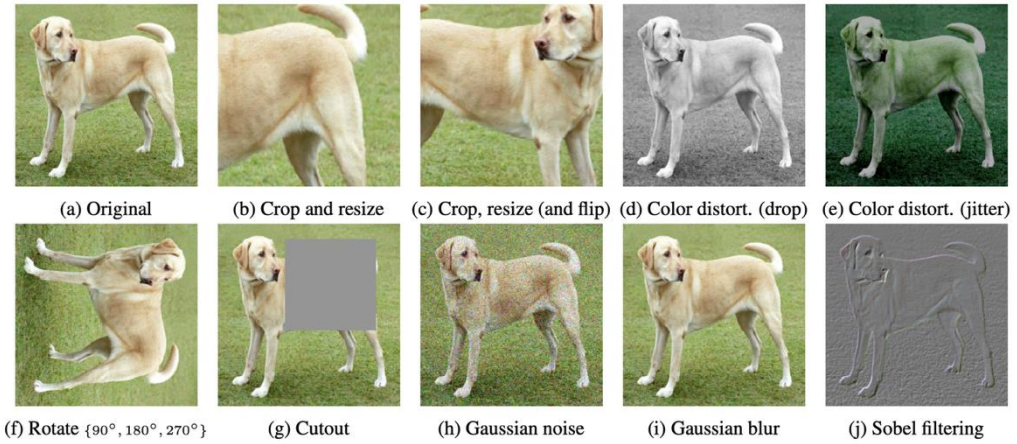
triplet loss (anchor, positive, negative)

general term: metric learning

similar samples:



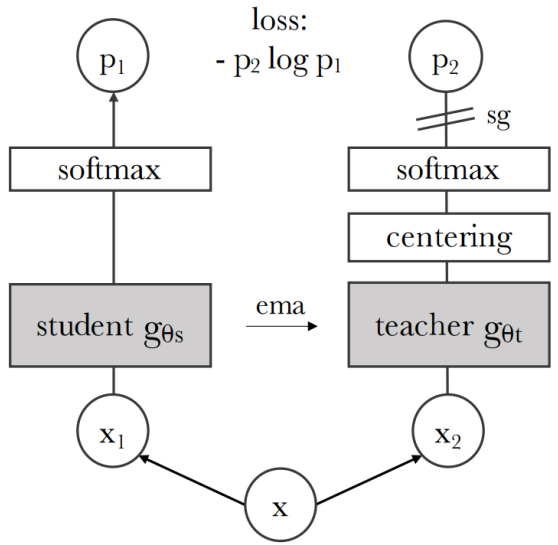
source



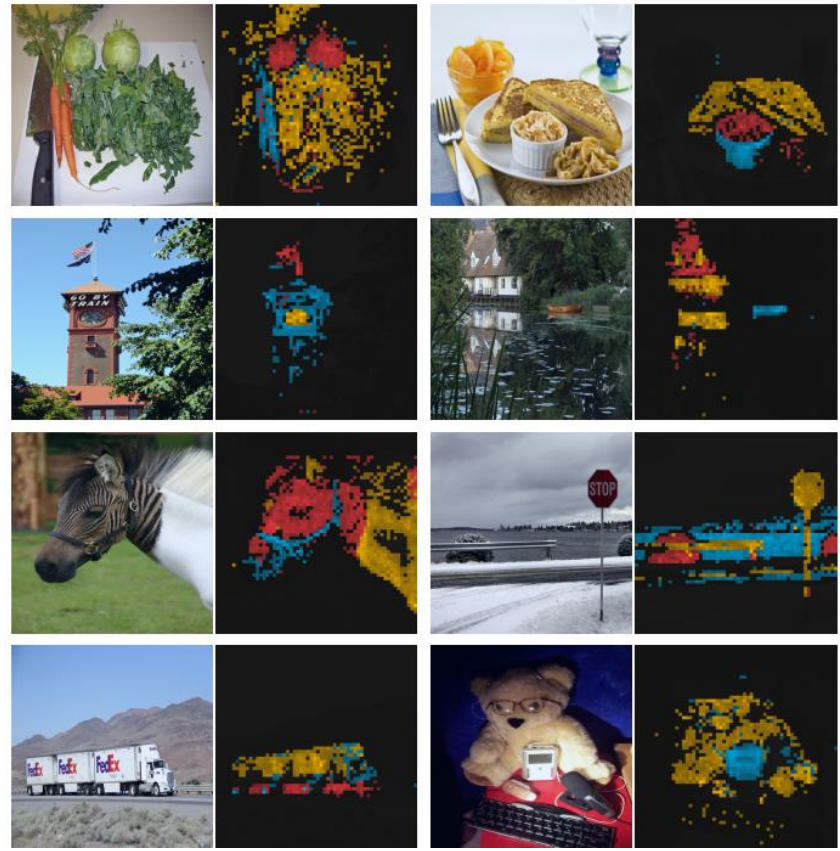
Self-Supervised Vision Transformers

DINO, DINOv2, DINOv3: knowledge **di**stillation with **no** labels (special form of contrastive learning)

foundation model: extraction of self-supervised ViT features



different attention heads:



self-attention of last layer:



Deep Learning for Instance Matching

semantic classifiers (e.g., ResNet) identify categories (e.g., "a car")

instance matching must identify the exact unique object (e.g., "this specific DeLorean") despite shifts in lighting, viewpoint, and occlusion

approach:

1. global embeddings (coarse search): compresses entire image into a single vector (e.g., DINOv2 [CLS] token)

   Enables fast kNN retrieval, but struggles when the target object is small or surrounded by clutter.

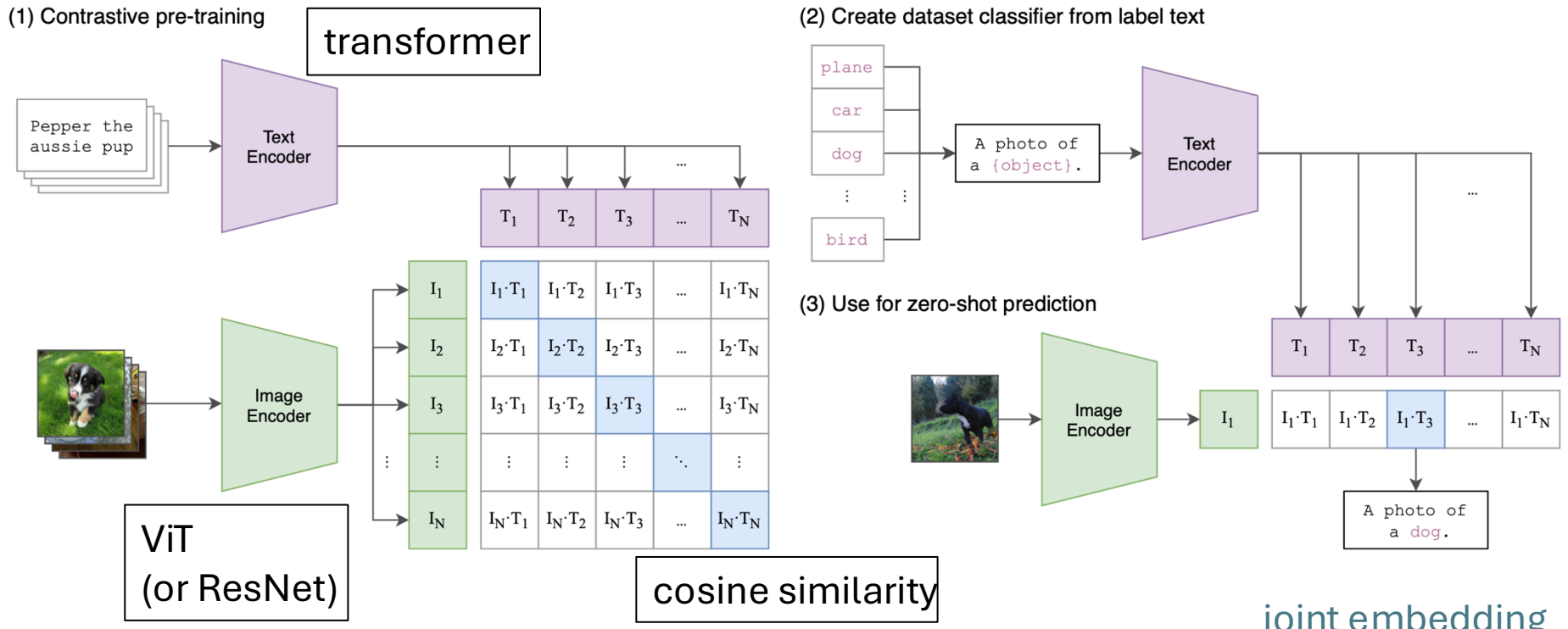
2. local feature matching (fine-grained detail): compares localized regions (e.g., DINOv2 patch tokens or keypoints) to recover fine details lost in global aggregation
   - option A (classical/hybrid pipeline): extract features (DINOv2 patches / SIFT), match via nearest neighbor, verify via RANSAC
   - option B (learned matcher pipeline): extract features, match & verify simultaneously via SuperGlue/LightGlue (learned transformers)
   - option C (end-to-end pipeline): skip explicit feature extraction entirely, direct dense-to-fine matching via LoFTR (*Local Feature Transformer*)

Combination of Vision and Text

example: CLIP (Contrastive Language-Image Pre-training)

learn image representations by predicting which caption goes with which image (pretraining)

→ zero-shot transfer (e.g., for image classification)



joint embedding